INTERVIEWHUB · INTERVIEW PREP

AI, LLM & MLOps for Platform Engineers — InterviewHub

Platform, SRE & DevOps interview guide

Questions	**150 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/ai_interview_guide.html**
Printed	**2026-10-23**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

interviewhub.eknathalabs.co

certs.eknathalabs.com
InterviewHub · interviewhub.eknathalabs

Q1 How does a large language model generate text, and what are tokens and context windows?

LLM basics

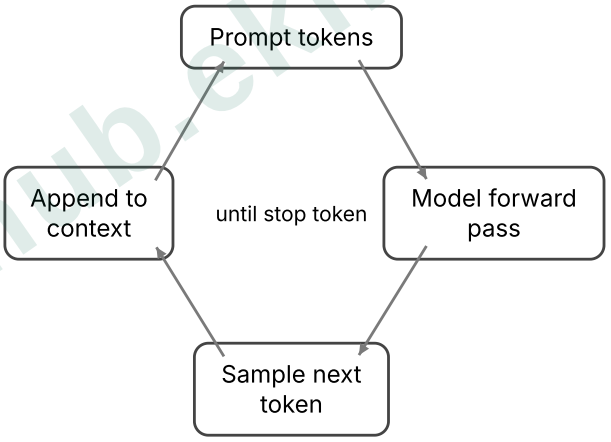
🎙 Spoken answer · ~59 s

A large language model is a neural network, almost always a transformer, trained to predict the next token given the tokens before it. Text is first split into tokens, which are sub-word pieces such as whole words, word fragments or punctuation, so a token is roughly three quarters of an English word. During generation, the model produces a probability distribution over its vocabulary for the next token, one token is chosen by a sampling strategy, appended to the input, and the loop repeats. The context window is the maximum number of tokens the model can attend to at once, covering the prompt, any retrieved documents, the conversation history and the output being generated. Exceeding it means truncating or summarising. For platform work, tokens matter because cost, latency and memory all scale with them, so I measure usage in tokens, not requests.

Likely follow-ups

- Why does output length affect latency more than input length?
- What happens when the context window overflows?

🗺 Autoregressive generation



Transformers predict the next token from a probability distribution, one token at a time. The context window bounds prompt plus output; cost, latency and memory all scale with token counts.

```
python -c "import tiktoken; print(len(tiktoken.get_encoding('cl100k_base').encode('hello world')))"
```

```
curl $API/v1/chat/completions -d '{"max_tokens":256,...}'
```

Interview tip: Say you track cost and latency in tokens, not requests.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Start here (official docs)**

- Hugging Face documentation

Q2 What do temperature, top-p and top-k control, and how do you choose them for production? LLM basics

🎙 **Spoken answer · ~58 s**

They control how the next token is sampled from the model's probability distribution. Temperature rescales the distribution: a low value, near zero, makes the model almost deterministic by favouring the most likely token, while a high value flattens it and increases variety and risk. Top-k restricts sampling to the k most likely tokens, and top-p, also called nucleus sampling, restricts it to the smallest set of tokens whose cumulative probability reaches p. For tasks that need consistency, such as classification, extraction, code generation and tool calls, I use low temperature, around zero to 0.3. For creative writing or brainstorming, higher values around 0.7 to 1.0 help. I usually change either temperature or top-p, not both. Even at temperature zero, outputs can vary slightly because of batching and floating point effects, so I do not rely on exact repeatability without testing.

Likely follow-ups

- Is temperature zero fully deterministic?
- Which parameter would you change for structured output?

Temperature rescales probabilities, top-k and top-p restrict the candidate set. Use low temperature for extraction, classification and tool calls; higher for creative work. Change one knob at a time.

```
curl $API/v1/chat/completions -d '{"temperature":0.2,"top_p":1.0,...}'
```

```
vllm serve model --generation-config vllm
```

Interview tip: Note that temperature zero is not a guarantee of identical output.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q3 What is the difference between pre-training, fine-tuning, instruction tuning and RLHF?

LLM basics

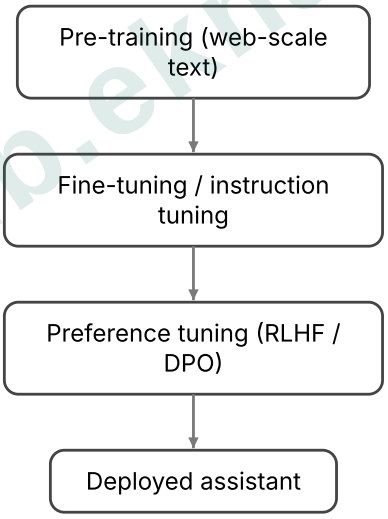
🎙 Spoken answer · ~56 s

They are successive stages that shape a model. Pre-training trains on a huge text corpus to predict the next token, which gives broad language and world knowledge but not helpful behaviour. Fine-tuning continues training on a smaller, task-specific dataset to specialise the model, for example for a domain or a format. Instruction tuning is fine-tuning on pairs of instructions and good responses, so the model learns to follow requests rather than just continue text. Preference-based alignment, such as reinforcement learning from human feedback or newer variants like direct preference optimisation, uses comparisons of better and worse answers to steer the model towards helpful, harmless and honest behaviour. As a platform engineer I rarely pre-train. I choose among prompting, retrieval and light fine-tuning, because they cost far less and are easier to operate and update.

Likely follow-ups

- When is fine-tuning better than RAG?
- What data do you need for instruction tuning?

🗺 Training stages



Pre-training learns language and knowledge, instruction tuning teaches following requests, and preference alignment shapes behaviour. Most teams use prompting, retrieval and light fine-tuning instead of pre-training.

```
huggingface-cli download meta-llama/Llama-3.1-8B-Instruct
```

```
accelerate launch train.py --method sft
```

Interview tip: Say that most production value comes from prompting and retrieval first.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Hugging Face documentation

Q4 What is fine-tuning with LoRA and why is it popular?

LLM basics

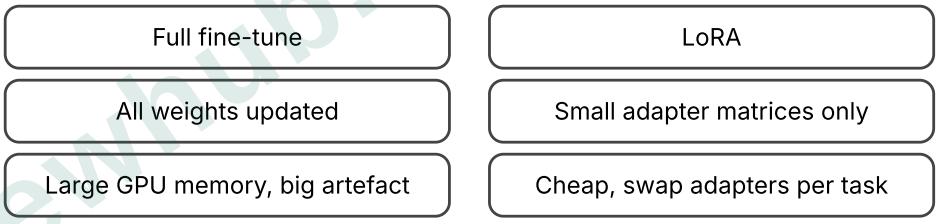
🎙 **Spoken answer · ~58 s**

LoRA, low-rank adaptation, fine-tunes a model by freezing the original weights and training small pairs of low-rank matrices that are added to selected layers, such as attention projections. Because only these adapter matrices are trained, the number of trainable parameters drops by orders of magnitude, memory for optimiser state shrinks, training is faster and cheaper, and the output is a small file, often a few megabytes to hundreds of megabytes, rather than a full model copy. Adapters can be swapped or merged at serving time, so one base model can serve many tenants or tasks with different adapters, which servers like vLLM support. QLoRA goes further by quantising the base model to four bits during training. The trade-off is slightly lower ceiling quality than full fine-tuning for large behavioural changes, but for most domain adaptation it is excellent value.

Likely follow-ups

- What is the rank in LoRA?
- How do you serve many adapters on one GPU?

🗺 **Full fine-tune vs LoRA**



LoRA freezes base weights and trains small low-rank adapters, cutting memory and cost and producing tiny, swappable files. QLoRA adds 4-bit quantisation. Great for per-task or per-tenant adapters.

```
peft_config = LoraConfig(r=16, lora_alpha=32, target_modules=['q_proj','v_proj'])
```

```
vllm serve base-model --enable-lora --lora-modules sql=/adapters/sql
```

Interview tip: Mention serving many adapters over one base model.

📚 **Official docs**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q5 What is quantisation and what are its trade-offs for inference?

LLM basics

🎙 Spoken answer · ~59 s

Quantisation stores and computes model weights, and sometimes activations and the KV cache, in lower precision, for instance 8-bit or 4-bit integers or low-bit floating point, instead of 16-bit. The benefit is a smaller memory footprint, so a model that needs 140 GB at 16-bit might fit in about 35 GB at 4-bit, plus faster inference because decoding is mostly memory-bandwidth bound. Common methods include GPTQ, AWQ and FP8 formats supported on newer GPUs. The cost is some loss of accuracy, usually small at 8-bit and more noticeable at 4-bit or below, particularly for reasoning, maths and long contexts, and kernel support differs by hardware. So I always evaluate the quantised model on my own task benchmark against the full-precision baseline, and choose the lowest precision that meets the quality bar, then confirm the latency and throughput gains in my serving stack.

Likely follow-ups

- Why is decoding memory-bandwidth bound?
- How do you decide between 8-bit and 4-bit?

Lower-precision weights shrink memory and speed up bandwidth-bound decoding at some accuracy cost, mild at 8-bit and larger at 4-bit. Always compare against a baseline on your own evaluation set.

```
vllm serve TheBloke/model-AWQ --quantization awq
```

```
python -m lm_eval --model hf --model_args pretrained=model --tasks hellaswag
```

Interview tip: Say you validate on your task, not on public benchmarks alone.

📚 Official docs

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q6 What is the KV cache and why does it dominate GPU memory at long contexts?

LLM basics

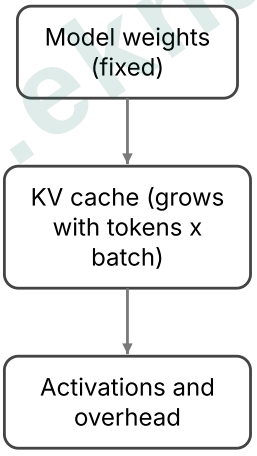
🎙 Spoken answer · ~58 s

During generation, each transformer layer computes keys and values for every token. Instead of recomputing them for all earlier tokens at each step, the server stores them in the KV cache, so each new token only computes its own and attends to the cached ones. This makes decoding fast but memory hungry: the cache grows linearly with sequence length, batch size, number of layers and hidden size, and for long contexts and many concurrent requests it can exceed the size of the model weights. That is why GPU memory, not compute, often limits concurrency. Techniques that help include PagedAttention, which manages the cache in blocks to avoid fragmentation, grouped-query or multi-query attention that shrinks the cache, KV cache quantisation, prefix caching to share common prompts, and offloading to CPU memory. Capacity planning for LLM serving is largely KV cache planning.

Likely follow-ups

- How does grouped-query attention reduce memory?
- What is prefix caching?

🗺 GPU memory at long context



The KV cache stores keys and values of past tokens so decoding is fast, but it grows with sequence length and batch size and often outweighs the weights. PagedAttention, GQA, quantisation and prefix caching help.

```
vllm serve model --gpu-memory-utilization 0.9 --max-model-len 16384
```

```
vllm serve model --enable-prefix-caching
```

Interview tip: Say capacity planning for LLMs is really KV cache planning.

📚 Official docs

- Kubernetes: Scheduling GPUs

certs.eknathalabs.com

InterviewHub - interviewhub.eknathalabs.com

Q7 Explain the difference between the prefill and decode phases of inference.

LLM basics

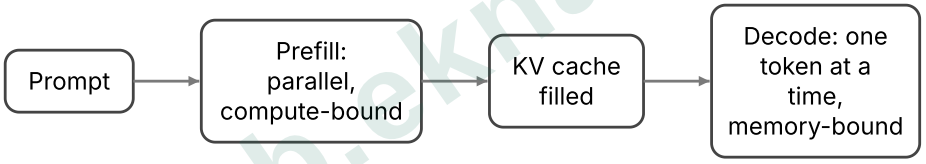
🎙 Spoken answer · ~62 s

Inference has two phases with different behaviour. Prefill processes the whole input prompt at once, computing the keys and values for all prompt tokens in parallel, which is compute bound and determines the time to first token. Decode then generates output tokens one at a time, each requiring a pass through the model while reading all the weights and the KV cache, which is memory-bandwidth bound and determines the inter-token latency. A long prompt makes prefill expensive, and a long answer makes decode expensive. This split matters operationally: you can optimise them separately, for example with chunked prefill so a long prompt does not stall other users' decoding, or by disaggregating them onto different GPU pools, one sized for compute and another for bandwidth. When I see a latency complaint I ask whether it is time to first token or the streaming speed, because the causes and fixes differ.

Likely follow-ups

- What is chunked prefill?
- Which metric reflects prefill cost?

🗺 Prefill vs decode



Prefill processes the prompt in parallel and is compute bound, defining time to first token. Decode generates token by token and is bandwidth bound, defining inter-token latency. Optimise and size them separately.

```
curl -w '%{time_starttransfer}\n' -o /dev/null -s $API/v1/chat/completions -d @req.json
```

```
vllm serve model --enable-chunked-prefill
```

Interview tip: Ask whether the complaint is time to first token or streaming speed.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q8 What are embeddings and how are they used?

LLM basics

🎙 Spoken answer · ~58 s

An embedding is a fixed-length vector of numbers that represents the meaning of a piece of text, an image or other data, produced by an embedding model so that similar items end up close together in the vector space. Closeness is measured with cosine similarity or dot product. This lets you search by meaning instead of exact keywords: embed your documents once and store the vectors, embed the user's query at request time, and retrieve the nearest vectors. Embeddings power semantic search, retrieval for RAG, clustering, deduplication, recommendations and classification. Important operational points are that you must use the same embedding model for indexing and querying, that changing the model means re-embedding everything, that dimensions affect storage and speed, and that quality depends on the model and chunking, so you should evaluate retrieval quality on your own queries.

Likely follow-ups

- Why must index and query use the same model?
- How do you pick the vector dimension?

Embeddings are vectors where semantically similar items are close. They enable semantic search and RAG. Index and query must use the same model, and changing it means re-embedding everything.

```
curl $API/v1/embeddings -d '{"model":"text-embedding-model","input":"how do I reset my password"}'
```

```
SELECT id FROM docs ORDER BY embedding <=> $1 LIMIT 5;
```

Interview tip: Mention the re-embedding cost of changing models.

📚 Official docs

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q9 What causes hallucinations in language models and how do you reduce them?

LLM basics

🎙 **Spoken answer · ~58 s**

A language model produces plausible text, not verified facts, so when it lacks knowledge, has ambiguous context, or is pushed to answer, it can generate confident but wrong statements. Contributing factors are gaps or errors in training data, outdated knowledge, over-long or noisy context, and prompts that demand an answer even when none is known. I reduce hallucination in layers. Ground answers in retrieved sources with RAG and require citations. Instruct the model to say it does not know when the context lacks the answer. Use low temperature, structured output and constrained formats. Add verification steps, such as a second model or rule that checks claims against sources, and validate tool outputs in code. For high-risk domains add human review. Finally, measure the rate with an evaluation set of tricky questions, because you cannot manage what you do not measure.

Likely follow-ups

- How would you measure faithfulness?
- When is human review required?

Models generate plausible text, not verified facts. Ground with RAG and citations, allow I-don't-know, lower temperature, validate outputs in code, add review for high-risk uses, and measure with an evaluation set.

```
ragas evaluate --metrics faithfulness answer_relevancy
```

```
grep -c 'I don.t know' answers.jsonl
```

Interview tip: Say you measure hallucination rate rather than assume it is solved.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q10 Compare using a hosted API model with self-hosting an open-weight model. LLM basics

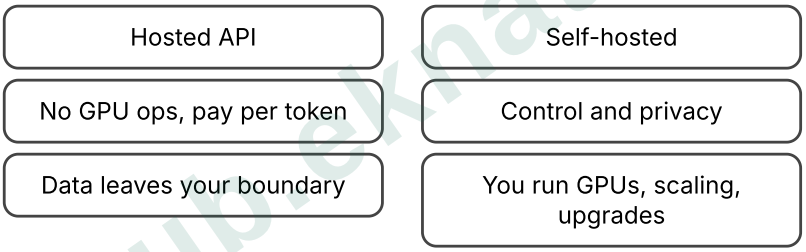
🎙 Spoken answer · ~58 s

A hosted API gives instant access to frontier-quality models, no GPU operations, automatic scaling and updates, and pay-per-token pricing, so it is the fastest way to start and often cheapest at low or spiky volume. The downsides are data leaving your boundary, vendor dependency, rate limits, changing model versions and cost that grows linearly with usage. Self-hosting open-weight models gives control over data residency, model versions, fine-tuning, latency and customisation, and at high steady volume the cost per token can be lower. But it brings real work: GPU capacity planning, serving stack operations, upgrades, monitoring, and usually somewhat lower capability than the best proprietary models. I decide on data sensitivity, volume, quality requirements and team skills, often starting with an API behind an abstraction layer so I can switch models, and self-hosting where privacy or economics justify it.

Likely follow-ups

- What is an LLM gateway?
- At what volume does self-hosting pay off?

🗺 Hosted API vs self-hosted



APIs offer top quality and zero ops but cost scales linearly and data leaves your boundary. Self-hosting gives control and lower marginal cost at steady volume but needs GPU operations. Abstract behind a gateway.

```
litellm --model gpt-4o --port 4000
```

```
vllm serve open-model --port 8000
```

Interview tip: Recommend a gateway so the choice stays reversible.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q11 What is a mixture-of-experts model and what does it mean for serving? LLM basics

🎙 Spoken answer · ~61 s

In a mixture-of-experts model, each transformer feed-forward block is replaced by several expert networks and a small router that picks a few experts, usually two, for each token. The model therefore has a very large total parameter count but only a fraction of the parameters are active per token, so it can reach strong quality with lower compute per token than a dense model of the same total size. For serving, the implications are different from dense models. All experts must still be held in memory, so the memory footprint matches the total size, which often requires several GPUs, and the experts are commonly spread across GPUs with expert parallelism. Routing creates uneven load and extra communication, so networking bandwidth and batching matter. Throughput can be excellent at high batch sizes, but latency at small batches may not improve as much as the active parameter count suggests.

Likely follow-ups

- Why does memory footprint not shrink?
- What is expert parallelism?

MoE models route each token to a few experts so compute per token is low, but all experts must sit in memory. Serving needs several GPUs, expert parallelism and good interconnects.

```
vllm serve mixtral-model --tensor-parallel-size 4
```

```
nvidia-smi topo -m
```

Interview tip: State clearly: compute is sparse, memory is not.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q12 What are context length limits and how do you handle documents that do not fit? LLM basics

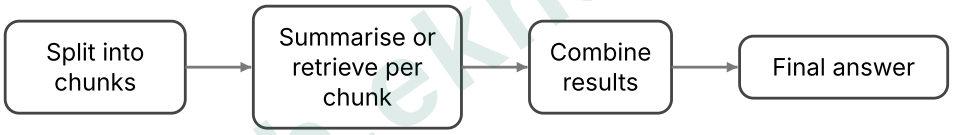
🎙 Spoken answer · ~58 s

Every model has a maximum context, and even within it, quality can degrade when the prompt is very long, with details in the middle being missed, and cost and latency rise with every token. For documents that do not fit, I choose among a few approaches. Retrieval: split the content into chunks, index them, and put only the relevant passages in the prompt, which is the standard RAG pattern. Map-reduce summarisation: summarise each part, then summarise the summaries. Hierarchical or iterative refinement for tasks that need the whole document. Compression: remove boilerplate, use structured extraction first, or trim conversation history by summarising old turns. Long-context models can reduce engineering effort but cost more per request and do not remove the need to test accuracy. I pick based on whether the task needs the whole document or just specific facts.

Likely follow-ups

- What is the lost in the middle problem?
- When is a long-context model preferable to RAG?

🗺 Long documents



Use retrieval for fact lookup, map-reduce or iterative summarisation for whole-document tasks, compress conversation history, and test accuracy because long contexts cost more and can miss details in the middle.

```
python chunk.py --size 800 --overlap 100 doc.pdf
```

```
vllm serve model --max-model-len 32768
```

Interview tip: Match the technique to whether you need facts or the whole document.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q13 How do you version and manage prompts, models and configuration as production assets?

LLM basics

🎙 Spoken answer · ~57 s

I treat prompts, model identifiers and generation parameters as code. They live in version control, are reviewed through pull requests, and are tied to a release, so any answer in production can be traced to the exact prompt template, model version and settings that produced it. Prompts are separated from application logic as templates with variables, stored in files or a prompt registry, and tested against an evaluation set in CI before merging, so regression in quality blocks the change. Model versions are pinned, never left on a moving alias, because providers update models and behaviour shifts. Rollout uses feature flags or canary traffic with comparison metrics, and rollback is a configuration change. I log the prompt version, model, parameters and a trace ID with each request, subject to privacy rules, which makes debugging and audits possible.

Likely follow-ups

- Why pin model versions?
- What belongs in the evaluation gate?

Prompts, model versions and parameters live in Git, reviewed and tested against an evaluation set in CI, pinned, rolled out with flags or canaries, and logged with each request for traceability.

```
git log --oneline -- prompts/support_v3.yaml
```

```
promptfoo eval -c promptfooconfig.yaml
```

```
kubectl set env deploy/api MODEL_ID=model-2026-06
```

Interview tip: Pin the model; moving aliases silently change behaviour.

📚 Official docs

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q14 What is a small language model and when is it the right choice?

LLM basics

🎙 Spoken answer · ~61 s

A small language model has far fewer parameters than frontier models, typically from a few hundred million to a few billion, and is designed or distilled to run cheaply. Its advantages are low latency, small memory, the ability to run on a single modest GPU, a CPU or even a phone, much lower cost per request, easier fine-tuning, and the option to keep data fully on premises or on the device. It is the right choice for narrow, well-defined tasks such as classification, routing, extraction, summarising short text, autocomplete, and as a first-pass filter in front of a larger model. It is weaker at open-ended reasoning, broad knowledge and complex multi-step tasks. A practical pattern is a cascade or router: send simple requests to a small model and escalate hard ones to a larger model, with evaluation to ensure the quality stays acceptable while cost drops.

Likely follow-ups

- How would you build a model router?
- How do you decide the escalation rule?

Small models offer low cost, latency and on-prem or edge deployment, ideal for narrow tasks like classification and extraction. Use a router to escalate hard requests to a larger model.

```
ollama run small-model
```

```
vllm serve small-model --max-model-len 4096
```

Interview tip: Propose a cascade; it shows cost-awareness.

📚 Start here (official docs)

- Hugging Face documentation

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q15 What makes a good production prompt? Describe a structure you would use. Prompting

🎙 Spoken answer · ~60 s

A good prompt is clear, specific and testable. I use a consistent structure. First, the role and objective, stating what the assistant is for. Second, the context, with only the information needed, clearly delimited so it is not confused with instructions. Third, the task and constraints, including what to do when information is missing. Fourth, the output format, ideally a strict schema such as JSON with named fields. Fifth, a few examples of inputs and desired outputs for tricky cases. I put stable instructions at the top, which also helps prompt caching, and variable data after. I avoid vague words like good or short and give measurable limits. Then I test the prompt on a representative evaluation set, iterate on failures, and record changes in version control. A prompt is not done when it works once; it is done when it passes the evaluation reliably.

Likely follow-ups

- Why delimit retrieved context?
- How many examples are enough?

Role and objective, delimited context, task with constraints and missing-info behaviour, strict output format, and few-shot examples. Put stable text first, test on an evaluation set and version it.

```
cat prompts/classify.yaml
```

```
promptfoo eval --prompts prompts/classify.yaml --tests tests.csv
```

Interview tip: Emphasise testing on an evaluation set, not one-off success.

📚 Official docs

- Prompt engineering overview (Claude docs)

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q16 Explain zero-shot, few-shot and chain-of-thought prompting and when to use each.

Prompting

🎙 Spoken answer · ~60 s

Zero-shot gives only an instruction and relies on what the model already knows. It is the cheapest option and fine for straightforward tasks. Few-shot adds several worked examples of input and output in the prompt, which teaches the format, tone and edge cases, and noticeably improves consistency when the task is unusual or the output structure is strict. The cost is extra tokens on every request. Chain-of-thought asks the model to reason step by step before answering, which helps on multi-step problems such as maths, planning and logic, although many modern reasoning models do this internally and may not need to be told. The trade-offs are longer outputs, higher latency and cost, and the risk of exposing reasoning you may not want to show users. I start zero-shot, add examples when failures show a pattern, and use reasoning only where accuracy gains justify the cost.

Likely follow-ups

- How do you pick good few-shot examples?
- Do reasoning models still need chain-of-thought prompts?

Zero-shot is cheapest, few-shot fixes format and edge cases at a token cost, and chain-of-thought improves multi-step reasoning at higher latency. Start simple and add only when evaluation shows a need.

```
messages=[{'role':'system','content':'Classify tickets...'},
{'role':'user','content':'Example 1...'}]
```

Interview tip: Say you add complexity only when evaluation failures justify it.

📚 Official docs

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q17 How do you get reliable structured output such as JSON from a model?

Prompting

🎙 Spoken answer · ~57 s

I combine several techniques. First, specify the exact schema in the prompt with field names, types and an example, and say that the response must contain only that JSON. Second, use the provider's structured output or JSON mode, or function calling with a schema, where available, which constrains decoding so the output is valid. For self-hosted models, constrained decoding with a grammar or JSON schema, supported by servers like vLLM through guided decoding, guarantees syntax. Third, validate every response in code with a schema validator such as Pydantic or JSON Schema, and on failure retry once with the error message, or fall back to a safe default. Fourth, keep the schema simple and the temperature low. Syntax validity does not guarantee correct values, so I also check business rules and log failures to improve the prompt.

Likely follow-ups

- What does constrained decoding guarantee?
- How do you handle repeated validation failures?

Give an explicit schema, use structured output or function calling or constrained decoding, validate in code with retry and fallback, keep temperature low. Valid syntax is not the same as correct values.

```
response_format={'type':'json_schema','json_schema':schema}
```

```
vllm serve model --guided-decoding-backend xgrammar
```

```
Model.model_validate_json(raw)
```

Interview tip: Say you validate in code regardless of how well the prompt works.

📚 Official docs

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q18 What is prompt injection and how do you defend against it?

Prompting

🎙 **Spoken answer · ~59 s**

Prompt injection is an attack where untrusted text, from a user or from retrieved content such as a web page, email or document, contains instructions that override or hijack the developer's intended behaviour. Direct injection comes in the user input, and indirect injection hides in data the model reads. There is no single complete fix, so I use layers. Treat all model input as untrusted and keep system instructions separate from data with clear delimiters. Give the model the least privilege: restrict tools, scope credentials, require confirmation for sensitive actions, and never let model output run as code or SQL unchecked. Filter and validate inputs and outputs, including for data exfiltration through links or markdown images. Isolate high-risk workflows, monitor and log, and red-team regularly. The core principle is to design so that even a fully manipulated model cannot cause serious harm.

Likely follow-ups

- What is indirect prompt injection?
- How does least privilege apply to agents?

Untrusted text can carry instructions. Defend in layers: separate instructions from data, least-privilege tools, human confirmation for sensitive actions, validate inputs and outputs, monitor, and red-team. Assume the model can be fooled.

```
garak --model_name my-model --probes promptinject
```

```
grep -E 'ignore (all|previous) instructions' app.log
```

Interview tip: Say design so a manipulated model still cannot do serious harm.

📚 **Official docs**

- OWASP Top Ten
- OWASP Top 10 for LLM Applications
- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q19 How do system prompts, user prompts and conversation history interact, and how do you manage long conversations?

Prompting

🎙 **Spoken answer · ~60 s**

The system prompt sets the persistent behaviour, role, policies and format, and models give it higher priority than user messages, though that priority is not a security guarantee. User messages carry requests, and the history is the list of earlier user and assistant turns that gives continuity. Because the model is stateless, the application must send the relevant history on every request, so cost and latency grow with each turn. To manage long conversations I use a sliding window of recent turns, summarise older turns into a compact memory, store key facts separately, such as user preferences, and retrieve them when needed. I keep the system prompt stable at the start to benefit from prompt caching. I also set limits on history size, trim tool outputs that are large, and test that summarisation does not lose important details such as constraints the user stated earlier.

Likely follow-ups

- Why is the model stateless?
- How does prompt caching help chat?

The system prompt sets persistent behaviour, messages carry requests and the app resends history each time. Control growth with a sliding window, summaries and separate fact memory, and keep the system prompt stable for caching.

```
messages=[system]+summary+last_turns(6)
```

```
curl $API/v1/chat/completions -d @conversation.json
```

Interview tip: Remember: stateless model means history costs tokens every turn.

📚 **Official docs**

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q20 What is prompt caching and how can you design prompts to benefit from it? Prompting

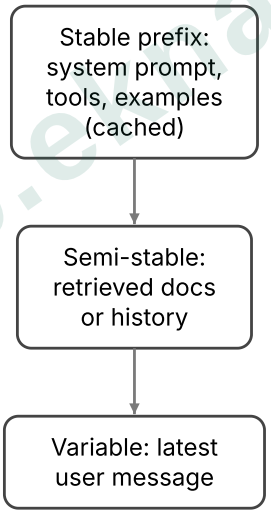
🎙 Spoken answer · ~60 s

Prompt caching lets the serving system reuse the computed attention state, the KV cache, for a prompt prefix it has seen before, so a repeated prefix does not have to be processed again. This reduces latency, particularly time to first token, and many hosted providers also charge less for cached input tokens. Self-hosted servers such as vLLM offer automatic prefix caching. To benefit, put static content first and variable content last: the system prompt, tool definitions, policy text, few-shot examples and long reference documents come at the beginning in a fixed order, and the user's question and dynamic data at the end. Any change early in the prompt, even a timestamp or a reordered list, invalidates the cache from that point. I also keep tool schemas stable and avoid inserting per-request identifiers near the top. Measuring the cache hit rate shows whether the design works.

Likely follow-ups

- What invalidates a cached prefix?
- How do you measure hit rate?

🗺 Prompt layout for caching



Caching reuses the KV state of a repeated prompt prefix, cutting latency and often cost. Put static instructions, tools and reference text first and dynamic content last; avoid timestamps near the top; track hit rate.

```
vllm serve model --enable-prefix-caching
```

```
curl -s localhost:8000/metrics | grep prefix_cache
```

Interview tip: Say static first, dynamic last, and watch the hit rate.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Prompt engineering overview (Claude docs)

Q21 How do you test and iterate on prompts systematically?

Prompting

🎙 **Spoken answer · ~58 s**

I treat it like software testing. I build an evaluation set of realistic inputs with expected outcomes or grading criteria, including easy cases, edge cases and known failures, and I keep growing it from production logs and user feedback. Each prompt change runs against this set automatically, and I compare metrics such as accuracy, format validity, refusal behaviour, length, latency and cost, against the current baseline. For subjective qualities, I use rubric-based grading, sometimes with a stronger model as judge, calibrated against human review. I change one thing at a time so I know what caused an improvement, and I watch for regressions on previously passing cases. Results are stored with the prompt version. Online, I use A/B tests or shadow traffic before full rollout. This discipline turns prompt work from guesswork into engineering with a feedback loop.

Likely follow-ups

- What are the risks of an LLM judge?
- How large should the evaluation set be?

Maintain an evaluation set from real inputs and failures, run it automatically on each change, compare to a baseline across quality, format, cost and latency, change one variable at a time, and canary online.

```
promptfoo eval --output results.json
```

```
python eval.py --prompt v4 --baseline v3
```

```
git tag prompt-v4
```

Interview tip: Say the evaluation set grows from production failures.

📚 **Official docs**

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q22 What is function calling or tool use, and how does it work end to end?

Prompting

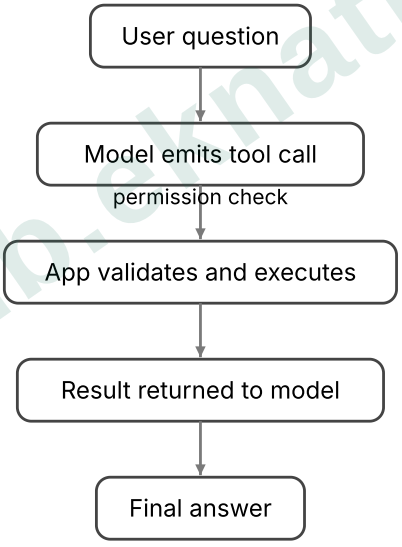
🎙 Spoken answer · ~56 s

Function calling lets the model request that your application run an action instead of only producing text. You describe each tool to the model with a name, description and a JSON schema for its arguments. When the model decides a tool is needed, it returns a structured call, such as the name get_weather with the argument city. Your code, not the model, validates the arguments, executes the function, and sends the result back as a tool message. The model then continues, either calling another tool or writing the final answer for the user. The model never executes anything itself, which is why the application is the security boundary: it must check permissions, validate inputs, limit rates, handle errors and timeouts, and log each call. Good tool descriptions and narrow, well-named functions greatly improve selection accuracy.

Likely follow-ups

- Where does the security boundary sit?
- How do you handle a tool that fails?

🗺 Tool call end to end



You describe tools with JSON schemas, the model returns a structured call, your code validates and executes it and returns the result, and the model continues. The application, not the model, enforces security.

```
tools=[{'type':'function','function':{'name':'get_weather','parameters':{...}}}]
```

```
tool_msg={'role':'tool','tool_call_id':id,'content':json.dumps(result)}
```

Interview tip: State plainly that the model never executes anything.

📚 Official docs

- Model Context Protocol

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q23 How do you control cost and latency when prompts grow large?

Prompting

🎙 **Spoken answer · ~58 s**

I attack the number of tokens and the way they are processed. Reduce input: remove redundant instructions, trim retrieved context to the top few relevant chunks, summarise history, and avoid sending large tool outputs or whole documents. Reduce output: ask for concise structured responses and set max tokens. Use caching: stable prefixes for prompt caching, and response caching for repeated identical or semantically similar queries. Choose model size by task, with a small model for simple steps and a larger one only where needed. Use streaming so users see output sooner, and parallelise independent calls. Batch offline work where latency does not matter, which is cheaper on many providers. Finally, measure tokens per request and per feature, set budgets and alerts, and review the highest-cost prompts regularly, since a small prompt tweak can save a large amount at scale.

Likely follow-ups

- When does semantic caching backfire?
- How do you set a per-feature budget?

Trim input and history, constrain output, use prompt and response caching, route tasks to right-sized models, stream, parallelise, batch offline work, and measure tokens per feature with budgets and alerts.

```
curl -s localhost:8000/metrics | grep -E 'prompt_tokens|generation_tokens'
```

```
redis-cli GET resp:$(sha256sum <<< "$PROMPT")
```

Interview tip: Name token metering per feature as the starting point.

📚 **Official docs**

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q24 What are guardrails and where do you place them in an LLM application?

Prompting

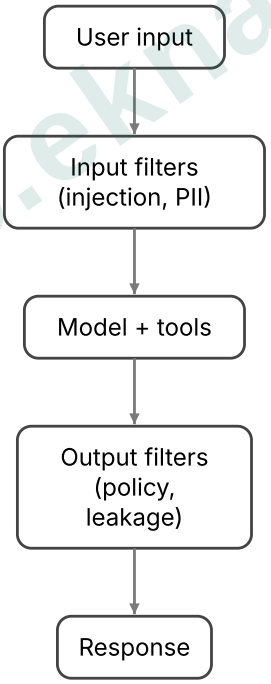
🎙 Spoken answer · ~58 s

Guardrails are checks and constraints that keep an application's inputs and outputs within acceptable bounds. I place them at several points. On input: detect and block prompt injection patterns, strip or mask sensitive data such as personal information, enforce length limits, and classify off-topic or disallowed requests. In the model call: a strong system prompt, restricted tools and constrained output schemas. On output: validate the format, scan for personal data, secrets and unsafe content, check that claims are supported by the retrieved sources, and refuse or rewrite when a policy is violated. Around actions: allow lists, permissions, rate limits and human approval for risky operations. Tools range from simple regexes and classifiers to dedicated frameworks, and the checks add latency and can create false positives, so I measure both safety catches and user friction, and tune thresholds with real traffic.

Likely follow-ups

- What are the costs of too many guardrails?
- How do you test guardrails?

🗺 Guardrail placement



Layered checks on input, around the model call, on output and around actions: injection and PII filters, schema validation, grounding and safety checks, allow lists and human approval. Measure catches and false positives.

```
nemoguardrails chat --config=./config
```

```
python guard.py --check pii --input msg.txt
```

InterviewHub - interviewhub.eknathalabs.com

Interview tip: Mention the false-positive cost to show balance.

📚 Official docs

- AWS Control Tower
- OWASP Top 10 for LLM Applications

Q25 How do you design prompts for classification, extraction and summarisation tasks differently?

Prompting

🎙 Spoken answer · ~63 s

Each task has its own failure modes. For classification, I define the labels with clear criteria and examples for borderline cases, ask for the label only or a label with a confidence field, include an other or unknown option so the model is not forced to guess, and use low temperature. For extraction, I define a schema with field types, say what to output when a field is absent, usually null, require exact quotes from the text for evidence where accuracy matters, and validate against the source. For summarisation, I specify the audience, length, format and focus, tell the model not to add information that is not in the text, and consider extractive checks to detect hallucinated content. In all three, I build an evaluation set with known answers, measure precision and recall or a rubric score, and check the cost, since a smaller model often handles these narrow tasks well.

Likely follow-ups

- Why include an unknown label?
- How do you evaluate summaries?

Classification needs clear labels with an unknown option, extraction needs a schema with nulls and evidence, summarisation needs audience, length and a no-new-facts rule. Evaluate each on a labelled set.

```
schema={'invoice_no':'string|null','total':'number|null','evidence':'string'}
```

```
python eval_extraction.py --gold gold.jsonl
```

Interview tip: Mention nulls for missing fields to prevent invention.

📚 Official docs

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q26 Explain retrieval-augmented generation (RAG) and its main components.

RAG & vectors

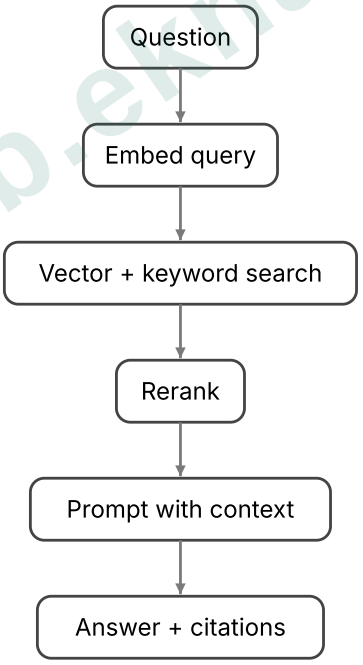
🎙 Spoken answer · ~62 s

RAG improves answers by fetching relevant information at request time and giving it to the model as context, instead of relying only on what it memorised. The pipeline has two sides. Offline ingestion loads documents, cleans and splits them into chunks, creates embeddings and stores them with metadata in a vector index or search engine. Online, the user's question is turned into a query, often rewritten, the retriever finds the most relevant chunks, optionally a reranker reorders them, and the prompt is built from the instructions, the retrieved context and the question. The model answers using that context, ideally with citations, and the application validates the output. RAG gives fresher knowledge, access to private data, source attribution, lower hallucination and no retraining when documents change. The trade-offs are more moving parts, retrieval quality as the dominant factor, added latency and the need to secure and update the index.

Likely follow-ups

- When is RAG better than fine-tuning?
- What are the two failure points?

🗺 RAG pipeline



Ingest, chunk, embed and index offline; at query time retrieve, rerank, build the prompt and generate with citations. RAG adds fresh private knowledge without retraining, but retrieval quality dominates.

```
python ingest.py --src docs/ --chunk 800 --overlap 100
```

```
curl localhost:6333/collections/docs/points/search -d @query.json
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Say retrieval quality is the thing that decides RAG quality.

📚 **Start here (official docs)**

- Hugging Face documentation

Q27 How do you choose chunking strategy and chunk size for RAG?

RAG & vectors

🎙 **Spoken answer · ~60 s**

Chunking determines what the retriever can find and what the model sees, so I treat it as a tuning parameter. Chunks that are too small lose context and produce fragments; chunks that are too large dilute the embedding, waste prompt tokens and bring in irrelevant text. A practical start is a few hundred tokens with a modest overlap, then I adjust by evaluation. I prefer structure-aware splitting that follows headings, paragraphs, code blocks and table boundaries, rather than blindly cutting at a character count, and I keep metadata such as title, section, source and date with every chunk. Advanced options include parent-child chunking, where small chunks are retrieved but the larger parent is sent to the model, and adding a short context summary to each chunk. I decide using a retrieval evaluation set, measuring recall and answer quality across different settings, not intuition.

Likely follow-ups

- What is parent-child retrieval?
- Why keep overlap between chunks?

Start with a few hundred tokens and small overlap, split along document structure, keep metadata, consider parent-child retrieval, and tune with a retrieval evaluation set rather than intuition.

```
splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100)
```

```
python eval_retrieval.py --chunk 400 600 800 --k 5
```

Interview tip: Say you tune chunking against measured recall.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q28 What is hybrid search and why combine keyword and vector retrieval?

RAG & vectors

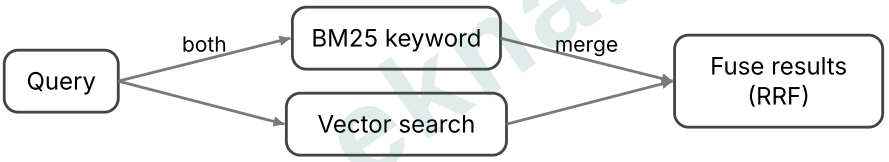
🎙 Spoken answer · ~58 s

Vector search finds semantically similar text, even with different wording, but it can miss exact terms such as product codes, error messages, names and rare acronyms. Keyword search, typically BM25, is excellent at exact matches but fails when the query uses different words than the document. Hybrid search runs both and merges their results, commonly with reciprocal rank fusion or a weighted score, so you get semantic recall and lexical precision. It is especially valuable for technical and enterprise content full of identifiers. Many systems, such as Elasticsearch, OpenSearch, and several vector databases, offer it natively. After merging, a reranker can further improve the ordering. I validate that hybrid actually helps on my own queries by comparing recall against vector-only retrieval, and I tune the fusion weights, because the best mix depends on the content and query types.

Likely follow-ups

- What is reciprocal rank fusion?
- Which queries do keyword indexes handle better?

🗺 Hybrid search



Vector search handles meaning, BM25 handles exact identifiers; hybrid merges both with rank fusion for better recall and precision. Confirm gains with a retrieval evaluation set.

```
GET docs/_search {"retriever":{"rrf":{"retrievers":[{"standard":{...}},{"knn":{...}}]}}}
```

```
curl localhost:6333/collections/docs/points/query -d @hybrid.json
```

Interview tip: Mention error codes and product names as the case for keywords.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q29 What does a reranker do and when is it worth the added latency?

RAG & vectors

🎙 Spoken answer · ~61 s

The first-stage retriever, usually embedding similarity, is fast but approximate: it scores the query and each chunk separately. A reranker is a cross-encoder model that reads the query and a candidate chunk together and produces a more accurate relevance score. The usual pattern is to retrieve a broad set, for example fifty candidates, rerank them, and send only the top handful to the language model. This raises precision, reduces noise and saves prompt tokens, which often improves answer quality more than switching the generator. The costs are extra latency, typically tens to a few hundred milliseconds depending on the model and number of candidates, and extra compute to host. I use a reranker when answers suffer from irrelevant context, when the corpus is large and noisy, or when the top few results matter most, and I confirm the benefit with an offline evaluation and a latency budget.

Likely follow-ups

- How many candidates would you rerank?
- Cross-encoder versus bi-encoder?

A cross-encoder reranker scores query and chunk together, so retrieving many and reranking to a few raises precision and cuts prompt tokens, at some latency. Use it when context noise hurts answers.

```
scores = reranker.predict([(query, c.text) for c in candidates])
```

```
curl $RERANK/v1/rerank -d '{"query":"...","documents":[...],"top_n":5}'
```

Interview tip: Say retrieve wide, rerank narrow, then check the latency budget.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q30 How do you choose a vector database or index for RAG?

RAG & vectors

🎙 **Spoken answer · ~59 s**

I compare options against requirements rather than pick by fashion. Scale: a few million vectors can live happily in pgvector inside PostgreSQL, while hundreds of millions to billions point to a specialised engine or a distributed one. Query features: filtering by metadata, hybrid search, multi-tenancy, and updates and deletes with good performance. Operations: managed service versus self-hosting, backups, high availability, security controls, and integration with existing platforms, which is why many teams start with PostgreSQL or OpenSearch they already run. Index types matter: HNSW offers high recall and speed but uses memory, IVF and product quantisation trade accuracy for size. Cost and latency targets complete the picture. I benchmark candidates on my own data and filters measuring recall at a given latency, because published benchmarks may not match my workload, and I design so the index can be rebuilt from source documents.

Likely follow-ups

- HNSW versus IVF?
- Why rebuildability matters?

Choose by scale, filtering and hybrid features, operations and security, index type and cost; benchmark recall versus latency on your data. pgvector is often enough to start, and the index must be rebuildable.

```
CREATE INDEX ON docs USING hnsw (embedding vector_cosine_ops);
```

```
SET hnsw.ef_search = 100;
```

Interview tip: Say start with what you already run, and benchmark on your data.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q31 How do metadata filters and access control work in RAG, and why are they critical? RAG & vectors

🎙 Spoken answer · ~58 s

Every chunk should be stored with metadata: source document, owner or team, access control list, tenant, classification, language, date and version. At query time the retrieval step must apply filters derived from the authenticated user, so the search only returns chunks the user is allowed to see. This is essential, because a RAG system that ignores permissions can leak confidential documents through the model's answer, even if the original system would have denied access. I enforce this in the retrieval layer, not in the prompt, since a prompt instruction cannot reliably stop a model from revealing text it was given. For multi-tenant systems, I use per-tenant indexes or mandatory tenant filters tested for leaks. Permissions also change over time, so I sync access control updates and deletions promptly, and I audit which sources were used for each answer.

Likely follow-ups

- Why not enforce access in the prompt?
- How do you keep ACLs in sync?

Store ACLs and tenant metadata with every chunk and filter at retrieval time using the user's identity. Never rely on the prompt for access control, sync permission changes and audit sources used.

```
filter={'must':[{'key':'acl','match':{'any':user.groups}}]}
```

```
SELECT id FROM docs WHERE tenant_id = $1 ORDER BY embedding <=> $2 LIMIT 5;
```

Interview tip: Enforce permissions in retrieval; a prompt is not an access control.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q32 How do you keep a RAG index fresh and handle updates and deletions?

RAG & vectors

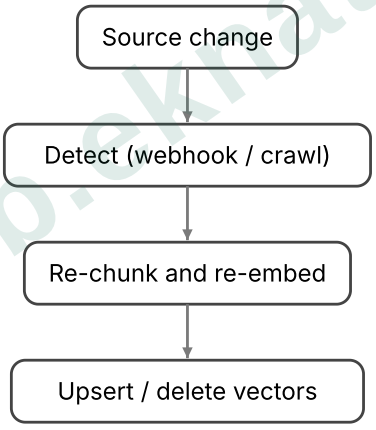
🎙 Spoken answer · ~58 s

I design ingestion as a pipeline with change detection. Sources emit events or are crawled on a schedule, and each document has a stable ID and a content hash, so I only re-process changed documents. Updating a document means deleting its old chunks by document ID and inserting the new ones, preferably in an atomic step or with versioning so queries never see a half-updated state. Deletions must propagate, which matters for compliance and for removing outdated or retracted content. I track freshness metadata, so the retriever can prefer recent versions and the answer can show the date. For embedding model changes or chunking changes, I build a new index alongside the old one, validate it on the evaluation set, then switch over, which gives a safe rollback. Monitoring ingestion lag, failures and index size keeps the system trustworthy.

Likely follow-ups

- How do you re-embed everything safely?
- What do you monitor in ingestion?

🗺 Index freshness



Ingestion with stable IDs and content hashes, delete-and-reinsert per document, propagated deletions, freshness metadata, and blue-green rebuilds for embedding or chunking changes. Monitor lag and failures.

```
sha256sum docs/*.md > manifest.new && diff manifest.old manifest.new
```

```
curl -X POST localhost:6333/collections/docs/points/delete -d '{"filter":{...doc_id...}}'
```

Interview tip: Blue-green indexes make embedding upgrades safe.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q33 How do you evaluate a RAG system, separating retrieval quality from generation quality?

RAG & vectors

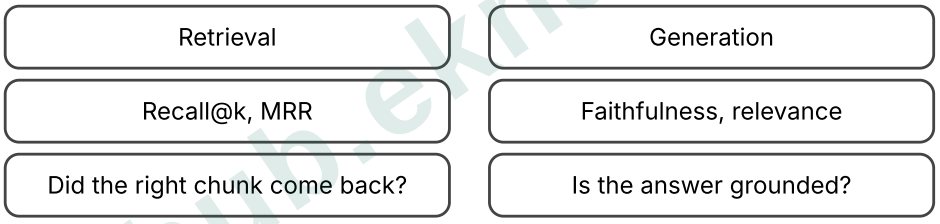
🎙 Spoken answer · ~58 s

I measure the two stages independently, because the fixes are different. For retrieval, I build a set of questions with known relevant documents or passages and compute recall at k, mean reciprocal rank and precision, which tells me whether the right context is found. For generation, given the retrieved context, I assess faithfulness, meaning whether the answer is supported by the context, answer relevance to the question, completeness, and citation correctness. Tools like RAGAS, TruLens and custom rubrics, with an LLM as a judge calibrated against human labels, automate this. I also test refusal behaviour when the answer is not in the corpus, and robustness to paraphrased or adversarial questions. End-to-end, I track user feedback and escalation rates in production. When quality drops, the split tells me whether to fix chunking, embeddings and search, or the prompt and model.

Likely follow-ups

- What is faithfulness?
- How do you build the question set?

🗺 Evaluate RAG in two parts



Evaluate retrieval with recall at k and MRR against labelled passages, and generation with faithfulness, relevance, completeness and citation accuracy, plus refusal tests. The split tells you what to fix.

```
ragas evaluate --dataset qa.jsonl --metrics context_recall faithfulness answer_relevancy
```

```
python eval_retrieval.py --k 5 --gold gold.jsonl
```

Interview tip: Separate retrieval from generation so you know which to fix.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q34 Your RAG chatbot gives wrong answers even though the right document is in the index. How do you debug? RAG & vectors

🎙 Spoken answer · ~62 s

I trace one failing question through every stage. First, was the right chunk retrieved? I log the retrieved chunks and scores. If it is missing, the cause is in chunking, embeddings, query phrasing or filters, and I can try query rewriting, hybrid search or a different chunk size. If it was retrieved but ranked low, a reranker or a better top-k helps. If it was in the prompt but the model ignored it, the context may be too long and noisy, the key fact may be buried in the middle, or the instructions may not insist on using the context; I shorten the context and strengthen the instruction. If the chunk was cut so the answer is split across chunks, I increase overlap or use parent retrieval. I also check stale versions and access filters. Then I add this failing question to the evaluation set to prevent regression.

Likely follow-ups

- How do you log retrieval for debugging?
- What is query rewriting?

Trace the failure stage by stage: was the chunk retrieved, ranked, included in the prompt and used? Fix chunking, hybrid search, reranking, context length or instructions accordingly, and add the case to the evaluation set.

```
python trace_query.py --q 'How do I rotate keys?' --show-chunks
```

```
jq '.retrieved[] | {id,score}' trace.json
```

Interview tip: Walk the pipeline stage by stage and add the case to the evaluation set.

📚 Start here (official docs)

- Hugging Face documentation

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q35 What is query rewriting and when does it help retrieval?

RAG & vectors

🎙 Spoken answer · ~60 s

User questions are often short, ambiguous, conversational or full of pronouns, which makes them poor search queries. Query rewriting uses a model to transform the input into one or more better queries before retrieval. Examples include resolving references using conversation history, so that 'what about the second one' becomes a standalone question, expanding abbreviations and synonyms, splitting a compound question into sub-questions, or generating several paraphrases and merging the results. Another technique, hypothetical document embeddings, asks the model to write a plausible answer and uses that as the search vector. These steps improve recall, especially in multi-turn chat and for complex questions, at the cost of an extra model call, added latency, and the risk of drifting from the original intent. I add rewriting when evaluation shows recall problems, use a small fast model, and keep the original query as one of the searches.

Likely follow-ups

- What is HyDE?
- How do you control the added latency?

A model rewrites or decomposes the query into standalone, expanded or multiple queries before retrieval, improving recall in chat and complex questions at the cost of latency and possible drift. Keep the original query too.

```
rewritten = llm('Rewrite as a standalone search query: ' + history + question)
```

```
results = retrieve(q) + retrieve(rewritten)
```

Interview tip: Keep the original query alongside the rewritten ones.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q36 How do you handle PDFs, tables, images and other messy documents in ingestion?

RAG & vectors

🎙 **Spoken answer · ~60 s**

Document quality sets the ceiling for RAG. For PDFs I first check whether they contain real text or scans; scans need OCR. Layout-aware parsers extract reading order, headings, columns, headers and footers, and I remove repeated boilerplate that pollutes chunks. Tables are tricky: flattening them to plain text loses structure, so I convert them to markdown or structured rows, keep the caption and headers with each row group, or index a generated description. Images and diagrams can be captioned by a vision model, with the caption indexed and the image reference kept. Code and configuration files are split by function or block. I preserve metadata such as page numbers for citation. I sample the parsed output and read it, because silent parsing errors are common, and I record parser versions so re-ingestion is reproducible. Good extraction usually helps more than a fancier model.

Likely follow-ups

- How do you test parsing quality?
- What do you do with scanned documents?

Use layout-aware parsing and OCR for scans, strip boilerplate, convert tables to structured text with headers, caption images, split code by block, keep page metadata, and read samples of the parsed output.

```
pdftotext -layout doc.pdf - | head -50
```

```
unstructured-ingest local --input-path docs/ --output-dir parsed/
```

```
tesseract scan.png out
```

Interview tip: Say you read samples of the parsed output; silent failures are common.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q37 What is the difference between RAG and fine-tuning, and when do you combine them?

RAG & vectors

🎙 **Spoken answer · ~57 s**

RAG supplies knowledge at request time. It is the right tool when the information changes, is private, large, or must be cited, and when you need to control access. Fine-tuning changes the model's behaviour: its style, format, domain vocabulary and task skill, and it is better for consistent tone, specialised output formats, or teaching a narrow skill. Fine-tuning is poor at injecting facts reliably and makes updates slow, since new knowledge needs retraining, while RAG updates by editing documents. They combine well: fine-tune for how the model should behave, for example how to read the retrieved context, cite and refuse, and use RAG for what it should know. I normally start with prompting, then add RAG, and only fine-tune when evaluation shows a behaviour gap that prompts cannot close, because fine-tuning adds data, training and versioning overhead.

Likely follow-ups

- Why is fine-tuning poor for facts?
- What data would you need to fine-tune?

RAG gives fresh, private, citable knowledge; fine-tuning changes behaviour and format. Combine them: fine-tune for how, retrieve for what. Start with prompts, add RAG, and fine-tune only for proven behaviour gaps.

```
python finetune.py --method lora --data sft.jsonl
```

```
python rag_query.py --q 'policy for refunds'
```

Interview tip: Summarise: fine-tune for how, retrieve for what.

📚 **Official docs**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q38 How do you add citations and grounding checks to RAG answers?

RAG & vectors

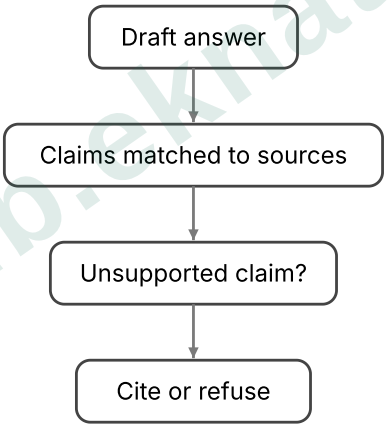
🎙 Spoken answer · ~64 s

I make the model show its evidence and then verify it in code. The prompt supplies each chunk with an ID and instructs the model to answer only from the provided sources and to mark each claim with the ID of the supporting chunk, or to say that the answer is not in the sources. The application then post-processes: it checks that every cited ID actually exists in the retrieved set, renders those IDs as links to the original documents with page or section, and optionally verifies support. Verification can use a natural language inference model or a second LLM call that checks each statement against its cited text, flagging unsupported claims. If a claim is unsupported, I remove it, regenerate, or show a lower confidence message. Citations build user trust, speed fact checking and give me data to evaluate faithfulness, but they do not prove correctness, so critical use still needs review.

Likely follow-ups

- Why do cited IDs need to be checked?
- What is an NLI check?

🗺 Grounding check



Give chunks IDs, require claim-level citations or an I-don't-know, then verify in code that IDs exist, link them to sources, and optionally check support with an NLI model or a second call. Citations aid trust but are not proof.

```
answer = llm(prompt_with_chunks)
```

```
assert all(c in retrieved_ids for c in extract_citations(answer))
```

Interview tip: Check cited IDs exist; models can invent them.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com

Q39 How do you reduce latency in a RAG pipeline?

RAG & vectors

🎙 Spoken answer · ~59 s

I start by measuring each stage: query rewriting, embedding, vector search, reranking, prompt build, time to first token and generation. Then I optimise the largest. Run independent steps in parallel, such as keyword and vector search. Cache query embeddings and frequent answers. Use a smaller, faster embedding model or a local one to avoid network hops, tune the index, for example the HNSW ef_search parameter and the number of candidates, and keep the index in memory. Limit the reranker to a modest candidate set, or use a lighter reranker. Send fewer, better chunks to the model, which cuts prefill time. Stream the answer so users see tokens early, and use prompt caching for the static instructions. Place the services close together to reduce network latency. Finally, set a latency budget per stage and track p95 and p99, not only the average.

Likely follow-ups

- Which stage usually dominates?
- How does streaming change perceived latency?

Measure each stage, parallelise independent steps, cache embeddings and answers, tune the index and candidate counts, shrink the context, stream output and use prompt caching, with budgets tracked at p95 and p99.

```
curl -w '@curl-format.txt' -o /dev/null -s $RAG/query -d @q.json
```

```
curl -s localhost:8000/metrics | grep time_to_first_token
```

Interview tip: Say you track p95 and p99 per stage.

📚 Official docs

- Prometheus: Histograms and summaries

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q40 What is GraphRAG and when would a knowledge graph help?

RAG & vectors

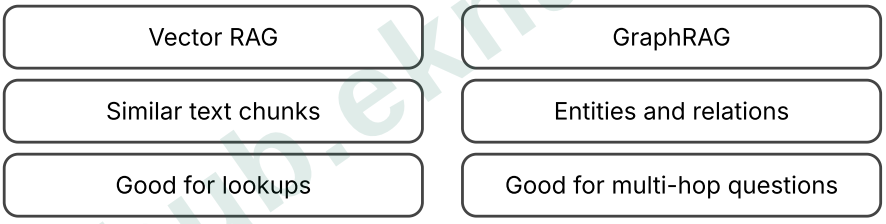
🎙 Spoken answer · ~60 s

Plain RAG retrieves isolated chunks, which struggles with questions that need connections across many documents, such as which services depend on a failing component, or what are the main themes in this corpus. GraphRAG builds a knowledge graph, extracting entities and relationships from the text, often with an LLM, and then answers using graph traversal, community summaries or a combination with vector search. It helps with multi-hop reasoning, global summarisation questions, and domains where relationships are central, for example infrastructure dependencies, compliance, supply chains and incident history. The costs are significant: extraction is expensive and can be noisy, graphs need maintenance as data changes, and the pipeline is more complex. I would try it when evaluation shows multi-hop or global questions failing with standard retrieval, and when an existing graph, such as a CMDB or service catalogue, can be reused instead of generated from scratch.

Likely follow-ups

- What kinds of questions fail in plain RAG?
- How do you keep a graph current?

🗺 Vector RAG vs GraphRAG



GraphRAG extracts entities and relations into a graph to answer multi-hop and global questions that chunk retrieval misses. It is costly and complex, so try it when evaluation shows such failures, ideally reusing an existing graph like a CMDB.

```
MATCH (s:Service)-[:DEPENDS_ON*1..3]->(d:Service {name:'auth'}) RETURN s
```

```
python build_graph.py --src docs/
```

Interview tip: Reuse an existing graph such as a service catalogue if possible.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q41 How do you secure a RAG system against data leakage and poisoned documents?

RAG & vectors

🎙 **Spoken answer · ~63 s**

There are two risks. Leakage comes from missing or weak access control, from logging sensitive prompts and retrieved text, and from sending data to third-party models without agreements. I enforce permissions at retrieval, mask sensitive data during ingestion where required, encrypt the index and logs, and set retention limits. Poisoning means an attacker places content in a source the system ingests, such as a wiki page, ticket or email, containing false facts or hidden instructions that the model will follow when retrieved, which is indirect prompt injection. Defences include controlling who can write to ingested sources, reviewing and scanning documents on ingestion, tracking provenance and trust levels by source, separating retrieved text from instructions in the prompt, limiting what actions the model can take after reading it, and monitoring outputs for anomalies. I also keep audit logs of which documents were used, so that I can trace and remove bad content quickly.

Likely follow-ups

- What is provenance tracking?
- How do you remove a poisoned document quickly?

Prevent leakage with retrieval-time access control, masking, encryption and retention. Prevent poisoning by controlling writers, scanning on ingestion, tracking provenance, separating data from instructions, limiting actions and auditing which documents were used.

```
detect-secrets scan docs/
```

```
python ingest.py --deny-sources untrusted --scan-injection
```

```
grep doc_id audit.log | sort | uniq -c
```

Interview tip: Name indirect prompt injection via ingested content.

📚 **Official docs**

- OWASP Top 10 for LLM Applications

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q42 What is an AI agent and how does it differ from a simple LLM call or a workflow?

Agents & tools

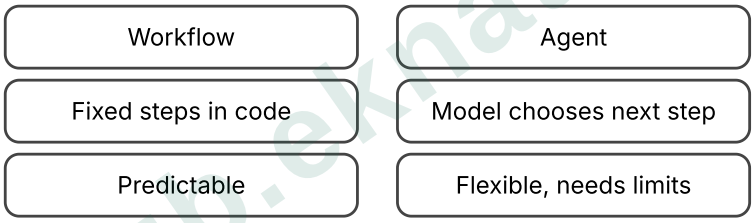
🎙 Spoken answer · ~62 s

A simple LLM call takes input and returns text. A workflow chains model calls and tools along a path that the developer defines in code, so the control flow is fixed and predictable. An agent lets the model decide the control flow: in a loop, it observes the state, chooses an action, usually a tool call, sees the result, and repeats until it decides the goal is met or a limit is reached. This flexibility lets agents handle open-ended tasks, such as investigating an incident across several systems, but it makes behaviour less predictable, harder to test, and more costly and slower, and it increases risk because the model chooses actions. My rule is to use the simplest design that works: a single call if possible, a workflow if the steps are known, and an agent only when the path genuinely cannot be predefined, with strong limits and monitoring.

Likely follow-ups

- When is a fixed workflow better than an agent?
- What limits would you put on an agent loop?

🗺 LLM call, workflow, agent



A call returns text, a workflow follows developer-defined steps, and an agent lets the model choose actions in a loop. Agents are flexible but less predictable, so use the simplest design that works and add limits.

```
while not done and steps < MAX_STEPS: action = llm(state); state = run(action)
```

Interview tip: Say simplest design that works: call, then workflow, then agent.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q43 What is the Model Context Protocol (MCP) and why does it matter for platform teams?

Agents & tools

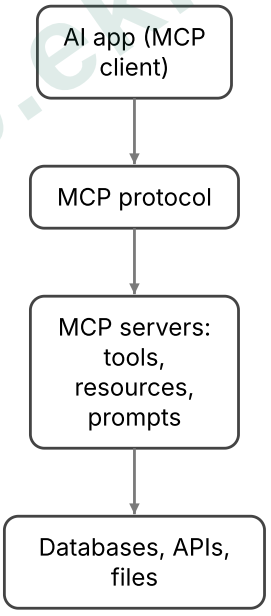
🎙 Spoken answer · ~61 s

The Model Context Protocol is an open standard that defines how AI applications connect to external tools, data sources and prompts through a common interface. An MCP server exposes capabilities, such as tools that perform actions, resources that provide data, and prompt templates, and an MCP client inside the AI application discovers and calls them using JSON-RPC over transports like stdio or streamable HTTP. Before it, each application needed custom integrations for every tool. With MCP, one server for a system such as a ticketing tool, a database or Kubernetes can be reused by many clients. For platform teams it means a standard way to publish internal capabilities to agents, but also a new security surface: servers must authenticate, authorise per user, validate inputs, rate limit and log, tool descriptions can carry injected instructions, and credentials must be scoped. I treat MCP servers like any production API.

Likely follow-ups

- What transports does MCP use?
- How do you authorise an MCP server per user?

🗺 Model Context Protocol



MCP standardises how AI apps discover and call tools, resources and prompts exposed by servers, replacing custom integrations. Treat servers as production APIs: authentication, per-user authorisation, validation, rate limits and audit.

```
npx @modelcontextprotocol/inspector node server.js
```

```
claude mcp add my-server -- node server.js
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Say MCP servers need the same security as any API.

📚 **Official docs**

- Model Context Protocol

Q44 How do you design tools for an agent so it uses them correctly?

Agents & tools

🎙 **Spoken answer · ~58 s**

Tools are the interface between the model and your systems, so I design them like a good API for a new colleague who cannot ask questions. Each tool has a clear name, a concise description saying when to use it and when not to, and a tight schema with typed, documented parameters and sensible defaults. I prefer a few well-scoped tools to many overlapping ones, because confusion rises with choice. Outputs are concise, structured and informative, returning only what the agent needs, since large dumps waste context, and errors are actionable, saying what went wrong and how to fix the call. I make operations idempotent, add dry-run options, limit blast radius with permissions and quotas, and require confirmation for destructive actions. Then I test with real agent runs, read the transcripts, and refine descriptions where it picks the wrong tool.

Likely follow-ups

- Why are fewer tools better?
- How do you make destructive tools safe?

Few, well-scoped tools with clear names and descriptions, typed schemas, concise structured outputs, actionable errors, idempotency, dry-run and confirmation for destructive actions. Refine from transcripts.

```
{"name":"restart_service","description":"Restart one service in staging
only...","parameters":{...}}
```

```
pytest tests/test_tools.py
```

Interview tip: Read agent transcripts to find where tool descriptions mislead.

📚 **Official docs**

- Model Context Protocol

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q45 How do you limit what an agent can do? Describe least privilege for agents.

Agents & tools

🎙 Spoken answer · ~61 s

I assume the model can be tricked or simply make mistakes, so the safety must be in the environment, not in the prompt. Each agent gets its own identity with narrowly scoped, short-lived credentials, only the tools it needs, and read-only access by default. Write actions are restricted by allow lists of resources, environments and commands, and executed through a gateway that enforces policy, rate limits and budgets. High-impact operations, such as deleting data, changing production, spending money or sending external messages, require human approval, with a clear summary of what will happen. Code execution happens in a sandbox with no ambient credentials, limited network and resource caps. Every action is logged with the agent identity, the user on whose behalf it acts, inputs and outcomes, and there is a kill switch. Finally, I test with adversarial inputs to see what a compromised agent could actually do.

Likely follow-ups

- Why short-lived credentials?
- What actions always need human approval?

Give each agent its own scoped, short-lived identity, read-only defaults, allow-listed tools and resources, a policy gateway, human approval for high-impact actions, a sandbox for code, full audit logs and a kill switch.

```
kubectl create role agent-readonly --verb=get,list --resource=pods
```

```
kubectl auth can-i delete pods --as=system:serviceaccount:ai:agent
```

Interview tip: Put safety in the environment, not in the prompt.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q46 What is the ReAct pattern and how do planning and reflection improve agents?

Agents & tools

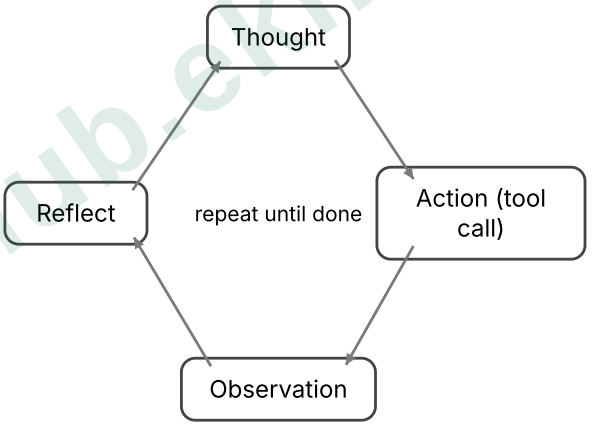
🎙 Spoken answer · ~61 s

ReAct, reasoning and acting, interleaves the model's reasoning with tool use: it thinks about what to do, takes an action, observes the result, and reasons again, which grounds its decisions in real feedback instead of guessing. Planning adds an explicit step where the agent breaks a goal into subtasks before executing, which helps with long tasks and lets you review or constrain the plan. Reflection or self-critique has the agent check its own result against the goal, notice mistakes, and retry, which can catch errors but also costs more tokens and can loop. In production I combine these with limits: a maximum number of steps, a token and cost budget, loop detection for repeated identical calls, and checkpoints where progress is saved. For reliability I prefer plans that are visible and verifiable, and external checks like tests or validators over asking the model to simply be careful.

Likely follow-ups

- How do you detect an agent stuck in a loop?
- Why prefer external verification over self-critique?

🗺 ReAct loop



ReAct alternates reasoning and tool actions grounded in observations; planning decomposes tasks; reflection checks results. Bound them with step, token and cost limits, loop detection and external validators.

```
if history[-3:] == [last_call]*3: raise LoopDetected()
```

```
pytest -q  # external check after agent edit
```

Interview tip: Say external tests beat asking the model to be careful.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com

Q47 How do you handle memory and state in agents across steps and sessions?

Agents & tools

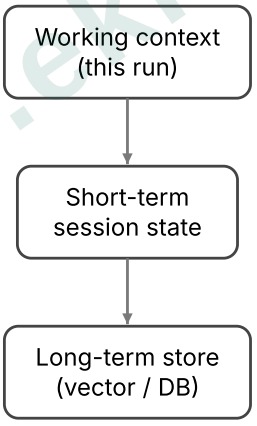
🎙 Spoken answer · ~62 s

I distinguish several kinds of memory. Working memory is the current context window, which holds the task, recent steps and tool results; I keep it small by summarising or dropping verbose outputs. Short-term session state, such as the plan and progress, is stored outside the model in a database or file, so the agent can resume after a failure. Long-term memory stores durable facts and preferences, with retrieval, using a vector or key-value store, and I include only relevant items in each prompt. I make memory writes deliberate, because blindly saving everything creates noise, stale facts and privacy risk. Memory entries carry timestamps and sources, can be reviewed and deleted by users, and are scoped per user or tenant. For long-running tasks, I use checkpoints and idempotent steps, so retries do not repeat side effects, and I treat remembered text as untrusted input because it may contain injected instructions.

Likely follow-ups

- What should never be stored in memory?
- How do checkpoints help with retries?

🗺 Agent memory



Keep working memory small, store plan and progress externally with checkpoints, use retrieval-based long-term memory with deliberate writes, timestamps and per-user scope, and treat remembered text as untrusted.

```
redis-cli HSET agent:123 step 4 plan '...'
```

```
SELECT fact FROM memory WHERE user_id=$1 ORDER BY embedding <=> $2 LIMIT 5;
```

Interview tip: Mention privacy, deletion and treating memory as untrusted input.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Start here (official docs)**

- Hugging Face documentation

Q48 What are multi-agent systems, and when are they worth the complexity? Agents & tools

🎙 **Spoken answer · ~58 s**

A multi-agent system uses several model-driven agents with different roles, tools and prompts that collaborate, for example a planner, a researcher, a coder and a reviewer, either coordinated by an orchestrator or communicating directly. The advantages are specialisation, smaller focused contexts, parallel work, and the possibility of independent review, which can improve quality on complex tasks. The disadvantages are real: more tokens and cost, higher latency, harder debugging because behaviour emerges from interactions, error propagation between agents, and a larger security surface. I try a single agent with good tools first, and move to multiple agents when a task naturally splits into independent parts, needs isolation of permissions, or benefits from a separate verifier. I keep roles and handoffs explicit, log each message, set budgets, and evaluate end to end, because adding agents does not guarantee better results.

Likely follow-ups

- What is an orchestrator-worker pattern?
- How do you debug multi-agent failures?

Multiple role-specific agents can specialise, parallelise and cross-check, but cost more and are harder to debug and secure. Start with one agent and split only for independent parts, permission isolation or independent review.

```
python orchestrator.py --agents planner,researcher,reviewer --budget 50000
```

```
jq '.messages[] | {from,to,tokens}' run.json
```

Interview tip: Start single-agent and justify each extra agent.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q49 How do you test and evaluate agents, given that their behaviour varies between runs?

Agents & tools

🎙 Spoken answer · ~61 s

I evaluate outcomes and behaviour across many runs. I build a suite of tasks with a clear success check, such as a test that passes, a state in a sandbox environment, or a rubric, and run each task several times to measure the success rate and variance, not a single outcome. I track efficiency, steps, tokens, cost and time, plus safety metrics: forbidden actions attempted, tool errors and policy violations. Tool calls are mocked or run in a sandbox so tests are repeatable and safe. I keep transcripts and review failures by hand to find patterns such as wrong tool choice, loops or lost context, then fix prompts, tools or limits and add the case to the suite. Regression runs gate changes to the model, prompt or tools. In production I sample real runs for review, and monitor success proxies like task completion and user corrections.

Likely follow-ups

- Why run each task several times?
- How do you sandbox tools for tests?

Build a task suite with automatic success checks, run each task multiple times for success rate and variance, track cost and safety violations, use sandboxed tools, review transcripts of failures and gate changes on regression runs.

```
python run_evals.py --tasks tasks/ --runs 5 --sandbox
```

```
jq '[.[] | .success] | add/length' results.json
```

Interview tip: Report success rate across runs, not one run.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q50 How would you build an on-call assistant agent for SRE, and what safeguards does it need?

Agents & tools

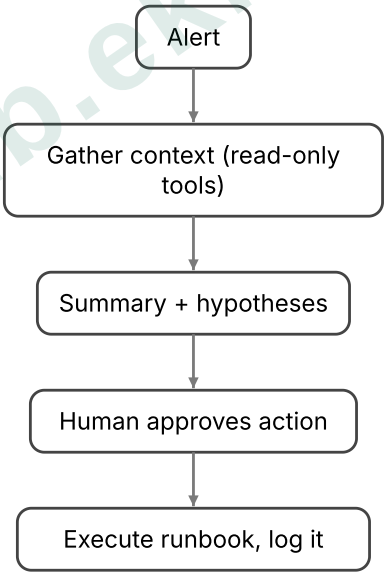
🎙 Spoken answer · ~60 s

I would start read-only. The agent receives an alert, gathers context through tools: recent deployments, relevant dashboards and metrics queries, logs, traces, runbooks retrieved from the knowledge base, and the service owner. It correlates these and produces a summary with the likely cause, the evidence and suggested next steps, linking each claim to the source. That alone saves minutes during an incident. Later, I may allow low-risk actions, such as running a diagnostic command, restarting a stateless pod in staging, or opening a ticket, each behind an allow list and approval. Safeguards include least-privilege credentials, no direct production writes without a human confirming, rate limits, an audit trail, protection against injected instructions in logs, a clear display of uncertainty, and a feedback button to measure usefulness. I would run it in shadow mode on past incidents first and measure accuracy before trusting it live.

Likely follow-ups

- How would you measure usefulness?
- Which actions would you never automate?

🗺 On-call assistant



Start read-only: gather alerts, deploys, metrics, logs and runbooks, then summarise cause and evidence with links. Add low-risk actions behind approvals, least privilege, audit and injection defences; validate in shadow mode first.

```
curl $PROM/api/v1/query --data-urlencode 'query=rate(http_requests_total{code=~"5.."}[5m])'
```

```
kubectl rollout history deploy/api
```

certs.eknathalabs.com

Interview tip: Say shadow mode on historical incidents comes before live use.

📚 **Official docs**

- Google SRE Workbook: On-Call

Q51 What is the difference between synchronous agents and long-running, asynchronous agent workflows, and how do you run them reliably?

Agents & tools

🎙 **Spoken answer · ~60 s**

A synchronous agent answers within a single request, which is fine for tasks that take seconds. Many useful tasks, like researching, migrating code or triaging a backlog, take minutes or hours, so they must be asynchronous. I run them as durable jobs: the request creates a task record and returns an ID, a worker executes the agent loop, state and progress are persisted after each step, and the client polls, subscribes or receives a webhook when it finishes. Workflow engines such as Temporal or a queue with a state store give retries, timeouts, resumption after crashes and visibility. Steps with side effects are idempotent or use idempotency keys. I set overall budgets and deadlines, allow cancellation, request human approval at checkpoints through a pause-and-resume mechanism, and emit traces so progress is observable. This turns an agent from a demo into a dependable service.

Likely follow-ups

- Why use idempotency keys?
- How does pause-and-resume work for approvals?

Run long agents as durable jobs with a task ID, persisted state after each step, a workflow engine or queue for retries and resumption, idempotent side effects, budgets, cancellation, approval checkpoints and tracing.

```
temporal workflow start --type AgentTask --task-queue agents --input @task.json
```

```
temporal workflow show -w task-123
```

Interview tip: Name durable execution and idempotency keys.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q52 How do you observe and debug agent behaviour in production?

Agents & tools

🎙 **Spoken answer · ~67 s**

I trace every run end to end. Each request gets a trace ID, and each model call, tool call, retrieval and decision becomes a span with inputs, outputs, latency, token counts, model version and cost, following OpenTelemetry conventions where possible, with sensitive fields redacted. This lets me replay what happened for any failure. Dashboards show success rate, steps per task, tool error rates, cost per task, latency percentiles, loop and limit hits, and guardrail triggers, and alerts fire on anomalies such as a spike in cost or a tool failing. Because outputs are non-deterministic, I keep transcripts for sampled and failed runs, support user feedback signals, and can replay a run against a new prompt or model offline. This is the same observability discipline as for services, extended with token and quality signals. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- What goes into a span for a model call?
- How do you replay a failed run?

Trace each run with spans for model calls, tools and retrievals including tokens, latency, versions and cost, with redaction. Dashboards and alerts on success, steps, tool errors and spend; keep transcripts to replay failures.

```
otel-cli span --name tool.call --attrs tool=restart,status=ok
```

```
jq '.spans[] | {name,duration_ms,attrs}' trace.json
```

Interview tip: Say it is ordinary observability plus token and quality signals.

📚 **Official docs**

- OpenTelemetry: GenAI semantic conventions

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q53 What is continuous batching and why is it central to LLM serving throughput? Serving

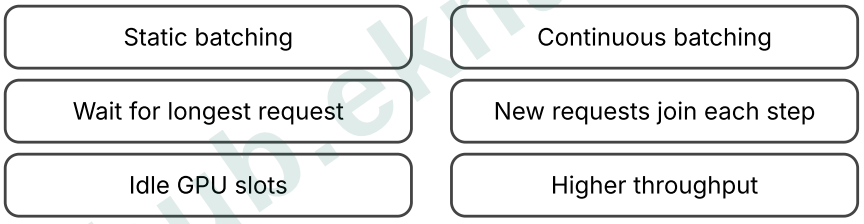
🎙 Spoken answer · ~63 s

Traditional static batching groups requests and processes them together, but because each request generates a different number of output tokens, the whole batch waits for the longest one, leaving the GPU idle for finished sequences. Continuous batching, also called in-flight batching, schedules at the level of individual decoding steps. After every step, finished sequences are removed from the batch and newly arrived requests are inserted immediately, so the GPU stays busy and short requests are not held back by long ones. Combined with efficient memory management for the KV cache, it can raise throughput several times over naive serving and cut queueing latency. Engines like vLLM, TensorRT-LLM and Text Generation Inference implement it. As an operator, the relevant knobs are maximum batch size, maximum number of batched tokens and scheduling policy, which trade throughput against per-request latency, and I tune them with load tests that mimic my real prompt and output lengths.

Likely follow-ups

- What knob trades latency for throughput?
- Why does static batching waste GPU time?

🗺 Static vs continuous batching



Continuous batching adds and removes sequences at every decoding step, so the GPU never waits for the longest request. It multiplies throughput and cuts queueing. Tune batch limits with realistic load tests.

```
vllm serve model --max-num-seqs 128 --max-num-batched-tokens 8192
```

```
python benchmark_serving.py --backend vllm --request-rate 10
```

Interview tip: Say you tune it with a load test that matches real lengths.

📚 Official docs

- vLLM documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q54 What is PagedAttention and how does vLLM manage GPU memory?

Serving

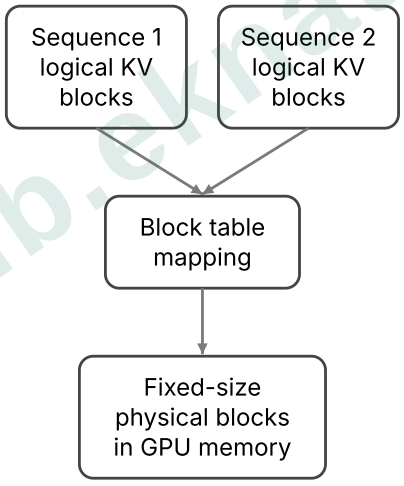
🎙 Spoken answer · ~60 s

The KV cache for each request grows as tokens are generated and its final length is unknown in advance. Allocating one contiguous block sized for the maximum length wastes huge amounts of memory through fragmentation and over-reservation, limiting how many requests fit. PagedAttention borrows the idea of virtual memory paging: the KV cache is divided into fixed-size blocks, allocated on demand and mapped through a block table, so a sequence's cache need not be contiguous and waste is minimal. Blocks can also be shared between requests that have the same prefix, for example parallel sampling or a common system prompt, with copy-on-write. The result is much higher batch sizes on the same GPU and therefore higher throughput. vLLM reserves a fraction of GPU memory for this cache, controlled by gpu-memory-utilization, and exposes cache usage as a metric that I watch to avoid preemption.

Likely follow-ups

- What happens when the KV cache is full?
- How does prefix sharing work?

🗺 PagedAttention



PagedAttention stores the KV cache in fixed-size on-demand blocks mapped by a block table, eliminating fragmentation and enabling prefix sharing. That fits more concurrent requests per GPU. Watch cache usage and preemptions.

```
vllm serve model --block-size 16 --gpu-memory-utilization 0.90
```

```
curl -s localhost:8000/metrics | grep gpu_cache_usage
```

Interview tip: Compare it to virtual memory paging to make it memorable.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Kubernetes: Scheduling GPUs
- vLLM documentation

Q55 How do you choose between vLLM, TensorRT-LLM, Text Generation Inference and llama.cpp?

Serving

🎙 **Spoken answer · ~58 s**

I choose by hardware, model coverage, performance needs and operational fit. vLLM is a flexible, widely adopted engine with broad model support, an OpenAI-compatible API, continuous batching, PagedAttention, quantisation and multi-LoRA, and is a strong default on NVIDIA and increasingly on other accelerators. TensorRT-LLM from NVIDIA compiles models for maximum performance on NVIDIA GPUs, which can win on latency and throughput, at the cost of a build step and tighter hardware coupling. Text Generation Inference from Hugging Face is a mature server with good integration into that ecosystem. llama.cpp targets CPUs and modest hardware, including laptops and edge devices, using GGUF quantised models, which is ideal for local and low-cost deployments, but not for high-concurrency serving. I benchmark candidates with my own model and traffic profile, check API compatibility and metrics, and weigh the upgrade and support burden.

Likely follow-ups

- What does OpenAI-compatible API buy you?
- When is llama.cpp the right answer?

vLLM is the flexible default, TensorRT-LLM maximises NVIDIA performance with a build step, TGI fits the Hugging Face ecosystem, llama.cpp suits CPU and edge. Benchmark on your own model and traffic.

```
vllm serve model --port 8000
```

```
./llama-server -m model.gguf -c 4096 --port 8080
```

```
trtllm-build --checkpoint_dir ckpt --output_dir engine
```

Interview tip: End with: I would benchmark on my workload.

📚 **Official docs**

- vLLM documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q56 How do you scale an LLM inference service on Kubernetes, and why is CPU-based autoscaling a poor fit?

Serving

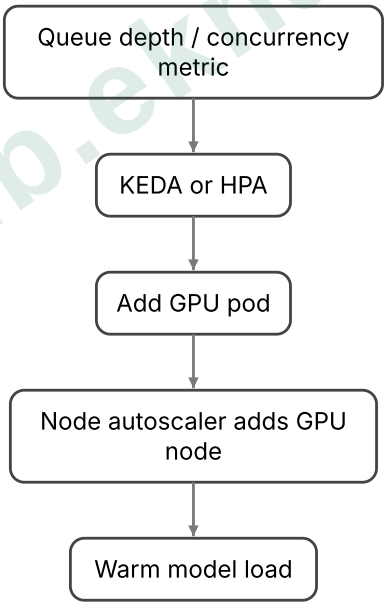
🎙 Spoken answer · ~63 s

Standard HPA on CPU or memory does not reflect load on a GPU-bound inference server, because the CPU may be idle while the GPU queue is saturated. I scale on signals that track demand: number of waiting or running requests, queue depth, KV cache utilisation, time to first token or tokens per second, exposed by the server as Prometheus metrics and fed to HPA through the custom metrics API, or to KEDA. Scaling out is slow because a new replica must schedule onto a GPU node, pull a large container image, and load tens or hundreds of gigabytes of weights, which can take minutes. So I keep warm capacity, use image and model caching on nodes, preload weights from fast storage, scale up early with conservative scale-down, and sometimes use predictive scaling. Cluster autoscaler or Karpenter provisions GPU nodes, and I set pod disruption budgets and graceful draining of in-flight requests.

Likely follow-ups

- Why is scale-up slow for LLM pods?
- Which metric would you scale on?

🗺 Autoscale inference



Scale on queue depth, running requests, KV cache use or time to first token via custom metrics or KEDA, not CPU. Cold starts are slow because of GPU nodes, image pulls and weight loading, so keep warm capacity and cache models.

```
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1 | jq .
```

```
kubectl apply -f keda-scaledobject.yaml
```

```
kubectl top pod -l app=llm
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Mention the multi-minute cold start as the design constraint.

📚 **Official docs**

- KServe documentation

Q57 How do you reduce model load time and cold start for large models?

Serving

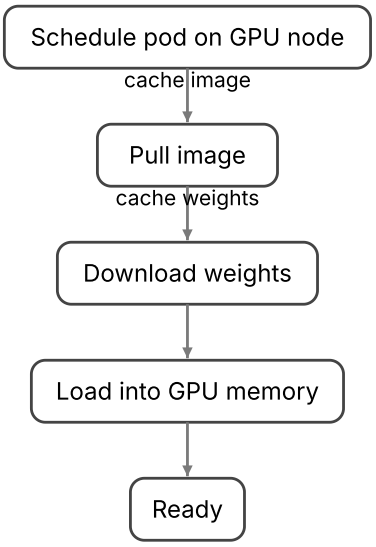
🎙 **Spoken answer · ~61 s**

Cold start has three parts: scheduling a GPU node, pulling the container image, and loading weights into GPU memory. I attack each. Keep a buffer of ready nodes or use fast provisioning so scheduling is quick. Keep images small, split the model out of the image, and pre-pull images to nodes with a DaemonSet or an image cache. Store weights on fast storage close to the GPU, such as local NVMe, a node-level cache populated once, or a high-throughput shared file system or object store with parallel readers, rather than downloading from the internet on every start. Use safetensors format and loaders that stream or memory-map weights, and tensor parallel loading across GPUs in parallel. Quantised weights are smaller and load faster. Finally, avoid frequent scale to zero for latency-sensitive services and use readiness probes that only pass after the model is fully loaded and warmed up.

Likely follow-ups

- Why a readiness probe after warm-up?
- What is the benefit of safetensors?

🗺 **Cold start path**



Pre-provision GPU nodes, keep images small and pre-pulled, cache weights on local NVMe or fast shared storage in safetensors format, use quantised weights, avoid scale to zero for latency-sensitive services, and gate readiness on warm-up.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
huggingface-cli download org/model --local-dir /models/model
```

```
kubectl apply -f daemonset-image-prepull.yaml
```

```
vllm serve /models/model --load-format safetensors
```

Interview tip: Break cold start into node, image and weights.

📚 **Official docs**

- AWS Lambda: Concurrency

Q58 What metrics do you monitor for an LLM inference service?

Serving

🎙 **Spoken answer · ~59 s**

I split them into user-facing, capacity and quality signals. User-facing: time to first token, inter-token latency, end-to-end latency percentiles, request success and error rates, and tokens per second. Capacity: requests running and waiting, queue time, batch size, KV cache utilisation, preemptions, GPU utilisation, memory use, memory bandwidth, temperature, power and errors such as ECC and XID from the driver, collected by DCGM exporter. Cost and usage: input and output tokens per tenant, model and route, cache hit rate for prompt caching, and cost per request. Quality and safety: guardrail triggers, refusal rate, format validation failures, user feedback and evaluation scores on sampled traffic. I set SLOs on time to first token and success rate, alert on queue growth and cache saturation before users feel it, and break dashboards down by model, version and tenant, so a regression is easy to localise.

Likely follow-ups

- Which metric would you alert on first?
- Why track tokens per tenant?

Track time to first token, inter-token latency, error rate and throughput; running and waiting requests, KV cache use, GPU health through DCGM; tokens and cost per tenant; and quality signals. Set SLOs and alert on queue growth.

```
curl -s localhost:8000/metrics | grep -E
'num_requests_(running|waiting)|time_to_first_token'
```

```
dcgmi dmon -e 1002,1004,203
```

```
nvidia-smi --query-gpu=utilization.gpu,memory.used --format=csv
```

Interview tip: Say you alert on queue growth, a leading indicator.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- OpenTelemetry: GenAI semantic conventions
- KServe documentation

Q59 How do you serve many tenants or models efficiently on shared GPUs?

Serving

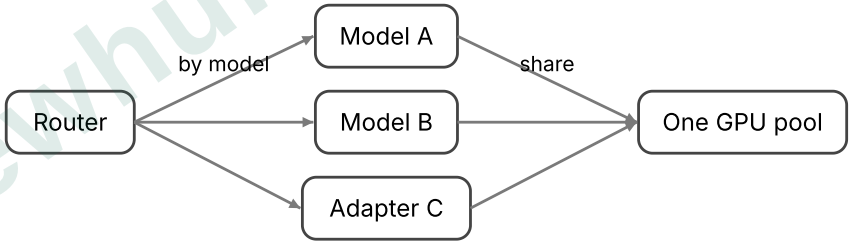
🎙 **Spoken answer · ~58 s**

I choose the isolation level by requirements. For many fine-tuned variants of the same base model, multi-LoRA serving loads one base model and swaps small adapters per request, which is the most efficient option. For multiple different models, I either dedicate pools per model, which is simple but can leave capacity idle, or use a multi-model server that loads and unloads models on demand, accepting load latency. GPU sharing methods include time slicing, which has no memory isolation, MPS for concurrent kernels, and MIG on supported GPUs, which partitions the hardware with strong isolation. A routing layer or LLM gateway applies authentication, per-tenant rate limits and quotas, model selection, and priorities so that one tenant cannot starve others. I add per-tenant metrics for chargeback, and I keep sensitive tenants on dedicated GPUs if the data policy demands it.

Likely follow-ups

- Time slicing versus MIG?
- How do you prevent noisy neighbours?

🗺 **Shared GPU serving**



Use multi-LoRA for variants of one base model, pools or on-demand loading for different models, and GPU sharing through MPS or MIG. A gateway enforces per-tenant limits, priorities and metering; dedicate GPUs where policy demands.

```
vllm serve base --enable-lora --max-loras 8
```

```
nvidia-smi mig -cgi 1g.10gb -C
```

```
kubectl get resourcequota -n tenant-a
```

Interview tip: Name MIG for hard isolation and multi-LoRA for efficiency.

interviewhub.eknathalabs.com
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Kubernetes: Scheduling GPUs

Q60 What is an LLM gateway and what does it provide?

Serving

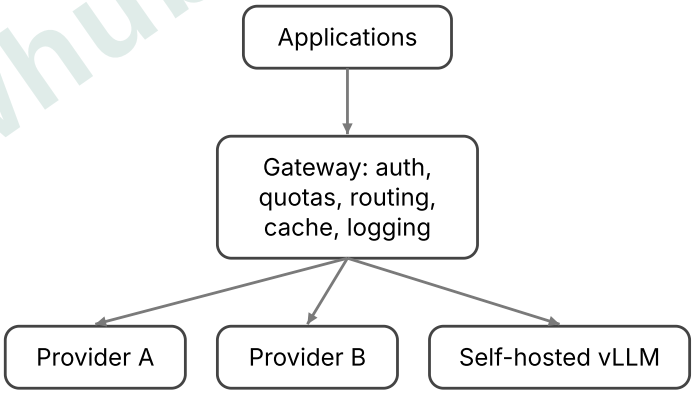
🎙 **Spoken answer · ~63 s**

An LLM gateway is a proxy that sits between applications and one or more model providers or self-hosted servers, presenting a unified API, often OpenAI compatible. It centralises concerns that every team would otherwise reimplement. Routing and fallback: choose a model by task, cost or availability, and fail over to another provider when one is down or rate limited. Authentication and authorisation: issue virtual keys per team or application, with model access policies. Rate limits, quotas and budgets per key, plus usage metering and cost attribution. Caching of identical or semantically similar requests, retries and timeouts. Guardrails such as PII redaction and content filtering, and logging, tracing and audit. It also lets me change models without touching applications. The risks are being a single point of failure and an added latency hop, so it needs high availability, horizontal scaling and careful handling of logged prompts, which may contain sensitive data.

Likely follow-ups

- What are the risks of logging prompts at the gateway?
- How does fallback routing work?

🗺 **LLM gateway**



A gateway provides one API in front of many models with routing and fallback, virtual keys and policies, quotas and cost metering, caching, guardrails and tracing. It needs HA and careful prompt logging.

```
litellm --config config.yaml --port 4000
```

```
curl localhost:4000/v1/chat/completions -H 'Authorization: Bearer sk-team-a' -d @req.json
```

Interview tip: Say it makes model changes invisible to application teams.

InterviewHub - interviewhub.eknathalabs.co

📚 **Start here (official docs)**

- Hugging Face documentation

Q61 How do you roll out a new model version safely?

Serving

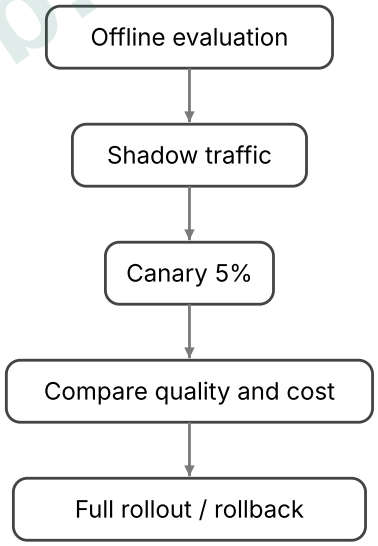
🎙 **Spoken answer · ~59 s**

I treat a model like any other release, with extra attention to behaviour changes that unit tests do not catch. First, offline evaluation: run the candidate on the evaluation set and compare quality, safety, format validity, latency and cost with the current model, with a clear go or no-go threshold. Second, shadow deployment, where real traffic is copied to the candidate and responses are compared but not shown to users. Third, a canary release to a small percentage of traffic or a few tenants, with monitoring of quality proxies, errors, latency, cost and user feedback, and automatic rollback criteria. Then a gradual ramp, keeping the previous version deployed and warm for quick rollback. Model, prompt and configuration are versioned together, and I pin versions explicitly. After release, I keep sampling outputs for review, because subtle quality regressions appear only on real traffic.

Likely follow-ups

- What is shadow traffic?
- What would trigger an automatic rollback?

🗺 **Safe model rollout**



Offline evaluation with thresholds, shadow traffic comparison, canary with quality, cost and latency monitoring and automatic rollback criteria, then gradual ramp while keeping the old version warm. Version model, prompt and config together.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
kubectl argo rollouts set image llm-api model=registry/model:v2
```

```
kubectl argo rollouts promote llm-api
```

```
curl -s $PROM/api/v1/query?query=llm_eval_score
```

Interview tip: Shadow traffic catches regressions without exposing users.

📚 **Start here (official docs)**

- Hugging Face documentation

Q62 What is speculative decoding and when does it help?

Serving

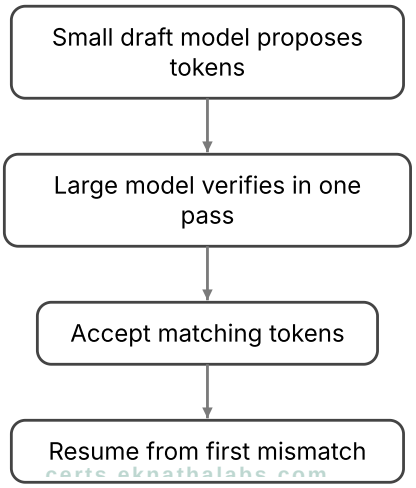
🎙 **Spoken answer · ~61 s**

Autoregressive decoding is slow because each token needs a full pass through a large model, and the GPU is underused during that memory-bound step. Speculative decoding uses a small, fast draft model, or a draft head, to guess several next tokens quickly, then the large target model verifies all of them in a single parallel pass, accepting the longest correct prefix and discarding the rest. Because the target model's verification determines the final output, the result is identical in distribution to normal decoding, while the speed can improve by roughly two to three times when the draft model's guesses are often accepted. It helps most for latency-sensitive, low-batch workloads and predictable text like code. It helps less at high batch sizes, where the GPU is already saturated, and it uses extra memory for the draft model. I benchmark the acceptance rate, since that determines the gain.

Likely follow-ups

- Does speculative decoding change the output quality?
- What determines the speed-up?

🗺 **Speculative decoding**



InterviewHub - interviewhub.eknathalabs.com

A small draft model proposes tokens and the large model verifies them in one pass, giving the same output distribution with up to two to three times lower latency when acceptance is high. Best for low-batch, latency-sensitive loads.

```
vllm serve big-model --speculative-config '{"model":"small-
draft","num_speculative_tokens":5}'
```

```
curl -s localhost:8000/metrics | grep spec_decode
```

Interview tip: Say acceptance rate decides the benefit.

📚 **Official docs**

- vLLM documentation

Q63 How do you implement streaming responses and handle timeouts and retries for LLM APIs?

Serving

🎙 **Spoken answer · ~61 s**

LLM responses can take many seconds, so I stream them using server-sent events or WebSockets, sending tokens as they are generated, which greatly improves perceived latency. The stack between client and model must support streaming: proxies and load balancers need buffering disabled and idle timeouts raised, because default timeouts, for instance 60 seconds, kill long generations. I set separate timeouts for connection, time to first token and total duration, and cancel upstream generation when the client disconnects, which saves GPU time. Retries need care: retry only on safe, transient failures such as 429, 503 and network errors, with exponential backoff and jitter and a retry budget, and never blindly retry after partial output without idempotency, since the user may already have seen text. Rate limit responses should honour the retry-after header. I also apply backpressure and queue limits, so overload fails fast instead of growing latency.

Likely follow-ups

- Why cancel generation on client disconnect?
- Which errors are safe to retry?

Stream with SSE, disable proxy buffering and raise idle timeouts, set separate connect, first-token and total timeouts, cancel on disconnect, and retry only transient errors with backoff, jitter and budgets.

```
curl -N $API/v1/chat/completions -d '{"stream":true,...}'
```

```
proxy_buffering off; proxy_read_timeout 300s;
```

Interview tip: Say disconnect cancellation saves GPU time.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- Microsoft: Cloud design pattern – Retry

Q64 How do you run LLM inference at the edge or on CPUs, and what are the limits? Serving

🎙 **Spoken answer · ~61 s**

For privacy, offline use, or low cost, small quantised models can run on CPUs, laptops, phones or edge boxes. The usual stack is a runtime such as llama.cpp or ONNX Runtime with models in a compact quantised format like GGUF at four or five bits, which fits a few-billion-parameter model into a few gigabytes of RAM. Performance depends on memory bandwidth more than core count, so unified-memory systems and fast RAM help, and CPU instruction sets such as AVX2, AVX-512 or ARM NEON matter. Expect much lower throughput than GPUs, a few to a few tens of tokens per second for small models, and limited concurrency, so it suits single-user or low-volume workloads, local assistants, and on-device features rather than a high-traffic service. Model quality is limited by size, so I choose narrow tasks, test accuracy on the target hardware, and handle updates and model distribution securely.

Likely follow-ups

- Why does memory bandwidth dominate?
- How do you distribute models to edge devices?

Run small quantised models with llama.cpp or ONNX Runtime on CPUs or edge devices. Throughput is bandwidth-limited and concurrency low, so use it for private, offline or low-volume narrow tasks and validate accuracy on the target hardware.

```
./llama-cli -m model-q4_k_m.gguf -p 'Summarise:' -n 128 -t 8
```

```
lscpu | grep -E 'avx|neon'
```

Interview tip: Position it for privacy and offline, not for high concurrency.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q65 How do you design disaster recovery and high availability for an LLM service? Serving

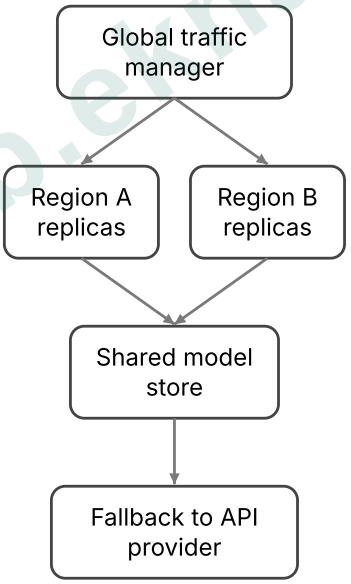
🎙 Spoken answer · ~65 s

I plan at several layers. Compute: run multiple replicas across availability zones, with pod anti-affinity and spare GPU capacity, since losing one GPU node should not exceed the SLO. Model artifacts: store weights in durable, versioned storage replicated to another region, so a recovery site can start without depending on an external download that may be unavailable. Dependencies: the vector database, caches, gateway and secrets need their own HA and backups. Failure of the model itself: a fallback path, to a smaller local model, a second provider, or a degraded mode with cached or retrieval-only answers, which keeps the product useful. Traffic management with health checks that test real generation, circuit breakers and load shedding that protects the system under overload. Finally I set RTO and RPO targets, test failover regularly with game days, and track GPU capacity in the secondary region, because GPUs are scarce and may not be available on demand during an outage.

Likely follow-ups

- Why is GPU capacity a DR risk?
- What is a degraded mode for an LLM product?

🗺 HA for LLM serving



Multi-AZ replicas with spare GPU capacity, replicated versioned model artifacts, HA for vector DB and gateway, fallback models or degraded modes, health checks that test generation, load shedding, and tested failover. Reserve DR GPU capacity.

```
kubectl get pods -l app=llm -o wide
```

```
aws s3 sync s3://models-primary s3://models-dr
```

```
curl $LB/healthz/generate
```

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Interview tip: Mention that GPUs may not be available on demand in a disaster.

📚 **Start here (official docs)**

- Hugging Face documentation

Q66 **How do you schedule GPU workloads on Kubernetes? Explain device plugins and resource requests.** GPU & K8s

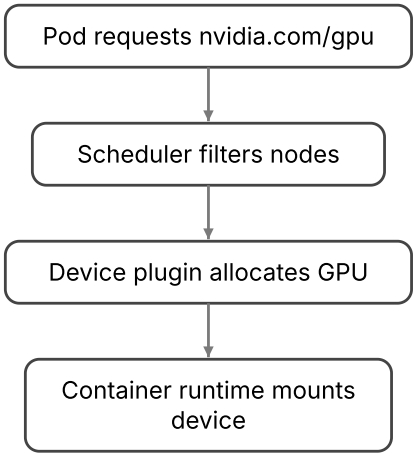
🎙 **Spoken answer · ~59 s**

GPUs are exposed to Kubernetes through the device plugin framework. The NVIDIA device plugin, usually installed with the GPU Operator, runs on each GPU node, advertises the extended resource nvidia.com/gpu to the kubelet, and handles allocation. A pod requests GPUs in its resource limits, for example nvidia.com/gpu: 1, and the scheduler places it on a node with enough free GPUs. GPUs are integer resources and cannot be overcommitted or fractionally requested by default. The container runtime, configured with the NVIDIA container toolkit, mounts the driver libraries and device files into the container. To control placement I label and taint GPU nodes, so only GPU workloads land there, and use node selectors or affinity to choose a GPU type. Newer Kubernetes features such as dynamic resource allocation provide richer device selection. For efficiency, I consider MIG or time slicing for small workloads.

Likely follow-ups

- Why taint GPU nodes?
- What does the container toolkit do?

🗺 **GPU scheduling on Kubernetes**



The device plugin advertises nvidia.com/gpu, pods request it in limits, and the NVIDIA container toolkit injects drivers. Taint and label GPU nodes, and use MIG or time slicing to share small workloads.

```
kubectl describe node gpu-1 | grep nvidia.com/gpu
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
resources:\n  limits:\n    nvidia.com/gpu: 1
```

```
kubectl taint nodes gpu-1 nvidia.com/gpu=present:NoSchedule
```

Interview tip: Say GPUs are integer resources by default.

📚 **Official docs**

- Kubernetes: Scheduling GPUs
- NVIDIA GPU Operator

Q67 What does the NVIDIA GPU Operator install and why use it?

GPU & K8s

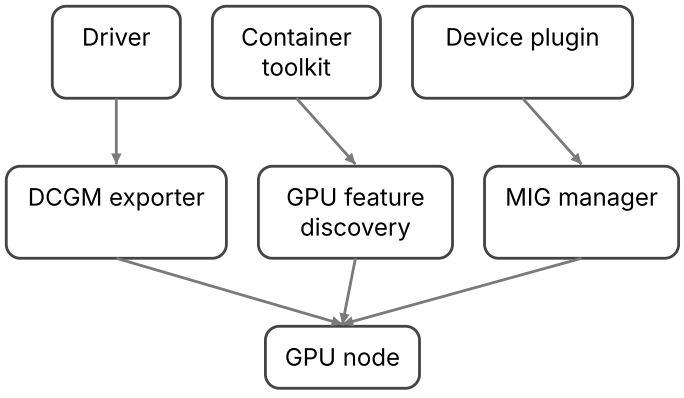
🎙 **Spoken answer · ~60 s**

The GPU Operator automates everything a Kubernetes node needs to use NVIDIA GPUs, as a set of managed components deployed through a custom resource. It installs and manages the GPU driver, the container toolkit, the device plugin, DCGM and the DCGM exporter for metrics, node feature discovery and GPU feature discovery that label nodes with GPU model and capabilities, and optionally MIG manager and the Kubernetes validator. Without it, you would bake drivers into node images and manage versions and upgrades by hand, which is error prone across many nodes. The operator makes upgrades declarative, keeps components consistent, and supports driver containers that match the kernel. Considerations are compatibility between driver, CUDA and Kubernetes versions, ensuring the operator's pods tolerate GPU node taints, and in managed clouds, where the provider may supply drivers, deciding whether to use the operator or the provider's integration.

Likely follow-ups

- What does GPU feature discovery label?
- When would you skip the operator?

🗺 **GPU Operator components**



certs.eknathalabs.com
InterviewHub - interviewhub.ekn

The Operator deploys drivers, container toolkit, device plugin, DCGM exporter, feature discovery and MIG manager declaratively, keeping versions consistent. Check driver and CUDA compatibility and cloud-provider driver options.

```
helm install gpu-operator nvidia/gpu-operator -n gpu-operator --create-namespace
```

```
kubectl get pods -n gpu-operator
```

```
kubectl get node gpu-1 -o json | jq '.metadata.labels |
with_entries(select(.key|test("nvidia")))'
```

Interview tip: List the components to show you know what it manages.

📚 **Official docs**

- Kubernetes: Scheduling GPUs
- NVIDIA GPU Operator

Q68 Explain MIG, time slicing and MPS for sharing a GPU. When would you use each?

GPU & K8s

🎙 **Spoken answer · ~59 s**

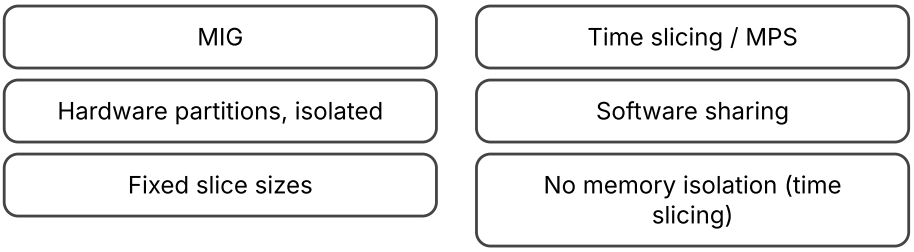
Multi-Instance GPU, available on some data-centre GPUs, partitions one physical GPU into several isolated instances, each with dedicated compute slices, memory and bandwidth, giving predictable performance and hard isolation, which is good for multi-tenant inference and small jobs. Partitions are fixed profiles, so changing them needs reconfiguration. Time slicing lets several pods share one GPU by taking turns, configured through the device plugin; it works on any GPU and is simple, but there is no memory isolation, so one pod can exhaust memory, and latency is variable. Multi-Process Service allows kernels from several processes to run concurrently on the GPU, improving utilisation for many small workloads, with optional resource limits but weaker fault isolation than MIG. I use MIG when isolation and predictability matter, time slicing for dev and test where cost matters most, and MPS to raise utilisation for cooperating workloads.

Likely follow-ups

- Which approach gives memory isolation?
- What are MIG's constraints?

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

🗺 **GPU sharing options**



MIG gives hard-partitioned isolated instances with predictable performance, time slicing is simple but has no memory isolation, and MPS runs kernels concurrently for better utilisation. Pick by isolation and predictability needs.

```
nvidia-smi mig -lgip
```

```
kubectl label node gpu-1 nvidia.com/mig.config=all-1g.10gb
```

```
kubectl get cm time-slicing-config -n gpu-operator -o yaml
```

Interview tip: Match each method to its isolation guarantee.

📚 **Official docs**

- Kubernetes: Scheduling GPUs
- Compute Engine: Managed instance groups
- NVIDIA GPU Operator

terviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q69 How do you monitor GPU health and utilisation, and what do the signals mean?

GPU & K8s

🎙 Spoken answer · ~60 s

I use the DCGM exporter, which publishes GPU metrics to Prometheus. Utilisation shows how busy the compute engines are, but it only says a kernel was running, not how efficiently, so I also watch memory used and memory copy utilisation, SM activity and tensor core activity, which show real work. Memory use against capacity predicts out-of-memory errors. Temperature, clock throttling reasons and power draw reveal cooling or power limits that reduce performance. Error signals matter most for reliability: ECC errors, XID errors in the kernel log, which indicate faults such as a fallen-off-the-bus GPU, and NVLink errors. A GPU with growing correctable errors or recurring XID events should be drained and repaired. I also correlate GPU metrics with application metrics, since low GPU utilisation with high queue time suggests a CPU or data-loading bottleneck, whereas high utilisation with growing latency means capacity is exhausted.

Likely follow-ups

- Why is utilisation misleading?
- What is an XID error?

Use DCGM exporter for utilisation, memory, SM and tensor activity, temperature, throttling, power and ECC and XID errors. Drain GPUs with repeated errors, and correlate with queue and latency metrics.

```
nvidia-smi dmon -s pucvmet
```

```
dmesg | grep -i xid
```

```
curl -s localhost:9400/metrics | grep DCGM_FI_DEV_GPU_UTIL
```

Interview tip: Say GPU utilisation alone is a weak efficiency signal.

📚 Official docs

- Kubernetes: Scheduling GPUs
- OpenTelemetry: GenAI semantic conventions

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q70 What is tensor, pipeline and data parallelism, and how do they affect inference deployment?

GPU & K8s

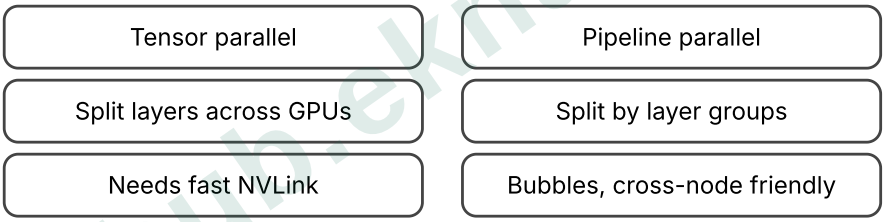
🎙 Spoken answer · ~61 s

These are ways to spread work across GPUs. Data parallelism runs a full copy of the model on each GPU and splits the requests, which scales throughput when the model fits on one GPU. Tensor parallelism splits each layer's matrices across GPUs, so every GPU computes part of every layer, which lets a model too big for one GPU run and cuts latency, but it needs very fast interconnect such as NVLink, because the GPUs exchange data at every layer. Pipeline parallelism splits the model by layers into stages on different GPUs, passing activations along, which needs less bandwidth and suits multi-node setups but introduces pipeline bubbles. For inference I use tensor parallelism within a node over NVLink, pipeline parallelism across nodes if needed, and data parallelism replicas for scale. I also check topology with nvidia-smi, because a poor PCIe layout can hurt tensor parallel performance badly.

Likely follow-ups

- Why does tensor parallelism need NVLink?
- How do you choose the tensor parallel size?

🗺 Parallelism for big models



Data parallelism replicates the model to scale throughput, tensor parallelism splits layers across GPUs to fit big models and cut latency but needs NVLink, pipeline parallelism splits by layers across nodes. Check topology.

```
vllm serve big-model --tensor-parallel-size 4 --pipeline-parallel-size 2
```

```
nvidia-smi topo -m
```

Interview tip: Say tensor parallel stays inside a node on NVLink.

📚 Official docs

- Terraform CLI: apply
- Ray documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q71 How do you size GPU memory and capacity for serving a given model?

GPU & K8s

🎙 **Spoken answer · ~60 s**

I start with arithmetic. Weight memory is the number of parameters times bytes per parameter: a seven-billion parameter model needs about 14 GB at 16-bit, 7 GB at 8-bit and about 4 GB at 4-bit. On top of that comes the KV cache, which depends on layers, hidden dimension, number of KV heads, sequence length and concurrent sequences, plus activation and framework overhead, so I leave 10 to 20 percent headroom. For a target concurrency and context length I compute the cache requirement and check it fits, then choose GPU memory and tensor parallelism accordingly. Throughput planning uses a benchmark with realistic prompts: measure tokens per second per GPU at my latency SLO, then divide the expected peak tokens per second by that number and add spare capacity for failures and growth. I validate with load tests, because theory and real engines can differ.

Likely follow-ups

- How do you estimate KV cache size?
- What headroom do you reserve?

Weights are parameters times bytes per parameter, add KV cache for concurrency and context, plus 10 to 20 percent overhead. Then benchmark tokens per second per GPU at the SLO and size for peak plus failure headroom.

```
python -c "print(7e9*2/1e9,'GB at fp16')"
```

```
python benchmark_serving.py --dataset prompts.json --request-rate 8
```

Interview tip: Show the arithmetic: parameters times bytes, plus cache, plus headroom.

📚 **Official docs**

- Kubernetes: Scheduling GPUs

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q72 How do you handle GPU node failures and drain them safely in Kubernetes? GPU & K8s

🎙 Spoken answer · ~61 s

GPU nodes fail in ways CPU nodes rarely do: driver crashes, ECC or XID errors, GPUs falling off the bus and overheating. I detect them with DCGM health checks, node problem detector and alerts on error counters, and I automate marking affected nodes unschedulable. To drain, I cordon the node, then evict pods with the eviction API so that pod disruption budgets are respected, giving inference servers time to finish in-flight requests through a pre-stop hook and a termination grace period long enough for streaming generations. For training jobs, I rely on checkpoints, so that the job resumes from the latest one on another node, and on a controller or framework that restarts failed workers, such as an elastic launcher. After draining, I run diagnostics, reset or replace the GPU, and return the node with a validation test. Spare capacity and node auto-repair reduce the impact.

Likely follow-ups

- Why a long termination grace period?
- How do checkpoints limit the cost of failure?

Detect with DCGM and error alerts, cordon and drain respecting PDBs, let inference drain in-flight streams with pre-stop hooks, resume training from checkpoints, then diagnose, repair and validate before returning the node.

```
kubectl cordon gpu-3 && kubectl drain gpu-3 --ignore-daemonsets --delete-emptydir-data
```

```
dcgmi diag -r 2
```

```
dmesg -T | grep -i xid
```

Interview tip: Mention checkpoints for training and drain time for inference.

📚 Official docs

- Kubernetes: Safely drain a node
- Kubernetes: Scheduling GPUs

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q73 What networking matters for multi-node GPU training and inference?

GPU & K8s

🎙 Spoken answer · ~61 s

Distributed training and large-model inference exchange large tensors between GPUs constantly, so the interconnect is as important as the GPUs. Within a node, NVLink and NVSwitch provide very high bandwidth between GPUs, far beyond PCIe. Across nodes, I need a high-bandwidth, low-latency fabric, usually InfiniBand or RoCE Ethernet at 200 to 800 gigabits per node, with RDMA, and GPUDirect RDMA so network cards read GPU memory directly without CPU copies. Collective libraries such as NCCL implement all-reduce and related operations, and tuning involves topology awareness, correct network interface selection, and avoiding oversubscribed switches. In Kubernetes, this means multiple network interfaces through Multus or the SR-IOV device plugin, host networking or RDMA device plugins, and careful pod placement within the same rack or spine for locality. Common problems are NCCL picking the wrong interface, packet loss on RoCE without proper congestion control, and straggler nodes slowing everyone.

Likely follow-ups

- What does GPUDirect RDMA avoid?
- How do you diagnose slow all-reduce?

Use NVLink within a node and InfiniBand or RoCE with RDMA and GPUDirect across nodes, NCCL tuned for topology, multiple interfaces in Kubernetes, and locality-aware placement. Watch for wrong interfaces, RoCE loss and stragglers.

```
NCCL_DEBUG=INFO python train.py
```

```
nccl-tests/build/all_reduce_perf -b 8 -e 1G -f 2 -g 8
```

```
ibstat
```

Interview tip: Say the interconnect often limits scaling more than the GPUs.

📚 Official docs

- Kubernetes: Scheduling GPUs

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q74 How would you build a shared GPU platform for several teams? Consider quotas, queues and fairness.

GPU & K8s

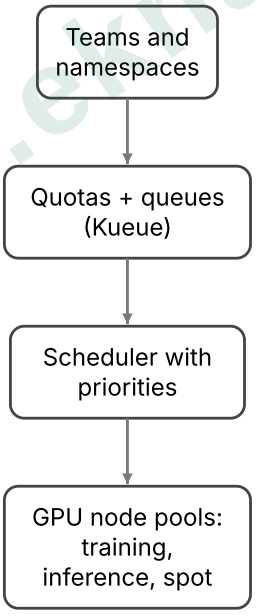
🎙 Spoken answer · ~60 s

I treat GPUs as a scarce shared resource with a proper scheduling layer. Teams get namespaces with resource quotas on GPU counts, plus priority classes so that production inference can preempt batch jobs. For batch and training, a queueing system such as Kueue, Volcano or a similar scheduler provides gang scheduling, so a multi-GPU job starts only when all its GPUs are available, and fair-share or borrowing between teams, so idle quota is used and returned when needed. Interactive notebooks get time limits and idle culling. Node pools are separated by GPU type with labels and taints, and users request by class rather than model name. Cost visibility through per-team usage reports and showback encourages efficient use. I add utilisation dashboards, since low-utilisation allocations are the usual waste, and policy on maximum job duration, with checkpointing encouraged, to keep the system fair and efficient.

Likely follow-ups

- What is gang scheduling?
- How does borrowing work between queues?

🗺 Shared GPU platform



Namespaces with GPU quotas, priority classes, a queueing layer such as Kueue or Volcano with gang scheduling and fair-share borrowing, node pools by GPU type, notebook culling and showback. Dashboards expose idle allocations.

```
kubectl get clusterqueue
```

```
kubectl create quota team-a-gpu --hard=requests.nvidia.com/gpu=8 -n team-a
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
kubectl get workloads -A
```

Interview tip: Name gang scheduling and fair-share borrowing.

📚 **Official docs**

- Kubernetes: Scheduling GPUs
- NIST AI Risk Management Framework

Q75 How do you improve low GPU utilisation in a training or inference workload? GPU & K8s

🎙 **Spoken answer · ~59 s**

I find the bottleneck before changing anything. Profile with the framework profiler, Nsight or DCGM metrics to see whether the GPU is waiting. Common causes: slow data loading, where CPU preprocessing or storage throughput starves the GPU, fixed with more workers, caching, faster storage, prefetching and storing data in efficient formats. Small batch sizes that do not fill the device, fixed by larger batches or continuous batching for inference. Python overhead and synchronisation points, reduced with compiled graphs, fused kernels and avoiding unnecessary host to device copies. Communication overhead in multi-GPU jobs, addressed by overlapping communication and compute, and improving the network. Uneven work between ranks, causing stragglers. Mixed precision such as bf16 improves speed and memory. I change one thing at a time and measure samples or tokens per second, not just the utilisation percentage, since efficiency is what ultimately matters.

Likely follow-ups

- How do you tell data loading from compute bottlenecks?
- Why is utilisation percentage not enough?

Profile first. Typical fixes: faster data pipelines and caching, larger batches or continuous batching, fewer host-device syncs, overlap communication with compute, mixed precision, and balanced ranks. Measure throughput, not utilisation alone.

```
nsys profile -o prof python train.py
```

```
py-spy dump --pid $(pgrep -f train.py)
```

```
iostat -x 1
```

Interview tip: Say you measure tokens or samples per second.

📚 **Official docs**

- Kubernetes: Scheduling GPUs

interviewhub.eknathalabs.com
carts.eknathalabs.com
InterviewHub - interviewhub.ekn

Q76 What are the options for getting GPU capacity, and how do you handle scarcity?

GPU & K8s

🎙 **Spoken answer · ~56 s**

Options include on-demand cloud GPUs, which are flexible but may be unavailable and costly; reserved or committed capacity, cheaper but locked in; spot or preemptible instances, much cheaper but interruptible, good for fault-tolerant batch work; specialised GPU cloud providers; and owning on-premises hardware, with the best unit economics at high steady utilisation but long lead times and operational burden. Because supply can be constrained, I plan capacity ahead, use a mix: a reserved baseline for production inference, spot for training with checkpoints, and on-demand burst for peaks. I reduce demand by quantisation, smaller models, batching and caching, and by routing easy requests to cheaper models. Multi-region or multi-provider flexibility helps availability, as does portable infrastructure code. I track utilisation and queue times, and review quotas with the provider early, since quota increases can take days.

Likely follow-ups

- Which workloads suit spot GPUs?
- How do you decide between buying and renting?

Mix reserved capacity for production inference, spot with checkpointing for training, and on-demand for bursts; consider specialised providers or owned hardware at high steady use. Cut demand with quantisation, smaller models and routing.

```
aws service-quotas get-service-quota --service-code ec2 --quota-code L-DB2E81BA
```

```
kubectl get nodes -l node.kubernetes.io/instance-type=p4d.24xlarge
```

Interview tip: Say quotas and lead times are part of capacity planning.

📚 **Official docs**

- Kubernetes: Scheduling GPUs

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q77 What is MLOps and how does it differ from DevOps?

MLOps

🎙 Spoken answer · ~57 s

MLOps applies DevOps principles, automation, version control, testing, continuous delivery and monitoring, to machine learning systems, but ML adds assets and failure modes that ordinary software does not have. In addition to code, you must version data, features, models and experiments, because a model is the product of code plus data. Testing includes data validation and model evaluation, not just unit tests, since a pipeline can run successfully and still produce a worse model. Deployment includes model registries, staged rollouts and rollback. Monitoring covers not only latency and errors but also data drift, prediction quality and fairness, because performance can decay silently as the world changes. Retraining pipelines, triggered by schedule, drift or new data, close the loop. As a platform engineer, I provide the shared infrastructure for these steps: pipelines, registries, feature stores, compute and observability.

Likely follow-ups

- What is data drift?
- Which artefacts need versioning in ML?

MLOps adds data, features, models and experiments to the versioned assets, data and model validation to testing, and drift and quality to monitoring, with retraining pipelines closing the loop.

```
dvc add data/train.parquet && git add data/train.parquet.dvc
```

```
mlflow models serve -m models:/churn/Production -p 5001
```

Interview tip: Say the new failure mode is silent quality decay.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q78 What does a production ML pipeline look like, from data to deployed model?

MLOps

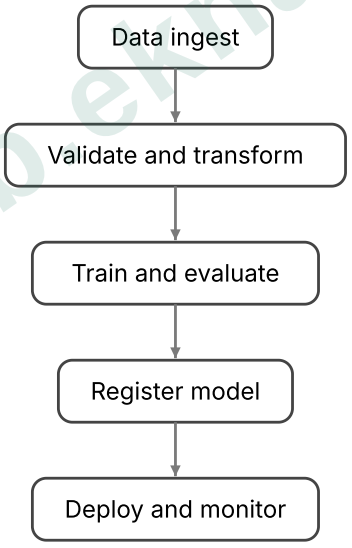
🎙 Spoken answer · ~60 s

A typical pipeline has stages that are automated and reproducible. Data ingestion pulls from sources into a versioned dataset, followed by validation that checks schema, ranges, missing values and drift against expectations. Feature engineering transforms raw data, ideally with the same code used at serving time, often through a feature store. Training runs on scalable compute with tracked parameters and seeds, and produces a model artefact. Evaluation compares the candidate against a baseline on held-out data and fairness or safety slices, with thresholds that gate promotion. The approved model is registered with its metadata, lineage and metrics, then deployed through a staged process, shadow or canary, to a serving platform. Monitoring tracks inference quality, latency and drift, and triggers retraining when needed. The orchestration is handled by tools such as Kubeflow Pipelines, Argo Workflows or Airflow, with everything defined as code and run in CI.

Likely follow-ups

- Why must feature code be shared between training and serving?
- Which stages gate promotion?

🗺 ML pipeline



Ingest and validate data, build features shared with serving, train with tracked parameters, evaluate against a baseline with gating thresholds, register, deploy gradually and monitor, with an orchestrator running it all as code.

```
argo submit train-pipeline.yaml -n mlops
```

```
kfp run submit -e prod -p pipeline.yaml
```

```
mlflow runs list --experiment-id 3
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Stress validation and evaluation gates; they are the ML-specific parts.

📚 **Start here (official docs)**

- Hugging Face documentation

Q79 What is training-serving skew and how do you prevent it?

MLOps

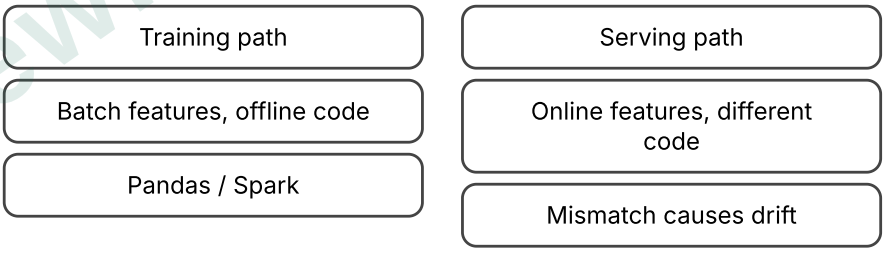
🎙 **Spoken answer · ~63 s**

Training-serving skew means the model sees different data or features in production than it saw during training, so its accuracy falls even though the model file is unchanged. Causes include different code paths for computing features offline and online, different handling of missing values, time zones or units, data leakage in training that is impossible at serving time, stale features, and changes in upstream data schemas. Prevention starts with using the same feature definitions and code for both, which is the main purpose of a feature store with an offline and an online store. Point-in-time correct joins avoid using future information in training. I log the features actually served and compare their distributions with the training data, alerting on drift or schema changes, and I validate inputs at the serving boundary. Integration tests that run the same examples through both paths, and shadow deployments, catch remaining differences before users are affected.

Likely follow-ups

- What is point-in-time correctness?
- How do you detect skew in production?

🗺 **Training-serving skew**



Skew arises when production features differ from training features. Share feature definitions through a feature store, use point-in-time joins, log served features and compare distributions, validate inputs and test both paths with identical examples.

```
feast materialize-incremental $(date -u +%Y-%m-%dT%H:%M:%S)
```

```
python compare_dist.py --train train.parquet --serve served_features.parquet
```

Interview tip: Say one definition of each feature, used in both paths.

InterviewHub - interviewhub.eknathalabs.co

📚 **Start here (official docs)**

- Hugging Face documentation

Q80 What is a feature store and when do you need one?

MLOps

🎙 **Spoken answer · ~63 s**

A feature store is a system that manages the definition, computation, storage and serving of features for ML. It has an offline store, holding historical feature values for training, often in a data warehouse or lake, and an online store, a low-latency key-value database for real-time inference, kept consistent with each other. It provides a registry so teams can discover and reuse features, point-in-time joins to avoid leakage, and monitoring of freshness and quality. You need one when multiple models or teams reuse the same features, when you serve real-time predictions that need consistent online and offline values, or when training-serving skew causes problems. For a single batch model with a few features, it is overkill, and a well-organised pipeline is enough. Options include Feast, Tecton, and the feature stores built into cloud platforms. The cost is another system to operate, so adopt it when the reuse and consistency benefits are real.

Likely follow-ups

- What is the difference between online and offline stores?
- When is it overkill?

A feature store provides a registry, offline storage for training, online storage for low-latency serving, point-in-time joins and freshness monitoring. Adopt it for reuse across models and real-time consistency; skip it for a single simple batch model.

```
feast apply
```

```
feast get-online-features --features user_stats:clicks_7d --entity user_id=123
```

```
feast materialize 2026-01-01T00:00:00 2026-10-01T00:00:00
```

Interview tip: Say it is justified by reuse and online-offline consistency.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q81 How do you use a model registry and track experiments?

MLOps

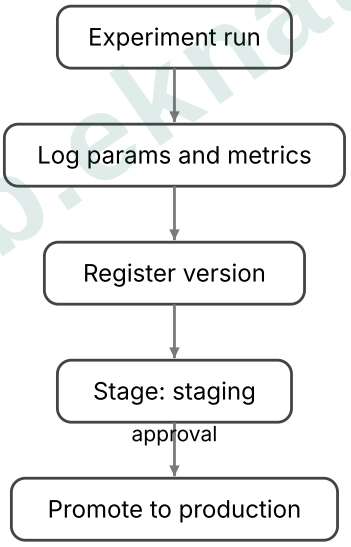
🎙 Spoken answer · ~59 s

Experiment tracking records, for every training run, the code version, data version, hyperparameters, metrics, plots and artefacts, so any result can be reproduced and compared. Tools like MLflow, Weights and Biases or cloud-native equivalents provide this. A model registry then manages the lifecycle of models that are worth keeping: each registered model has versions, lineage back to the run and data, metrics, a description, and stages or aliases such as staging and production. Promotion from one stage to the next is a controlled action, gated by automated evaluation and sometimes human approval, and the deployment system pulls by alias, so rollback is just pointing the alias at a previous version. I add access control, audit logs and signed or checksummed artefacts so that what runs in production is exactly what was approved. This gives traceability, which regulators and incident reviews often require.

Likely follow-ups

- What lineage should a registered model have?
- How does rollback work with aliases?

🗺 Model registry flow



Track runs with code, data, parameters and metrics for reproducibility, and use a registry with versions, lineage, aliases and gated promotion. Deploy by alias so rollback is repointing, with access control and audit.

```
mlflow.register_model('runs:/<run_id>/model','churn')
```

```
mlflow models set-alias churn champion 7
```

```
mlflow artifacts download -u models:/churn@champion
```

Interview tip: Emphasise lineage and alias-based rollback.

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- MLflow documentation

Q82 How do you detect and respond to data drift and concept drift?

MLOps

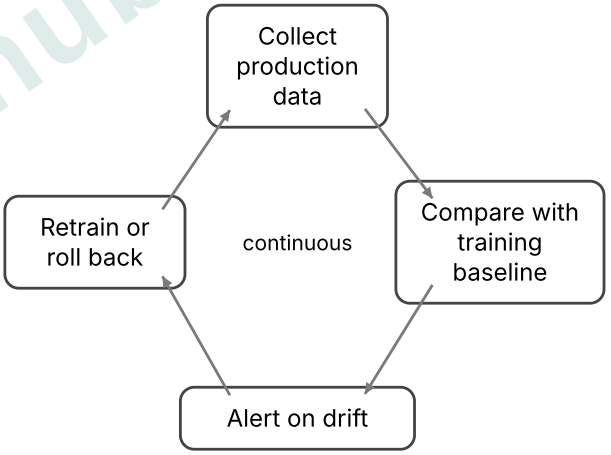
🎙 **Spoken answer · ~62 s**

Data drift means the distribution of input features changes from the training data, for example a new customer segment or a changed sensor. Concept drift means the relationship between inputs and the target changes, so the same inputs now imply a different outcome. To detect data drift, I compare production feature distributions with a reference window using statistical measures such as population stability index, Kolmogorov-Smirnov tests or divergence measures, per feature, with alert thresholds tuned to avoid noise. Concept drift shows up as degrading performance, which needs ground-truth labels that may arrive late, so I track proxy metrics and delayed accuracy when labels become available. Responses depend on severity: investigate upstream data issues first, since many drift alerts are really pipeline bugs, then retrain on recent data, adjust thresholds, or fall back to a simpler model. Automating retraining with validation gates helps, but a human should review significant changes.

Likely follow-ups

- Why do labels often arrive late?
- Which drift alerts are really pipeline bugs?

🗺 **Drift monitoring**



Compare production feature distributions with a reference using PSI or statistical tests for data drift, and watch delayed accuracy and proxies for concept drift. Check upstream pipelines first, then retrain with gated validation.

```
evidently report --reference ref.parquet --current cur.parquet --out drift.html
```

```
python psi.py --feature age --ref ref.csv --cur cur.csv
```

certs.eknathalabs.com

Interview tip: Say many drift alerts turn out to be upstream data bugs.

📚 **Start here (official docs)**

- Hugging Face documentation

Q83 How do you reproduce a model training run months later?

MLOps

🎙 **Spoken answer · ~57 s**

Reproducibility needs every input captured. The code is pinned by Git commit. Data is versioned, with tools like DVC or lakehouse table snapshots, or immutable dated partitions, so I can retrieve exactly the training set. Environment is captured with a container image built from pinned dependencies and recorded by digest, with the CUDA and driver versions noted. Configuration and hyperparameters are stored in files or the tracking system, and random seeds are set and logged. Non-determinism from GPUs and parallelism can still cause small differences, so I record whether deterministic modes were used, and accept small numerical variation. The experiment tracker links all of these to the resulting artefact in the registry. I test reproducibility periodically by re-running an old job and comparing the metrics within a tolerance, because untested reproducibility is usually broken in some detail.

Likely follow-ups

- Why can GPU training be non-deterministic?
- How do you version large datasets?

Pin code by commit, version data with DVC or table snapshots, capture the environment by container digest and driver versions, log configuration and seeds, link everything in the tracker, and periodically re-run to verify.

```
dvc checkout
```

```
docker run --rm registry/train@sha256:abc123 python train.py --config cfg.yaml --seed 42
```

```
git rev-parse HEAD
```

Interview tip: Say you actually re-run an old job to test reproducibility.

📚 **Official docs**

- MLflow documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q84 Compare batch, real-time and streaming inference patterns.

MLOps

🎙 **Spoken answer · ~58 s**

Batch inference runs the model over a large dataset on a schedule, writing predictions to a store, for example nightly recommendations or risk scores. It is the simplest and cheapest, with no latency requirement, and it uses compute efficiently, but predictions can be stale. Real-time or online inference serves predictions per request over an API with tight latency, perhaps tens to hundreds of milliseconds, which needs always-on, autoscaled serving, online feature retrieval and careful monitoring, and is more expensive and complex. Streaming inference processes events continuously, for example fraud scoring on a transaction stream, combining windowed features with a model, and sits between the two. I choose by freshness needs: if predictions can be a few hours old, batch is far simpler. Many systems mix them, precomputing most results in batch and calling real-time inference only where fresh context matters.

Likely follow-ups

- When would you precompute and cache predictions?
- What makes online serving harder to operate?

Batch is simple and cheap but stale, real-time serves per request with strict latency and higher cost, and streaming scores events continuously. Choose by freshness need and often combine batch with selective real-time.

```
spark-submit batch_score.py --date 2026-10-20
```

```
kserve inference service apply -f model.yaml
```

Interview tip: Ask how fresh predictions need to be before choosing.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q85 What is KServe and how does it help serve models on Kubernetes?

MLOps

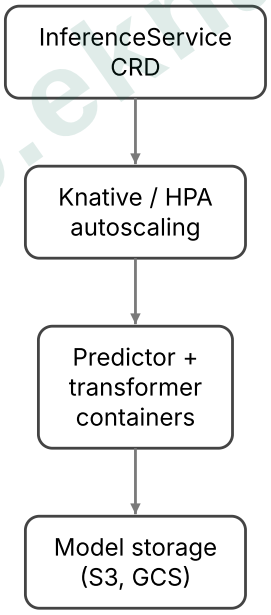
🎙 Spoken answer · ~59 s

KServe is a Kubernetes-native model serving platform. You declare an InferenceService custom resource that points to a model and a runtime, and KServe creates the deployment, networking, autoscaling and routing. It supports standard predictors for frameworks such as scikit-learn, XGBoost, PyTorch and TensorFlow, and runtimes for LLMs, such as vLLM and Hugging Face servers, behind a consistent inference protocol. Key features include autoscaling with Knative, including scale to zero for idle models, canary rollouts by splitting traffic between revisions, transformers for pre and post processing, explainers, model monitoring hooks, and ModelMesh for high-density serving of many small models. For a platform team it standardises how models are deployed, so data scientists need not write Kubernetes manifests, while the platform controls security, quotas and observability. The trade-offs are the additional components such as Knative or Istio, and a learning curve for troubleshooting.

Likely follow-ups

- What does scale to zero cost you?
- How does canary rollout work in KServe?

🗺 KServe



KServe turns an InferenceService resource into a managed deployment with autoscaling, canary traffic splitting, pre and post processing and many runtimes including LLM servers, standardising model serving on Kubernetes.

```
kubectl apply -f inference-service.yaml
```

```
kubectl get inferenceservice -n models
```

certs.eknathalabs.com

```
kubectl patch isvc churn -p '{"spec":{"predictor":{"canaryTrafficPercent":10}}}'
```

Interview tip: Mention scale to zero and the cold-start trade-off.

📚 **Official docs**

- KServe documentation

Q86 How do you orchestrate ML workflows with Kubeflow Pipelines, Argo or Airflow?

MLOps

🎙 **Spoken answer · ~61 s**

These tools define a workflow as a graph of steps with dependencies, run them on a schedule or trigger, retry failures and record the history. Kubeflow Pipelines is ML-specific: each step is a containerised component with typed inputs and outputs, artefact and metadata tracking, and caching of unchanged steps. Argo Workflows is a general Kubernetes-native engine in which each step is a pod, well suited for parallel fan-out jobs and CI-like flows. Airflow is a mature scheduler for data pipelines, strong for scheduled ETL with a large operator ecosystem, and is often used to trigger ML jobs. I choose based on existing skills and needs: Kubeflow or Argo if the team is Kubernetes-centric and wants container-native steps, Airflow if the organisation already runs it for data engineering. Whichever I choose, steps are idempotent, parameterised, resource-limited, and the pipeline definitions live in Git and go through review.

Likely follow-ups

- Why make steps idempotent?
- What does step caching save?

Kubeflow Pipelines offers typed ML components with artefact tracking and caching, Argo is a general Kubernetes-native DAG engine, Airflow is a mature data scheduler. Choose by existing skills; keep steps idempotent and definitions in Git.

```
argo submit --watch train.yaml -p dataset=2026-10
```

```
kfp run list
```

```
airflow dags trigger train_model
```

Interview tip: Say the choice follows existing skills, not fashion.

📚 **Official docs**

- Kubeflow documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q87 How do you apply CI/CD to machine learning, including what gets tested?

MLOps

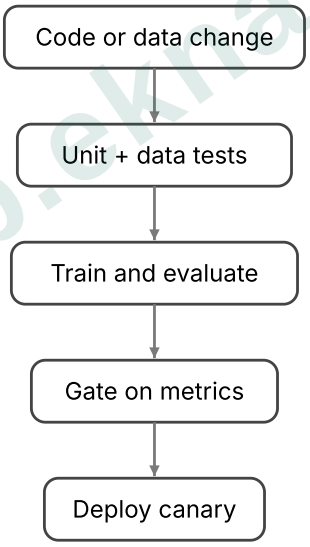
🎙 Spoken answer · ~61 s

I extend normal CI with ML-specific checks. On every pull request: linting and unit tests of data and feature code, schema and data validation tests on sample data, and a quick training smoke test on a small dataset to ensure the pipeline runs end to end. For significant changes, a full or scheduled training run produces a candidate model, which is evaluated automatically against a baseline on held-out data, with thresholds for accuracy, latency, fairness slices and size, and failing any threshold blocks promotion. Continuous delivery then packages the model and serving code into a versioned image, deploys it to staging, runs integration and load tests, then performs a canary rollout in production with automatic rollback on monitored metrics. Continuous training pipelines retrain on a schedule or on drift, using the same gates. Everything is reproducible, with data and model versions recorded in the release metadata.

Likely follow-ups

- What is a smoke test for training?
- Which thresholds block promotion?

🗺 CI/CD for ML



PR checks run unit tests, data validation and a training smoke test; candidate models are evaluated against baselines with gating thresholds; delivery uses staging, load tests and canary with rollback; retraining uses the same gates.

```
pytest tests/ -q
```

```
python train.py --smoke --rows 1000
```

```
python gate.py --candidate runs:/abc --baseline models:/churn@champion --min-auc 0.85
```

Interview tip: Name the evaluation gate; it replaces 'tests pass' for models.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Hugging Face documentation

Q88 How do you manage data quality and validate data in ML pipelines?

MLOps

🎙 **Spoken answer · ~56 s**

Poor data silently produces poor models, so I validate at pipeline boundaries. Schema checks confirm columns, types and nullability. Statistical checks verify ranges, distributions, uniqueness, cardinality and freshness against expectations or recent history. Referential and business rules catch impossible values, for instance negative ages. Tools such as Great Expectations, Pandera, TFX Data Validation or Soda express these as tests. Failures can block the pipeline, quarantine bad records, or alert, depending on severity. I also monitor labels for leakage and imbalance, and check training and serving data for skew. Data contracts agreed with upstream producers make schema changes visible before they break consumers. Findings are logged and linked to model versions, so when a model misbehaves I can look at the exact data quality at training time. Treat data tests as seriously as code tests.

Likely follow-ups

- What is a data contract?
- When should bad data block the pipeline?

Validate schema, distributions, freshness and business rules at pipeline boundaries with tools like Great Expectations, block or quarantine on failures, agree data contracts with producers, and log results with model versions.

```
great_expectations checkpoint run daily_orders
```

```
soda scan -d warehouse -c config.yml checks.yml
```

Interview tip: Say data tests deserve the same seriousness as code tests.

📚 **Official docs**

- Kubeflow documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q89 What is LLMOps and how does it differ from classical MLOps?

MLOps

🎙 Spoken answer · ~63 s

LLMOps adapts MLOps to applications built on large language models, where you usually consume a pre-trained model rather than train one from scratch. The central assets change: prompts, retrieval indexes, tool definitions and guardrail configurations replace much of the feature and training pipeline, and you version them like code. Evaluation is harder because outputs are open-ended text, so you rely on curated evaluation sets, rubric grading, LLM judges calibrated with human review, and online feedback. Cost is token-driven and varies per request, so metering and budgets are essential. Latency has new dimensions such as time to first token. Risks are different as well: hallucination, prompt injection, data leakage and unsafe content need guardrails and red teaming. Monitoring includes traces of chains and agents, quality signals and token usage. Fine-tuning, when used, is a smaller part, often with LoRA, and model providers update models underneath you, so pinning and regression testing matter.

Likely follow-ups

- Which assets does LLMOps version?
- Why do provider-side model updates matter?

🗺 MLOps vs LLMOps

Classical MLOps	LLMOps
Train your own models	Prompts, retrieval, evals
Feature stores, retraining	Token cost and guardrails

LLMOps versions prompts, indexes and tool definitions, evaluates open-ended output with curated sets and judges, meters tokens, adds guardrails and traces, and handles provider model updates through pinning and regression tests.

```
promptfoo eval
```

```
langfuse traces list --limit 20
```

```
git diff prompts/
```

Interview tip: Name the new assets: prompts, indexes, tools, guardrails.

📚 Official docs

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q90 How do you handle model and data governance, lineage and auditability?

MLOps

🎙 Spoken answer · ~61 s

Governance is the ability to answer who trained what, on which data, for what purpose, and who approved it. I capture lineage automatically in the pipeline: dataset versions and sources, code commit, parameters, environment, evaluation results and the approver, stored with each registered model. Access control limits who can read sensitive data, run training, promote models and deploy, with audit logs of those actions. Data governance covers consent, retention and deletion, personal data handling and licensing of training data, so a deletion request can be traced to the models affected. Model documentation, such as model cards, describes intended use, limitations, evaluation including fairness slices, and known risks. High-risk systems need human review, impact assessments and monitoring plans, aligned with frameworks like the NIST AI Risk Management Framework and regulations that apply, such as the EU AI Act. The platform's role is to make these records automatic.

Likely follow-ups

- What does a model card contain?
- How would you trace a deletion request to models?

Capture lineage automatically in the pipeline, enforce access control and audit logs, manage consent and retention, publish model cards, and align high-risk systems with frameworks such as NIST AI RMF and relevant regulation.

```
mlflow models describe -m models:/churn/7
```

```
git log --format='%H %an %s' -- pipelines/
```

Interview tip: Say the platform should make governance records automatic.

📚 Official docs

- NIST AI Risk Management Framework

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q91 How do you control and cache costs of ML training jobs on Kubernetes?

MLOps

🎙 **Spoken answer · ~65 s**

I combine scheduling, infrastructure and engineering practices. For compute: use spot or preemptible instances for fault-tolerant training with frequent checkpoints and automatic resume, right-size GPU requests, use queueing with fair share so GPUs do not sit idle, and shut down idle notebooks. For efficiency: mixed precision, efficient data loading, early stopping and hyperparameter search strategies like Bayesian optimisation instead of exhaustive grids. For repeated work: cache pipeline steps whose inputs did not change, reuse preprocessed datasets and pre-trained checkpoints, and avoid re-downloading data and images with node-level caches. For visibility: tag jobs with team and project, measure cost per experiment and per training run, set quotas and budgets, and report to teams. The cheapest training run is the one you do not need to repeat, so reproducibility and caching are also cost controls. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- How do checkpoints enable spot usage?
- What does step caching skip?

Use spot with checkpoints, right-size GPUs, queue and fair-share, efficient training practices, cached pipeline steps and datasets, and per-team cost tags and budgets. Reproducibility and caching are cost controls.

```
kubectl get nodes -l karpenter.sh/capacity-type=spot
```

```
python train.py --resume-from s3://ckpt/latest
```

```
kubectl get workloads -n team-a
```

Interview tip: Link reproducibility to cost: avoid repeating work.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q92 How do you build an evaluation set for an LLM application?

Evaluation

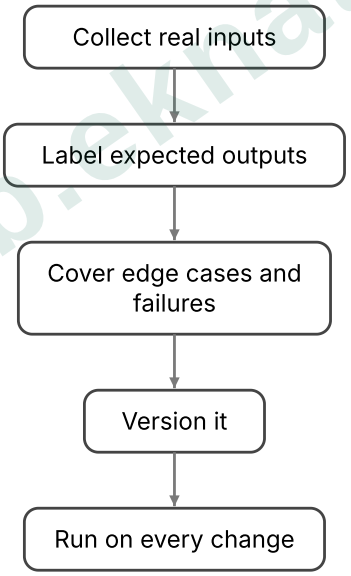
🎙 Spoken answer · ~62 s

I start from real use. I collect representative inputs from production logs, support tickets, or domain experts, covering common requests, edge cases, ambiguous questions, adversarial prompts, and known past failures. For each, I define what a good output means: a reference answer, a set of required facts, a schema, a rubric, or a pass or fail check in code. I get subject matter experts to label a sample, and agree on grading guidelines so different reviewers are consistent. The set is split by category so I can see where the system is weak, and I keep a frozen set for regression and a growing one fed by production failures. Size depends on the variance and importance, but even a hundred well-chosen cases beat anecdotes. I version the set, avoid leaking it into prompts or fine-tuning data, and refresh it as the product evolves, because stale tests give false comfort.

Likely follow-ups

- How do you avoid leakage into prompts?
- How big should an evaluation set be?

🗺 Build an evaluation set



Collect real inputs including edge and adversarial cases and past failures, define expected outputs or rubrics with expert agreement, slice by category, version it, keep it out of training and prompts, and grow it from production failures.

```
jq -c 'select(.thumbs_down==true)' logs.jsonl > candidates.jsonl
```

```
promptfoo eval -c eval.yaml --output results.json
```

Interview tip: Say you feed production failures back into the set.

certs.eknathalabs.com

InterviewHub - interviewhub.eknathalabs.com

📚 **Start here (official docs)**

- Hugging Face documentation

Q93 What are LLM-as-a-judge evaluations and what are their pitfalls?

Evaluation

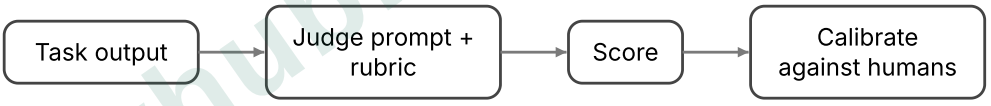
🎙 **Spoken answer · ~59 s**

LLM-as-a-judge uses a strong model to grade the outputs of another model against a rubric, for example scoring helpfulness, faithfulness to sources or following a style guide. It scales far beyond human review and is useful for open-ended outputs where exact matching fails. The pitfalls are real. Judges show biases: preference for longer answers, for their own style or model family, position bias when comparing two answers, and sensitivity to prompt wording. They can be wrong or inconsistent, and may miss subtle factual errors. I mitigate by using clear, specific rubrics with examples, requiring reasoning before a score, randomising answer order in pairwise comparisons, using a different, strong model family as judge, and calibrating against human-labelled samples to measure agreement. I use judge scores as a signal alongside programmatic checks, and I periodically audit them, rather than treating them as ground truth.

Likely follow-ups

- What is position bias?
- How do you measure judge agreement with humans?

🗺 **LLM as judge**



A strong model grades outputs against a rubric, scaling review but with bias toward length and style, position effects and inconsistency. Use specific rubrics, randomised order, a different judge family, calibration against human labels and audits.

```
python judge.py --rubric rubric.md --pairs pairs.jsonl --randomize-order
```

```
python agreement.py --human labels.csv --judge scores.csv
```

Interview tip: Say you calibrate the judge against human labels.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q94 Which metrics do you use to evaluate generation quality beyond accuracy? Evaluation

🎙 Spoken answer · ~58 s

It depends on the task, but I group them. Correctness: exact match or F1 for extraction, pass rate of unit tests for code, accuracy for classification, and factual consistency scores. Faithfulness or groundedness for RAG, meaning every claim is supported by the supplied context. Relevance and completeness: does the answer address the question fully. Format validity: percentage of outputs that parse against the schema. Safety: toxicity, policy violations, leakage of personal data, and refusal rates, both over-refusal and under-refusal. Style and tone adherence. Operational metrics: latency, tokens and cost per request. User-level metrics: thumbs up and down, edits made by users, task completion and escalation rates. Overlap metrics such as BLEU or ROUGE are weak for open-ended text and I use them only as rough signals. I report a balanced scorecard, because optimising one number can harm another.

Likely follow-ups

- Why are BLEU and ROUGE weak for LLM outputs?
- What is over-refusal?

Combine task correctness, faithfulness, relevance, format validity, safety including over-refusal, style, operational cost and latency, and user-level signals. Avoid optimising a single number.

```
python metrics.py --pred out.jsonl --gold gold.jsonl --metrics f1 schema_valid faithfulness
```

```
jq '[.[] | .tokens] | add/length' results.json
```

Interview tip: Present a scorecard, not a single metric.

📚 Official docs

- OpenTelemetry: GenAI semantic conventions

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q95 How do you run online evaluation, A/B tests and monitor quality in production for LLM apps?

Evaluation

🎙 Spoken answer · ~58 s

Offline tests are necessary but not sufficient, because real users behave differently. For online evaluation, I log traces with the prompt version, model, retrieved context and output, with privacy controls. I collect explicit feedback, such as thumbs and comments, and implicit signals like regenerations, edits, copy actions, abandonment and escalation to a human. I sample production traffic for automated scoring by an LLM judge and for periodic human review, and I alert on drops. For changes, I run A/B tests or interleaving between prompt or model variants, with predefined primary metrics and guardrail metrics for cost, latency and safety, and enough traffic to reach statistical significance, taking care with novelty effects. Findings flow back into the evaluation set. I also monitor drift in the input distribution, since new topics can expose gaps, and keep a rollback ready for any variant.

Likely follow-ups

- What are guardrail metrics in an A/B test?
- How do you sample for human review?

Log traces with versions, gather explicit and implicit feedback, score sampled traffic with judges and humans, A/B test variants with primary and guardrail metrics, and feed findings back into the evaluation set.

```
langfuse scores create --trace-id abc --name helpfulness --value 4
```

```
python ab_analyze.py --exp prompt_v4_vs_v3 --metric thumbs_up_rate
```

Interview tip: Mention guardrail metrics so an A/B win does not hide a cost or safety loss.

📚 Official docs

- OpenTelemetry: GenAI semantic conventions

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q96 How do you red team an LLM application?

Evaluation

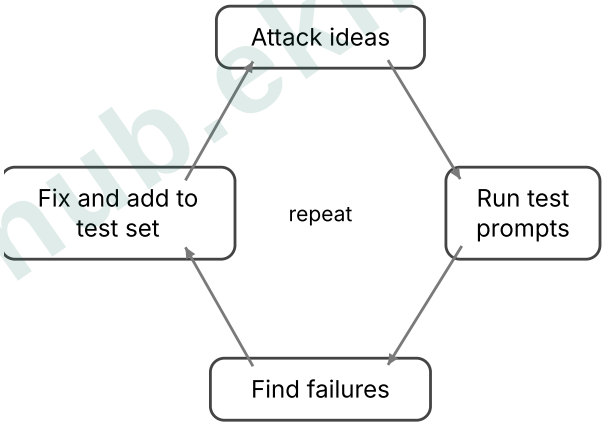
🎙 Spoken answer · ~59 s

Red teaming is structured adversarial testing to find failures before attackers or users do. I define the scope and threat model: what harms matter, such as data leakage, policy-violating content, unauthorised actions through tools, and brand damage. Then I generate attacks across categories: direct and indirect prompt injection, jailbreaks, requests for secrets or system prompts, attempts to access other users' data, harmful or biased content, and abuse of tools, with both manual creativity from experts and automated tools like garak or PyRIT that produce many variants. I run them against the full application, including retrieval and tools, not just the bare model, record successes with severity, and fix through layered mitigations: prompt changes, input and output filters, permission reductions, and monitoring. Successful attacks become regression tests. I repeat red teaming on every significant change and periodically, because new attack techniques keep appearing.

Likely follow-ups

- Why test the full application and not only the model?
- How do successful attacks become regression tests?

🗺 Red team cycle



Define threat model and harms, attack with manual and automated techniques across injection, jailbreak, leakage and tool abuse, test the whole application, fix with layered mitigations, and add successes as regression tests; repeat regularly.

```
garak --model_type rest --model_name app --probes promptinject,leakreplay
```

```
python pyrit_run.py --target app --scenario data_exfil
```

Interview tip: Say attacks become regression tests.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- OWASP Top 10 for LLM Applications

Q97 How do you benchmark an inference service for latency and throughput?

Evaluation

🎙 **Spoken answer · ~62 s**

I replicate real traffic as closely as I can. First I characterise the workload from production logs: the distribution of prompt lengths and output lengths, request arrival pattern, concurrency, and share of cached prefixes. Then I use a load generator that replays or samples that distribution, with open-loop arrivals at target rates, not only closed-loop fixed concurrency, because real users do not wait for each other. I increase the request rate in steps and record time to first token, inter-token latency, end-to-end latency percentiles at p50, p95 and p99, throughput in tokens per second, error rate, and GPU metrics. The key result is the saturation point: the highest rate at which latency still meets the SLO. I run long enough to include warm-up and steady state, repeat for each configuration, such as quantisation or batch limits, and document hardware and software versions so results are comparable and reproducible.

Likely follow-ups

- Why use open-loop load?
- What is the saturation point?

Model the production distribution of prompt and output lengths and arrival patterns, drive open-loop load in steps, record TTFT, inter-token latency percentiles, throughput and errors, and find the rate that still meets the SLO.

```
python benchmark_serving.py --backend vllm --dataset-name sharegpt --request-rate 4 --num-prompts 500
```

```
genai-perf profile -m model --concurrency 16
```

Interview tip: Say the saturation point at the SLO is the number you report.

📚 **Official docs**

- KServe documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q98 How do you compare two models for a task fairly?

Evaluation

🎙 Spoken answer · ~58 s

I control everything except the model. Use the same evaluation set, the same prompt where sensible, and separately tuned prompts where each model deserves its best effort, and report both. Fix the sampling parameters, and for stochastic settings run multiple samples and average, with confidence intervals so I can tell real differences from noise. Grade with the same rubric or judge, blind to which model produced the answer, and use pairwise comparisons with randomised order for subjective tasks. Slice the results by category, difficulty and input length to see where each model is better, because the overall average hides trade-offs. Compare the total picture: quality, latency, throughput, cost per task, context limits, safety behaviour, licensing, data residency and operational complexity. Finally, validate the winner in a shadow or canary deployment, since public leaderboards rarely match your own data and constraints.

Likely follow-ups

- Why report both shared and tuned prompts?
- How do you know a difference is significant?

Same evaluation set, controlled parameters, multiple samples with confidence intervals, blind grading, results sliced by category, and a full comparison of cost, latency, safety and licensing. Confirm in shadow or canary.

```
python compare.py --a model-a --b model-b --set eval.jsonl --samples 5 --seed 1
```

```
scipy.stats.bootstrap((diff,), np.mean)
```

Interview tip: Say your own evaluation beats public leaderboards for decisions.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q99 How do you test for bias and fairness in an AI system?

Evaluation

🎙 Spoken answer · ~62 s

I begin by defining the context: who is affected, which groups matter, what a harmful outcome looks like, and which fairness notion is appropriate, such as equal error rates or equal opportunity across groups, because different definitions can conflict. Then I build evaluation data that includes relevant slices, either from labelled attributes where it is lawful and appropriate, or from counterfactual tests, where I change only a name, gender or dialect in an otherwise identical input and check whether the output changes. I measure performance and error rates per slice, and for generative models test for stereotyping, toxicity and differences in tone or refusal across groups. Findings guide mitigations: better data, prompt and policy changes, post-processing, or restricting the use case. I document the results in a model card, monitor in production, and involve legal and domain experts, since fairness is a socio-technical question, not only a metric.

Likely follow-ups

- What is a counterfactual test?
- Why can fairness definitions conflict?

Define affected groups and the fairness notion, test per-slice error rates and counterfactual variants, check generative outputs for stereotyping and refusal differences, mitigate, document in a model card, monitor, and involve legal and domain experts.

```
python fairness.py --pred preds.csv --attr group --metric equal_opportunity
```

```
python counterfactual.py --template tmpl.txt --names names.txt
```

Interview tip: Say fairness is socio-technical, not only a metric.

📚 Official docs

- NIST AI Risk Management Framework

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q100 What is regression testing for prompts and models, and how do you automate it in CI?

Evaluation

🎙 **Spoken answer · ~63 s**

Because changing a prompt or switching a model can break behaviours that previously worked, I keep a regression suite of cases that must continue to pass. It includes golden examples with expected outputs or assertions, past bugs, safety cases, format checks and cost and latency limits. In CI, a change to a prompt, tool definition, retrieval configuration or model version triggers the suite: the system runs against the cases, assertions check deterministic properties like valid JSON, required fields, forbidden content and tool call correctness, and a judge scores subjective quality against a rubric. Results are compared with the baseline stored from the main branch, and the pull request fails if quality drops beyond a tolerance or cost rises sharply. To manage cost and flakiness, I use a small fast subset on every commit and the full suite nightly, set low temperature or multiple samples, and cache results where inputs are unchanged.

Likely follow-ups

- How do you handle flaky LLM tests?
- What is a tolerance threshold?

Maintain golden cases, past bugs and safety cases; on any prompt, tool, retrieval or model change run assertions and judge scoring against the main-branch baseline, failing on quality drops or cost spikes. Use a fast subset per commit and full runs nightly.

```
promptfoo eval --ci --fail-on-error
```

```
pytest tests/llm -q --maxfail=3
```

Interview tip: Say you compare against a stored baseline with a tolerance.

📚 **Official docs**

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q101 How do you measure and reduce hallucinations in a customer-facing assistant?

Evaluation

🎙 **Spoken answer · ~61 s**

I define hallucination operationally: statements not supported by the provided sources or by verified facts. I measure it with a test set of questions that includes answerable ones, unanswerable ones where the correct behaviour is to say it does not know, and traps that tempt the model to invent details. For RAG systems, I compute faithfulness by checking each claim against the retrieved context using a judge or an entailment model, and I sample production answers for human review. Reduction strategies work in layers: better retrieval, instructions that require citations and allow uncertainty, shorter and cleaner context, low temperature, constrained output formats, tool use for facts such as order status, and post-generation verification that removes or flags unsupported claims. For high-stakes topics, I add human review or restrict the assistant to approved answers. I track the rate over time as a key metric, and alert on increases.

Likely follow-ups

- Why include unanswerable questions in the test set?
- How do you verify claims automatically?

Define hallucination as unsupported claims, measure with answerable, unanswerable and trap questions plus faithfulness scoring and human sampling, and reduce it with better retrieval, citations, I-don't-know behaviour, tools, verification and human review for high stakes.

```
ragas evaluate --metrics faithfulness --dataset qa.jsonl
```

```
python verify_claims.py --answers out.jsonl --sources ctx.jsonl
```

Interview tip: Say unanswerable test questions measure honesty.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q102 Walk through the OWASP Top 10 for LLM applications. Which risks matter most for a platform team?

Security

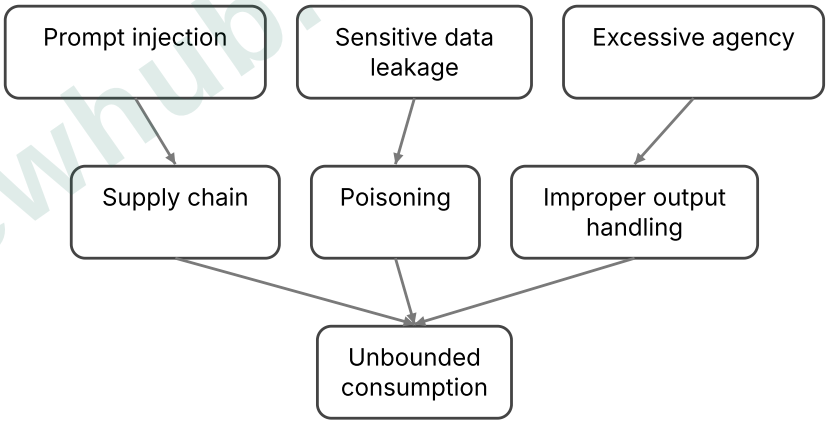
🎙 Spoken answer · ~65 s

The OWASP list names the main risk categories for LLM applications. Prompt injection, direct or indirect, manipulating the model's behaviour. Sensitive information disclosure, where the model reveals secrets or personal data from training, context or tools. Supply chain risks from third-party models, datasets, plugins and libraries. Data and model poisoning, tampering with training or retrieval data. Improper output handling, where model output is passed unchecked to a browser, shell or database. Excessive agency, giving the model too many permissions or autonomy. System prompt leakage. Weaknesses in vectors and embeddings, such as access control gaps in retrieval. Misinformation, meaning hallucination with consequences. And unbounded consumption, covering denial of service and runaway costs. For a platform team, I prioritise excessive agency, improper output handling, access control in retrieval, supply chain and unbounded consumption, because these are controlled by infrastructure design, permissions, validation, scanning and quotas. The exact list is updated over time, so I check the current edition.

Likely follow-ups

- What is improper output handling?
- Why is excessive agency an infrastructure problem?

🗺 OWASP LLM risk areas



Prompt injection, sensitive disclosure, supply chain, poisoning, improper output handling, excessive agency, prompt leakage, retrieval and embedding weaknesses, misinformation and unbounded consumption. Platform teams own agency limits, validation, retrieval access, supply chain and quotas.

```
garak --model_type rest --probes promptinject,leakreplay,xss
```

```
trivy image registry/llm-api:1.4
```

Interview tip: Say you check the current edition, since the list evolves.

certs.eknathalabs.com

📚 **Official docs**

- OWASP Top Ten
- OWASP Top 10 for LLM Applications
- NIST AI Risk Management Framework

Q103 How do you protect sensitive data and PII when using LLMs?

Security

🎙 **Spoken answer · ~66 s**

I start with data classification: what may be sent to which model and where. Then technical controls. Minimise: send only the fields needed, and mask or tokenise personal data before it reaches the model, using detectors such as Presidio, then restore it afterwards where required. Choose the deployment to match sensitivity, such as private endpoints, a region with data residency, a self-hosted model, or a provider with contractual guarantees of no training on my data and defined retention. Encrypt in transit and at rest, and restrict who can access prompts and logs, since logs often contain the most sensitive content, with short retention and redaction. Enforce access control in retrieval so users only see data they are permitted to. Add output filters for leakage, and keep audit trails. Finally, train developers, use data loss prevention for developer tools and browsers, and make the approved path easier than the shortcut, so people do not paste data into unapproved services.

Likely follow-ups

- What contractual terms matter with a provider?
- How do you handle PII in logs?

Classify data, minimise and mask PII before sending, pick deployment by sensitivity with residency and no-training terms, encrypt and tightly control prompt logs with short retention, enforce retrieval ACLs, filter outputs, and make the approved path easy.

```
presidio-analyzer --text 'Call me at 555-0100' --language en
```

```
aws bedrock put-model-invocation-logging-configuration --cli-input-json file://logging.json
```

Interview tip: Say logs of prompts are often the largest leak risk.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q104 How do you secure the AI supply chain: models, datasets, dependencies and containers?

Security

🎙 **Spoken answer · ~63 s**

Models are code-like artefacts that can contain malicious payloads, for example pickle files that execute code on load, so I prefer safe formats such as safetensors, and scan model files. I pull models only from trusted sources, pin versions by hash, verify signatures or checksums, and mirror approved models into an internal registry, so production never downloads arbitrary files from the internet. Datasets get provenance tracking, licence checks and validation against poisoning. Dependencies and container images go through the usual controls: SBOMs, vulnerability scanning, pinned versions, signed images and admission policies that only allow signed artefacts. The ML ecosystem moves fast and ships large dependency trees, so I watch advisories for serving frameworks and GPU drivers. I restrict network egress from training and inference pods, run them with minimal privileges, and keep an inventory of which model versions run where, so I can respond when a vulnerability or licence issue appears.

Likely follow-ups

- Why is pickle dangerous?
- How do you restrict model downloads in production?

Prefer safetensors, scan models, pull from trusted sources pinned by hash and mirrored internally, track dataset provenance, apply SBOMs, signing and admission policies to images, restrict egress and keep an inventory of model versions.

```
modelscan -p model.pkl
```

```
cosign verify registry/llm-server:1.4 --key cosign.pub
```

```
sha256sum model.safetensors
```

Interview tip: Say pickle files can execute code on load.

📚 **Official docs**

- GitHub Actions: Security hardening

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q105 How do you prevent an LLM agent or application from leaking secrets or system prompts?

Security

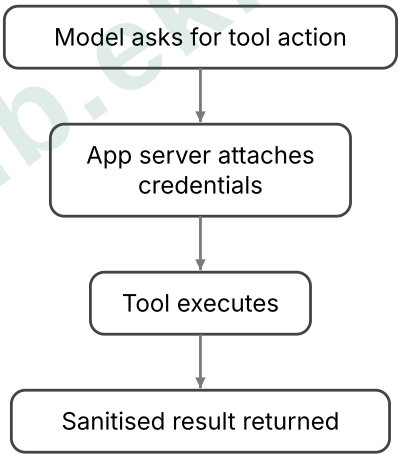
🎙 Spoken answer · ~63 s

I assume anything the model can see, it can be tricked into revealing, so the main defence is to keep secrets out of the context entirely. API keys, credentials and tokens belong in the tool execution layer, not in prompts: the model requests an action and the application attaches credentials server-side, scoped to the user. The system prompt should contain behaviour instructions, not confidential logic or keys, and I treat it as potentially extractable, so I do not rely on it for security. Output filters scan for patterns of secrets and for verbatim system prompt text, and egress controls prevent exfiltration through links, images or tool calls to unapproved domains. Tool results are filtered before returning to the model, to avoid passing secrets from internal systems. I also test with extraction prompts during red teaming and monitor for suspicious outputs. Rotation and short-lived credentials limit the damage when something does leak.

Likely follow-ups

- Why should the system prompt not hold secrets?
- How can output exfiltration happen through markdown links?

🗺 Keep secrets out of context



Keep secrets out of the model context, attach credentials in the tool layer, treat the system prompt as extractable, filter outputs and tool results, restrict egress, red team extraction attempts and use short-lived credentials.

```
grep -R 'sk-' prompts/ && echo 'secret in prompt!'
```

```
trufflehog filesystem prompts/
```

Interview tip: Say design as if the prompt will be seen.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

📚 **Official docs**

- OWASP Top 10 for LLM Applications
- Prompt engineering overview (Claude docs)

Q106 How do you handle prompt injection through tool outputs, web pages and documents the agent reads?

Security

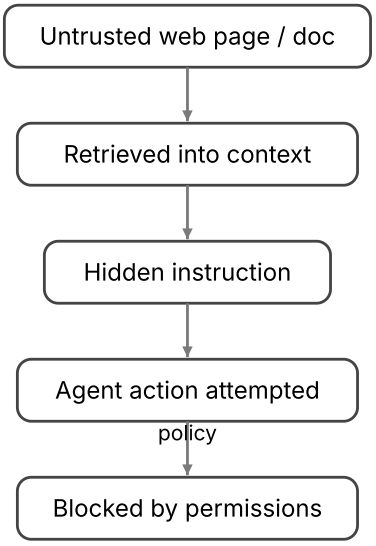
🎙 **Spoken answer · ~63 s**

Indirect injection is hard because the model cannot reliably tell data from instructions. I combine design and controls. Mark and delimit untrusted content, and tell the model that content inside the markers is data, never instructions, knowing this only reduces the risk. Limit what the agent can do after reading untrusted content, a principle sometimes described as separating privileged and untrusted contexts: for example, an agent that reads arbitrary web pages should not also hold credentials or the ability to send data outward in the same session. Require human confirmation for sensitive actions and show the user exactly what will happen. Sanitise content, strip hidden text and active markup, and block data exfiltration channels such as arbitrary URL fetches and image rendering. Use a separate classifier to detect injection attempts. Log tool calls and review anomalies. The goal is containment: even if the model is manipulated, the blast radius is small.

Likely follow-ups

- What is the lethal trifecta idea?
- How do you present confirmations to users?

🗺 **Indirect prompt injection**



Delimit untrusted content, separate privileged and untrusted contexts, require confirmation for sensitive actions, sanitise content, block exfiltration channels, run an injection classifier and log tool calls. Aim for containment.

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
kubectl apply -f networkpolicy-agent-egress-allowlist.yaml
```

```
python sanitize.py --strip-hidden-text page.html
```

Interview tip: Say the goal is containment, not perfect detection.

📚 **Official docs**

- OWASP Top Ten
- Model Context Protocol
- OWASP Top 10 for LLM Applications

Q107 How do you secure model serving endpoints against abuse and denial of service?

Security

🎙 **Spoken answer · ~60 s**

Inference is expensive, so abuse can cause both outages and large bills. I put the endpoint behind an API gateway with authentication, per-key and per-user rate limits, and quotas on tokens, not just requests, since one request can be very long. I enforce limits on input size, output length and context, timeouts, and concurrency, and reject oversized or malformed requests early. Queue limits with load shedding keep the system responsive under overload, with priorities for important tenants. I protect against extraction and abuse patterns: monitoring for unusual query volumes, systematic probing to clone a model, and automated scraping. Network controls include TLS, private endpoints, and WAF rules. Budgets with alerts and automatic cut-offs prevent runaway costs, including from a bug in our own retry logic. Audit logs record who called what. Autoscaling should have a hard upper limit, so an attack cannot scale spending indefinitely.

Likely follow-ups

- Why limit tokens and not only requests?
- What is a model extraction attack?

Authenticate through a gateway, rate limit and quota by tokens, cap input, output and concurrency, shed load, monitor for scraping and extraction, use TLS and private endpoints, and set budgets and autoscaling ceilings.

```
limit_req_zone $binary_remote_addr zone=llm:10m rate=5r/s;
```

```
curl -s $GW/admin/usage?key=team-a
```

```
kubectl get hpa llm -o jsonpath='{.spec.maxReplicas}'
```

Interview tip: Say token quotas and hard autoscaling ceilings.

certs.eknathalabs.com

📚 **Start here (official docs)**

- Hugging Face documentation

Q108 What is model poisoning and how do you defend against it in training and RAG?

Security

🎙 **Spoken answer · ~63 s**

Poisoning means an attacker corrupts the data a model learns from, or retrieves from, to change its behaviour: inserting backdoor triggers that produce attacker-chosen outputs, degrading accuracy, or planting false or malicious content in a knowledge base. In training, the risk comes from scraped web data, user-submitted data used for fine-tuning, and third-party datasets. Defences include trusted data sources with provenance and integrity checks, validation and anomaly detection on new data, deduplication and outlier filtering, limiting who can contribute to training sets, holding out clean evaluation sets that include trigger tests, and monitoring for behaviour changes after each training run. In RAG, I control write access to indexed sources, scan content on ingestion, track provenance and trust levels, and can quickly remove a document and its chunks. For both, I keep versioned data so I can roll back, and I treat user feedback loops with caution, since attackers can influence them.

Likely follow-ups

- What is a backdoor trigger?
- How do you roll back a poisoned index?

Poisoning corrupts training or retrieval data to change behaviour. Defend with trusted, provenance-tracked sources, anomaly checks, restricted contributors, clean evaluation and trigger tests, ingestion scanning, versioned data and quick removal.

```
dvc diff HEAD~1 -- data/
```

```
python scan_ingest.py --src new_docs/ --check injection pii
```

```
git revert <bad-data-commit>
```

Interview tip: Mention versioned data for rollback.

📚 **Official docs**

- OWASP Top 10 for LLM Applications

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q109 How do you apply authentication and authorisation for AI agents acting on behalf of users?

Security

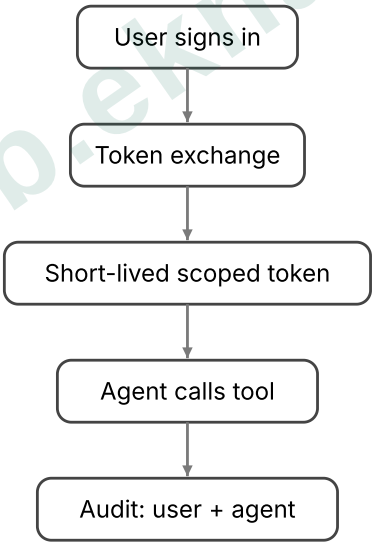
🎙 Spoken answer · ~62 s

The agent needs an identity and a delegated, limited authority. I authenticate the user in the normal way, then give the agent short-lived credentials scoped to that user's permissions and the specific task, using token exchange or on-behalf-of flows, so the agent can never do more than the user could, and often does less. For service-to-service calls, each agent has its own workload identity with least-privilege roles. Authorisation is enforced by the target systems and a policy layer, not by the prompt: each tool call is checked against the user and agent context, with an allow list of operations. Sensitive operations need step-up approval from the user. Credentials are never exposed to the model. Audit logs record both identities, the user and the agent, along with the request and result. I also design for revocation, so a compromised session or agent can be cut off immediately without affecting others.

Likely follow-ups

- What is an on-behalf-of token exchange?
- How do you log both user and agent identity?

🗺 Delegated agent identity



Give agents their own identity and short-lived, user-scoped delegated credentials through token exchange, enforce authorisation in target systems and a policy layer, require step-up approval for sensitive actions, never expose credentials to the model, log both identities and support revocation.

```
curl -X POST $STS/token -d grant_type=urn:ietf:params:oauth:grant-type:token-exchange -d
subject_token=$USER_TOKEN -d scope=tickets.read
```

```
opa eval -d policy.rego -i input.json 'data.agent.allow'
```

Interview tip: Say the agent must never have more authority than the user.

certs.eknathalabs.com
InterviewHub - interviewhub.ekna

📚 **Start here (official docs)**

- Hugging Face documentation

Q110 What are the privacy and compliance considerations for AI systems, such as GDPR and the EU AI Act?

Security

🎙 **Spoken answer · ~70 s**

Privacy law applies to AI as it does to any processing of personal data. Under GDPR I need a lawful basis and a clear purpose, data minimisation, transparency to users, retention limits, security measures, and respect for rights such as access, correction and deletion, which is hard once data is embedded in a trained model, so I prefer designs where personal data stays in retrievable stores rather than model weights. Automated decisions with significant effects need human oversight and the ability to contest them. Data transfers and processors need contracts and assessments. The EU AI Act classifies systems by risk, with prohibited practices, strict obligations for high-risk uses such as hiring or credit, including risk management, data governance, documentation, logging, human oversight and accuracy, and transparency duties for chatbots and generated content. I work with legal and privacy teams early, keep records of processing and model documentation, and build logging and review features into the platform. This is not legal advice; requirements depend on jurisdiction and use case.

Likely follow-ups

- Why is deletion hard for trained models?
- What counts as a high-risk use?

Apply GDPR principles: purpose, minimisation, transparency, retention, security and data rights, preferring retrievable stores over weights. Prepare for the EU AI Act risk tiers with documentation, logging and human oversight, and involve legal early.

```
python dsar_export.py --user 123 --stores postgres,vectordb
```

```
jq '.records[] | select(.purpose==null)' processing_register.json
```

Interview tip: Note that you are not giving legal advice and involve legal and privacy teams.

📚 **Official docs**

- NIST AI Risk Management Framework

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q111 How do you monitor and detect abuse or attacks against an LLM application in production? Security

🎙 Spoken answer · ~58 s

I treat it like application security with new signals. I log prompts, retrieved context, tool calls and outputs, with redaction, and tag them with user, tenant and session IDs. Detection rules and models look for known injection phrases, jailbreak patterns, attempts to extract the system prompt, very high token usage, repeated refusals, unusual tool call sequences, access to unusual resources, and outputs containing secrets or personal data. I baseline normal behaviour per tenant, and alert on deviations such as sudden volume increases or a new geography. Guardrail trigger rates are tracked over time. Alerts feed into the security operations process with runbooks for responses like throttling a user, revoking credentials, disabling a tool, or blocking a pattern. Sampled conversations are reviewed by humans regularly. Findings from incidents update the guardrails and the red team test set, so detection improves continuously.

Likely follow-ups

- What baselines would you compute per tenant?
- How do you respond to an active abuser?

Log redacted traces with identities, detect injection patterns, extraction attempts, token spikes, unusual tool sequences and secret leakage, baseline per tenant, alert into SecOps with runbooks, and feed incidents back into guardrails and tests.

```
jq 'select(.guardrail=="injection")' traces.jsonl | wc -l
```

```
curl -s $PROM/api/v1/query?query=sum(rate(llm_tokens_total[5m]))by(tenant)
```

Interview tip: Say incidents update guardrails and the red-team set.

📚 Official docs

- OpenTelemetry: GenAI semantic conventions

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q112 How do you isolate and sandbox code execution by an AI agent?

Security

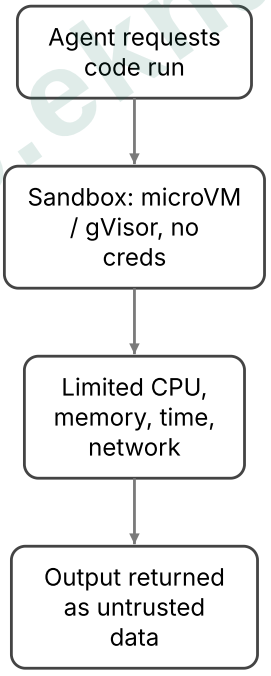
🎙 Spoken answer · ~64 s

Code written by a model must be treated as untrusted. I run it in a sandbox with strong isolation: a container with a hardened profile is a minimum, and for stronger guarantees a microVM such as Firecracker or a user-space kernel like gVisor. The sandbox has no ambient credentials, a read-only base file system with a small writable workspace, no or tightly allow-listed network access, CPU, memory, disk and time limits, and no access to the host or cluster API. Each session gets a fresh environment destroyed afterwards, so state cannot leak between users. Dependencies are installed from an internal mirror. Output sizes are limited, and results are treated as untrusted data when passed back to the model. I log the executed code and outputs. For Kubernetes, I use a dedicated node pool and namespace with network policies, pod security restricted profile, and runtime classes. Testing includes attempts to escape or reach internal services.

Likely follow-ups

- Why a microVM rather than a plain container?
- How do you restrict network access?

🗺 Code execution sandbox



Run generated code in a hardened sandbox, preferably a microVM or gVisor, with no credentials, restricted or no network, resource limits, a fresh disposable environment per session, dedicated nodes and network policies, and logged execution.

```
kubectl apply -f runtimeclass-gvisor.yaml
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

```
securityContext:\n  runAsNonRoot: true\n  readOnlyRootFilesystem: true\n
allowPrivilegeEscalation: false
```

```
kubectl apply -f networkpolicy-deny-all.yaml
```

Interview tip: Say model-written code is untrusted by default.

📚 **Start here (official docs)**

- Hugging Face documentation

Q113 What is excessive agency and how do you design against it?

Security

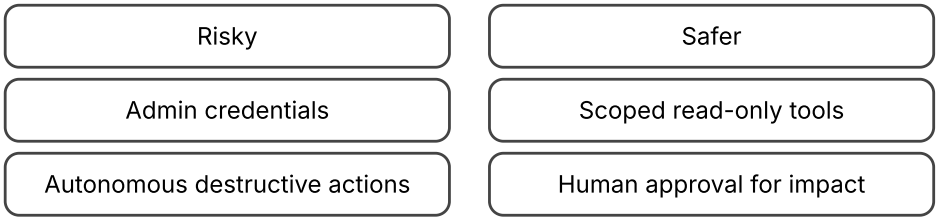
🎙 **Spoken answer · ~66 s**

Excessive agency is when an LLM-based system has more functionality, permissions or autonomy than the task needs, so that mistakes or manipulation lead to harmful actions. Examples are an email assistant that can delete mail and send as the user when it only needed to read, an agent with a database admin credential, or an agent that executes plans without any review. I design against it with three principles. Minimise functionality: expose only the tools needed, with narrow operations instead of general shells. Minimise permissions: scoped, short-lived credentials, read-only by default, restricted to the user's data and to specific environments. Minimise autonomy: require human approval for high-impact or irreversible actions, with rate limits and spending caps, and add undo or dry-run for others. Complement these with logging, anomaly detection and a kill switch. I test by asking what the worst case is if the model is fully controlled by an attacker, and reducing the answer until it is acceptable.

Likely follow-ups

- What actions need human approval?
- How do you do a worst-case analysis?

🗺 **Excessive agency**



Excessive agency means too many tools, permissions or autonomy. Minimise functionality, permissions and autonomy, add approvals, limits, logging and a kill switch, and test the worst case if the model is fully attacker-controlled.

```
kubectl auth can-i --list --as=system:serviceaccount:ai:agent
```

InterviewHub - interviewhub.eknathalabs.com

```
aws iam simulate-principal-policy --policy-source-arn $ARN --action-names s3:DeleteObject
```

Interview tip: Frame the test as: what if an attacker controls the model?

📚 **Official docs**

- OWASP Top 10 for LLM Applications

Q114 How do you secure the training and fine-tuning environment and the resulting model artefacts? Security

🎙 **Spoken answer · ~61 s**

The training environment holds valuable data and produces valuable models, so I treat it as a high-security zone. Access to datasets is governed by least privilege, with audit logs, and sensitive data is encrypted and ideally kept in a controlled enclave or virtual private cloud without direct internet access. Training jobs run in isolated namespaces with network policies, pinned and scanned images, and allowed package mirrors. Secrets and cloud credentials are short-lived and provided through workload identity. Model artefacts are stored encrypted with versioning, integrity hashes and signatures, and promoted through a registry with controlled access, so only approved models can be deployed, and weights are protected against theft because they represent intellectual property. Notebook environments are a common weak point, so I enforce idle shutdown, no persistent credentials and egress controls. Finally, I monitor for unusual data downloads or model exports, and keep logs for audit.

Likely follow-ups

- Why protect model weights as sensitive assets?
- What makes notebooks risky?

Least-privilege dataset access with audit, isolated namespaces and egress controls, scanned images, workload identity, encrypted versioned signed artefacts in a controlled registry, hardened notebooks and monitoring of unusual exports.

```
kubectl apply -f networkpolicy-training-egress.yaml
```

```
aws s3api put-bucket-encryption --bucket models --server-side-encryption-configuration
file://sse.json
```

```
cosign sign --key kms://... registry/model:1.0
```

Interview tip: Mention notebooks as the usual weak point.

📚 **Official docs**

- Hugging Face documentation

certs.eknathalabs.com

InterviewHub - interviewhub.eknathalabs.com

Q115 How do you build a responsible AI governance process that does not block delivery?

Security

🎙 **Spoken answer · ~60 s**

I make governance proportionate and automatic. I classify use cases by risk, so a low-risk internal summariser follows a light path, while a system that affects people's rights goes through deeper review. Standard checklists and templates cover purpose, data sources, evaluation, fairness testing, security review, human oversight and incident plans, and the platform embeds them: pipelines enforce evaluation gates, record lineage automatically, produce model cards, and apply guardrail defaults, so compliance is a by-product of the normal path. A small cross-functional review group, with security, legal, privacy, product and engineering, handles the high-risk cases with clear service levels, so reviews do not stall teams. Monitoring and an incident process cover post-deployment issues, with defined escalation. I measure the process by time to approval and by issues caught, and I iterate. Making the safe path the easy path wins more adoption than policy documents alone.

Likely follow-ups

- How do you classify use cases by risk?
- What is the review group's service level?

Classify use cases by risk, provide templates, embed evaluation gates, lineage, model cards and guardrails in the platform, keep a small cross-functional review for high-risk cases with service levels, and measure and iterate.

```
python risk_classify.py --use-case chatbot.yaml
```

```
mlflow models describe -m models:/assistant/3 | grep -E 'card|eval'
```

Interview tip: Say you make the safe path the easy path.

📚 **Official docs**

- NIST AI Risk Management Framework

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q116 How do you estimate and control the cost of an LLM feature?

Cost

🎙 Spoken answer · ~64 s

Cost is mainly tokens, so I model it from the workload. I estimate average input and output tokens per request, including the system prompt, retrieved context and history, multiply by expected request volume and the per-token price of the model, then add retrieval, embedding, storage, orchestration and monitoring costs. I validate with a pilot using real traffic. To control cost: choose the smallest model that meets the quality bar, route simple requests to cheaper models, trim context, cache prompts and responses, limit output length, batch offline work, and use committed capacity where usage is steady. Governance matters as much as optimisation: per-feature and per-tenant token metering through a gateway, budgets and alerts, quotas, and dashboards for cost per task. I also track unit economics, cost per resolved ticket for example, so decisions link to business value, and I review the top spending prompts regularly, since a verbose prompt multiplies across millions of calls.

Likely follow-ups

- How do you attribute cost per tenant?
- What is a unit economics metric?

🗺 Cost model



Model tokens per request times volume times price plus retrieval and infrastructure, validate with a pilot, then use right-sized models, routing, context trimming, caching, limits and batching. Meter per feature and tenant with budgets and track unit economics.

```
curl -s $GW/metrics | grep -E 'tokens_(in|out)_total'
```

```
python cost_model.py --in-tokens 1800 --out-tokens 300 --rps 20
```

Interview tip: Link cost to a business unit metric such as cost per resolved ticket.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q117 When does self-hosting an LLM become cheaper than using an API, and how do you calculate it?

Cost

🎙 Spoken answer · ~64 s

I compare the full cost of each path at the expected volume. For the API, the cost is tokens times price, which scales linearly, with no idle cost. For self-hosting, the main cost is GPU capacity: instance or hardware cost per hour times the number of GPUs needed to meet peak demand and redundancy, plus storage, networking, engineering and operations time, monitoring, and the cost of idle capacity at low load. Throughput per GPU determines the effective cost per token: benchmark tokens per second at my latency SLO, then divide the hourly cost by the tokens produced per hour at realistic utilisation. Self-hosting wins when the volume is high and steady enough to keep GPUs well utilised, when a smaller open model is good enough, or when privacy requirements force it. The API wins at low or spiky volume and for top-end quality. I recalculate periodically, since prices and hardware efficiency change quickly.

Likely follow-ups

- What utilisation makes self-hosting attractive?
- What hidden costs are commonly missed?

Compare API token cost with GPU hours, redundancy, storage, operations and idle capacity, using benchmarked tokens per second at the SLO and realistic utilisation. Self-hosting wins at high steady volume, with adequate small models or strict privacy needs.

```
python break_even.py --gpu-hour 4.2 --tokens-per-sec 2500 --utilization 0.45 --api-price 0.6
```

```
python benchmark_serving.py --request-rate 8
```

Interview tip: Include idle capacity and engineering time in the calculation.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q118 How do semantic caching and response caching reduce cost, and what are the risks?

Cost

🎙 Spoken answer · ~67 s

Many requests repeat or are near duplicates, such as common support questions. Exact response caching stores the output keyed by a hash of the normalised prompt, which gives an instant, free answer for identical requests. Semantic caching embeds the query and returns a cached answer when a new query is close enough to a previous one, which raises the hit rate for paraphrases. The savings can be large for repetitive workloads, together with lower latency. The risks are serving a wrong or stale answer: a similarity threshold that is too loose returns an answer to a different question, answers depend on user permissions or time-sensitive data, and cached content may contain personal data that must not leak across users. I scope caches per tenant and permission set, set short time to live values for changing facts, exclude personalised or sensitive requests, tune the threshold using evaluation data, and monitor hit rate and correctness, with an easy way to invalidate entries.

Likely follow-ups

- How do you choose the similarity threshold?
- Which requests must never be cached?

Exact caching reuses identical prompts, semantic caching reuses answers for close paraphrases. Both cut cost and latency but risk wrong, stale or leaked answers, so scope per tenant, use TTLs, tune thresholds on evaluation data and exclude personalised requests.

```
redis-cli SETEX resp:$HASH 3600 "$ANSWER"
```

```
python tune_threshold.py --pairs paraphrases.jsonl --min-precision 0.98
```

Interview tip: Mention scoping caches by tenant and permissions.

📚 Official docs

- NIST AI Risk Management Framework

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q119 How do you route requests between models to balance quality and cost?

Cost

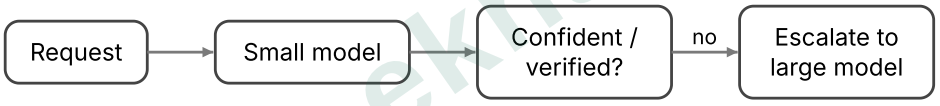
🎙 Spoken answer · ~65 s

A router decides per request which model to call. The simplest approach uses rules: by task type, such as classification and extraction going to a small model and complex reasoning to a large one, by input length, or by tenant tier. More advanced routers use a lightweight classifier, or a small model, to predict difficulty, or a cascade: try the cheap model first, check confidence or validate the output with a verifier, and escalate to the expensive model only when needed. I evaluate routing on a labelled set, measuring the quality loss and cost saving at different thresholds, and choose the point that meets the quality bar. Operationally, I need fallback for outages and rate limits, logging of which model served each request, monitoring of quality per route, and the ability to change rules without redeploying applications, which an LLM gateway provides. Routing adds latency and complexity, so I adopt it when volume makes the savings worthwhile.

Likely follow-ups

- What is a cascade?
- How do you detect that the small model failed?

🗺 Model routing cascade



Route by rules, difficulty classifiers or cascades that escalate when confidence or verification fails. Evaluate quality loss versus savings, log routes, add fallbacks and manage rules in a gateway.

```
router.yaml: routes: [{match: {task: classify}, model: small}, {match: {task: reason}, model: large}]
```

```
python eval_router.py --threshold 0.7 --set eval.jsonl
```

Interview tip: Say you pick the threshold from measured quality loss.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q120 What are the main cost drivers in a RAG system beyond the generation model? Cost

🎙 Spoken answer · ~57 s

Several components contribute. Embedding costs arise at ingestion, for every chunk, and again for every query, and re-embedding the corpus when changing models can be a large one-off. Vector storage depends on the number of vectors, dimensions and index type, since HNSW indexes are memory heavy, and replication multiplies it. Query compute covers search and reranking, and a cross-encoder reranker on many candidates adds notable GPU or CPU cost. Ingestion pipelines, such as parsing, OCR and optional LLM-based enrichment, can be expensive for large document sets. The prompt itself: sending many large chunks increases generation cost on every request, so retrieving fewer, better chunks saves tokens. Then observability storage for traces and logs, and engineering effort. Optimisations include smaller embedding dimensions or quantised vectors, tiered storage, caching, better chunking to reduce tokens, and removing duplicate documents.

Likely follow-ups

- How does vector quantisation reduce cost?
- Why does retrieving fewer chunks save money?

Costs include embedding at ingestion and query, vector storage and memory-heavy indexes, reranking, parsing and enrichment, prompt tokens from retrieved chunks, and logging. Reduce with smaller or quantised vectors, caching, fewer better chunks and deduplication.

```
SELECT pg_size_pretty(pg_relation_size('docs_embedding_idx'));
```

```
python dedupe.py --src docs/ --threshold 0.95
```

Interview tip: Say fewer, better chunks save generation tokens on every request.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q121 How do you implement FinOps for AI workloads, from tagging to showback?

Cost

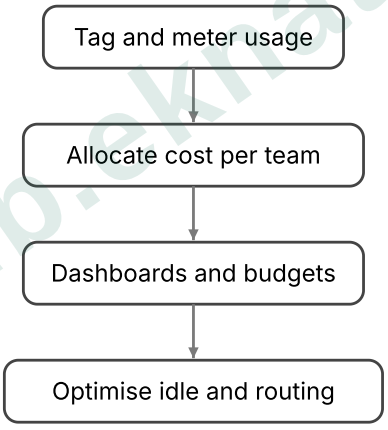
🎙 Spoken answer · ~60 s

I start with visibility. Every GPU node, job, endpoint and API key carries tags for team, project, environment and use case, enforced by policy, so costs can be attributed. Kubernetes cost tools such as OpenCost or Kubecost allocate cluster spend to namespaces and workloads, including GPU hours, and the LLM gateway meters tokens per team and feature. A dashboard combines cloud bills, GPU utilisation and token usage, showing unit costs such as cost per thousand requests or per training run. Then accountability: showback or chargeback reports to teams, budgets and alerts, and quotas for GPUs and tokens. Then optimisation: find idle or low-utilisation GPUs, shut down unused notebooks and endpoints, right-size, use spot capacity for batch work, and commit to reservations for steady baselines. Regular reviews with engineering and finance keep the focus on value delivered per dollar, not just on spending less.

Likely follow-ups

- What is the difference between showback and chargeback?
- Which AI cost anomalies do you alert on?

🗺 AI FinOps



Enforce tags, allocate Kubernetes and GPU spend with OpenCost or similar, meter tokens at the gateway, publish unit-cost dashboards and showback, set budgets and quotas, then optimise idle capacity, spot use and reservations.

```
kubectl cost namespace --window 7d --show-all-resources
```

```
aws ce get-cost-and-usage --time-period Start=2026-10-01,End=2026-10-20 --granularity DAILY
--metrics UnblendedCost --group-by Type=TAG,Key=team
```

Interview tip: Say unit cost and value per dollar, not only total spend.

📚 Official docs

- OpenCost documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q122 How do you reduce idle GPU spend in a shared platform?

Cost

🎙 Spoken answer · ~58 s

Idle GPUs are the biggest avoidable waste, so I attack them systematically. First measure: report allocation versus actual utilisation per team and job, and highlight long-running allocations with low utilisation. Notebooks and dev environments get idle detection and automatic shutdown, with a quick resume. Batch workloads use queues, so GPUs are occupied by whichever job is waiting, with fair share and borrowing of unused quota. Inference deployments use autoscaling on queue-based metrics with scale down when quiet, and for lightweight models, GPU sharing through MIG or time slicing, or multi-model serving to consolidate. Cluster autoscaling removes empty GPU nodes quickly, balanced against warm capacity needed for latency. Spot or preemptible nodes handle interruptible training. I set time limits for jobs and notify owners about idle resources. Regular reviews convert findings into policies, and visible per-team reports create the right incentive.

Likely follow-ups

- How do you detect idle notebooks?
- What is the trade-off with scaling down quickly?

Measure allocation versus utilisation, auto-shutdown idle notebooks, queue batch jobs with fair share, autoscale inference on queue metrics, share small workloads with MIG or multi-model serving, use spot, limit job duration and report per team.

```
kubectl get pods -A -o json | jq '.items[] |
select(.spec.containers[].resources.limits."nvidia.com/gpu")'
```

```
dcgmi stats -e && dcgmi stats -v -j 1
```

Interview tip: Say visibility per team creates the incentive.

📚 Official docs

- Kubernetes: Scheduling GPUs

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q123 How do you set up budgets, rate limits and kill switches to prevent a runaway LLM bill?

Cost

🎙 **Spoken answer · ~66 s**

Runaway cost usually comes from a loop, a retry storm, a leaked key or an unexpected traffic spike, so I add defences at several levels. At the gateway: per-key and per-tenant token and request budgets per minute, day and month, with hard stops and soft alerts at thresholds such as 50, 80 and 100 percent. In the application: maximum steps and tokens per agent run, maximum output length, retry limits with backoff and a retry budget, and loop detection. In infrastructure: autoscaling maximums, quotas on GPU nodes, and cloud budget alerts with automated actions, for example disabling a key or scaling to zero when a limit is crossed. Keys are scoped, rotated and monitored for use from unexpected locations. A runbook describes how to revoke keys and switch to a degraded mode. I test these controls with simulated overruns, and review anomalies in daily cost reports, because a limit that has never been tested often fails when needed.

Likely follow-ups

- What belongs in a runaway-cost runbook?
- How do you test budget enforcement?

Budgets and hard stops at the gateway, step, token and retry limits in the application, autoscaling and quota ceilings, cloud budget alerts with automated actions, scoped rotated keys and a tested runbook for revoking keys and degrading.

```
aws budgets create-budget --account-id $ID --budget file://llm-budget.json --notifications-with-subscribers file://notify.json
```

```
curl -X POST $GW/admin/keys/team-a/disable
```

Interview tip: Say you test the kill switch with a simulated overrun.

📚 **Official docs**

- RFC 6585: HTTP 429 Too Many Requests
- Microsoft: Cloud design pattern – Throttling
- Martin Fowler: Feature Toggles

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q124 What is AIOps and where does it genuinely help SRE and platform teams?

AI for Ops

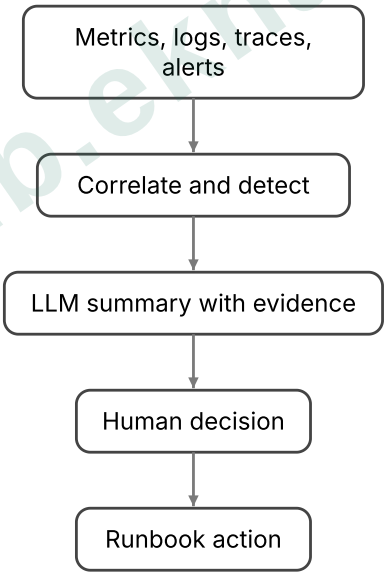
🎙 Spoken answer · ~63 s

AIOps applies machine learning and increasingly language models to operations data, such as metrics, logs, traces, alerts and tickets, to reduce toil and speed up detection and diagnosis. It helps most in a few places. Alert noise reduction: grouping related alerts, deduplicating and correlating them into one incident. Anomaly detection for metrics with seasonality, where static thresholds are brittle. Incident summarisation and context gathering: pulling recent changes, related alerts, logs and runbooks into one view, which saves minutes at the start of an incident. Log analysis and pattern clustering to find unusual errors. Ticket triage and routing, and drafting postmortems from timelines. It helps less where the data is poor or the problem is novel. I adopt it incrementally: start with read-only assistance, measure whether it reduces mean time to acknowledge or resolve, keep humans in control of actions, and be sceptical of vendor claims without evidence from my own data.

Likely follow-ups

- Which AIOps use cases give quick value?
- How do you measure its benefit?

🗺 AIOps pipeline



AIOps helps with alert correlation, anomaly detection, incident summarisation and context gathering, log clustering, triage and postmortem drafting. Start read-only, measure MTTA and MTTR, keep humans in control.

```
amtool alert query --alertmanager.url=$AM
```

```
promtool query instant $PROM 'anomaly_score > 3'
```

Interview tip: Say you measure it against MTTA and MTTR on your own data.

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Introduction (what is SRE)

Q125 How would you use an LLM to summarise and triage incidents and alerts safely?

AI for Ops

🎙 **Spoken answer · ~62 s**

I would build a read-only pipeline first. When an alert fires, the system gathers context through defined integrations: the alert details, recent deployments and config changes, related alerts, key dashboards and queries, relevant runbook sections through retrieval, and the service owner from the catalogue. The model then produces a structured summary, with the likely area, the evidence with links, suggested checks and a confidence level, and it labels uncertainty. Every statement links back to a source so engineers can verify quickly. Safeguards include scoping data to what the on-call engineer may see, redacting secrets and personal data, treating log content as untrusted to resist injection, and displaying the summary as a suggestion, never as an automatic action. I run it in shadow mode against historical incidents first, compare with human postmortem findings, measure accuracy and usefulness through feedback, and only then expand, for example to drafting status updates.

Likely follow-ups

- Why must every claim link to a source?
- How would you evaluate it before launch?

Gather alert, change, related-alert, dashboard and runbook context, then produce a structured summary with linked evidence and uncertainty. Scope data, redact secrets, treat logs as untrusted, keep it advisory, and validate on historical incidents first.

```
python gather_context.py --alert $ALERT_ID --include deploys,runbooks,owners
```

```
python eval_incidents.py --history incidents-2026Q2.jsonl
```

Interview tip: Shadow-mode evaluation on past incidents is the credibility step.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q126 How do you apply anomaly detection to metrics without drowning in false positives?

AI for Ops

🎙 **Spoken answer · ~59 s**

Static thresholds fail on seasonal metrics, but naive anomaly detection can be noisy. I start by choosing metrics that matter, tied to user experience or SLOs, rather than everything. I use methods that handle seasonality and trend, such as comparing to the same time last week, seasonal decomposition, or forecasting-based bands, and keep them simple, because simple methods are easier to understand and tune. I require persistence, an anomaly must last for several intervals, and combine signals, for example latency rising together with error rate, before alerting. Sensitivity is tuned per metric using historical data and past incidents, and I measure precision and recall from labelled events. Anomalies do not need to page: low-confidence ones can create tickets or annotations on dashboards, while paging is reserved for SLO-impacting conditions. Feedback from engineers, marking alerts as useful or noise, feeds continuous tuning.

Likely follow-ups

- Why require persistence?
- What methods handle weekly seasonality?

Pick SLO-relevant metrics, use seasonal baselines or forecasting bands, require persistence and combined signals, tune sensitivity against labelled history, route low-confidence anomalies to tickets instead of pages, and learn from feedback.

```
avg_over_time(http_requests_total[5m]) / avg_over_time(http_requests_total[5m] offset 1w) > 1.5
```

```
python seasonal_decompose.py --metric latency_p95 --period 168h
```

Interview tip: Say not every anomaly deserves a page.

📚 **Official docs**

- OpenTelemetry: GenAI semantic conventions

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q127 How can LLMs help with log analysis, and what are the limits?

AI for Ops

🎙 Spoken answer · ~64 s

Logs are voluminous, so the first step is reduction, not feeding everything to a model. I cluster similar log lines into templates using parsers, count them, compare with a baseline period, and surface new or rare templates and sudden changes in volume. An LLM then helps interpret the small set that matters: explaining an unfamiliar error message, correlating errors across services by trace ID, summarising a time window, and suggesting likely causes and commands to check, with references to documentation. It can also help write queries in the log language from a natural language description. The limits are important: context windows are small relative to log volume, models can confidently misread an error, internal systems are unfamiliar without retrieval of runbooks and architecture docs, logs can contain sensitive data and injected text, and costs rise with volume. So I keep it as an assistant, verify with queries, and keep humans responsible for conclusions.

Likely follow-ups

- Why cluster logs before using an LLM?
- How do you handle sensitive data in logs sent to a model?

Reduce first: template clustering, baselines and rare-pattern detection. Then use an LLM to explain errors, correlate by trace ID, summarise windows and draft queries. Limits: context size, misreadings, sensitive data and injection; keep humans responsible.

```
drain3 --input app.log --output templates.json
```

```
logcli query '{app="api"} |= "error"' --since=30m --limit=200
```

Interview tip: Say reduction comes before the model.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q128 How do you use AI to assist with root cause analysis across services?

AI for Ops

🎙 Spoken answer · ~65 s

I give the model, or a small agent, structured access to the evidence that humans use. That includes the service dependency graph from the catalogue or traces, timelines of deployments and configuration changes, metrics for the golden signals of affected services and their dependencies, error logs and exemplar traces, and the runbooks. The assistant proposes hypotheses ranked by evidence, such as a deploy to a dependency just before errors began, saturation in a database, or a regional issue, and suggests queries to confirm or refute each. It reports what it checked and what it found, not only a conclusion. Traditional techniques complement it: change correlation, topology-aware ranking of suspects, and statistical comparison of affected and unaffected cohorts. Engineers decide and act. I evaluate the assistant by replaying past incidents with known root causes and measuring how often the right cause appears in its top suggestions, and I record cases where it misled, to improve prompts and tools.

Likely follow-ups

- What is cohort comparison?
- How do you measure top-k accuracy of suggested causes?

Provide dependency graphs, change timelines, golden-signal metrics, logs, traces and runbooks; the assistant ranks hypotheses with evidence and suggested confirming queries. Humans decide. Evaluate by replaying past incidents and measuring top-k accuracy.

```
curl $TEMPO/api/search?tags=service.name=api%20http.status_code=500
```

```
git log --since='2 hours ago' --oneline -- deploy/
```

Interview tip: Say the assistant must show what it checked, not only a verdict.

📚 Official docs

- Google SRE Book: Postmortem Culture

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q129 How do you use AI coding assistants safely within a platform engineering team?

AI for Ops

🎙 **Spoken answer · ~63 s**

They boost productivity, but they need guardrails. Policy: decide which tools are approved, what code and data may be shared with them, and require enterprise agreements with no training on our data. Security: scan generated code like any other, with static analysis, secret scanning, dependency checks and licence checks, because assistants can suggest vulnerable patterns, hallucinated packages and copied licensed code. Review: AI-generated changes need the same or stricter code review, and for infrastructure code, a plan review and policy checks before apply, since a wrong Terraform or Kubernetes change can cause outages. Access: assistants and agents with repository or terminal access get least-privilege credentials, sandboxed environments, and no production credentials. Quality: encourage tests, small changes and human understanding of what is merged. I measure outcomes such as cycle time and defect rates, not only usage, and gather feedback to improve prompts, context files and conventions shared across the team.

Likely follow-ups

- What is a hallucinated package risk?
- How do you give an assistant safe repo access?

Approve tools and data policies, scan and review generated code, enforce plan review and policy checks for infrastructure, give least-privilege sandboxed access and no production credentials, and measure outcomes like cycle time and defects.

```
semgrep --config auto src/
```

```
terraform plan -out=tfplan && conftest test tfplan.json
```

```
gitleaks detect --source .
```

Interview tip: Mention hallucinated packages and review of infrastructure changes.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q130 How would you build a natural-language interface to your observability data, such as text to PromQL or text to SQL?

AI for Ops

🎙 Spoken answer · ~63 s

I would retrieve first and generate second. The assistant receives the user's question plus context: the schema, available metrics and labels with descriptions, a handful of example queries for common questions, and the user's scope such as the service they are viewing. The model generates a query, and the system validates it before running: syntax check, allow list of data sources and functions, read-only access, enforced time-range and cardinality limits, and query cost estimation, so a model-written query cannot overload the backend. The result is shown with the generated query visible, so users can verify and learn, and charted with an explanation. Ambiguity triggers a clarifying question rather than a guess. I evaluate with a set of questions and known-good queries, measuring execution accuracy, not just similarity of the text, and I log failures to extend examples. Permissions follow the user, and the feature cannot reach data they cannot normally see.

Likely follow-ups

- Why show the generated query?
- How do you prevent expensive queries?

Give the model schema, metric descriptions and example queries, validate generated queries against allow lists, read-only access and limits, show the query with the result, ask clarifying questions, evaluate on execution accuracy and respect user permissions.

```
promtool check metrics < metrics.txt
```

```
curl $PROM/api/v1/query --data-urlencode 'query=sum(rate(http_requests_total{service="api"}[5m]))'
```

Interview tip: Say you evaluate by execution accuracy, not string similarity.

📚 Official docs

- Prometheus: Querying basics (PromQL)
- OpenTelemetry: GenAI semantic conventions

certs.eknathalabs.com
InterviewHub · interviewhub.eknathalabs

Q131 How do you automate runbooks with AI while keeping change safe?

AI for Ops

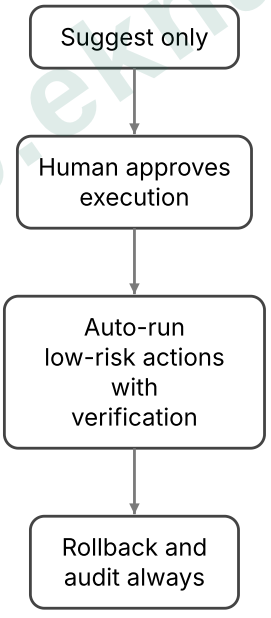
🎙 Spoken answer · ~60 s

I automate in stages of increasing autonomy. First, make runbooks executable and tested: convert prose into scripts or workflows with parameters, checks and rollback, which has value even without AI. Then let an assistant select and fill in a runbook from the incident context, but still ask a human to approve execution. For low-risk, well-understood actions, such as restarting a stateless pod in staging, clearing a known cache, or scaling within bounds, allow automatic execution within guardrails: allow lists, rate limits, blast radius limits, business-hours and change-freeze awareness, and automatic verification that the action helped, with rollback if not. High-risk actions always need human approval. Every action is logged with the reasoning and the evidence, and subject to review. I measure outcomes: time to mitigation, error rate of automated actions and how often humans override. Autonomy expands only as the track record justifies it.

Likely follow-ups

- What conditions block an automatic action?
- How do you verify that an action helped?

🗺 Runbook automation levels



Make runbooks executable first, then let AI propose runbooks for human approval, automate only low-risk actions within allow lists, rate limits, blast-radius and freeze rules with verification and rollback, and expand autonomy as evidence grows.

```
ansible-playbook runbooks/restart_pod.yml -e ns=staging -e pod=api-7f9 --check
```

```
kubectl rollout status deploy/api --timeout=120s
```

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Interview tip: Say autonomy expands only as the track record justifies.

📚 **Start here (official docs)**

- Hugging Face documentation

Q132 What are the pitfalls of using AI for capacity planning and autoscaling decisions?

AI for Ops

🎙 **Spoken answer · ~64 s**

Forecasting models can help predict demand, but they have pitfalls. They fit the past, so they struggle with new products, campaigns, outages and regime changes, and a confident forecast can be wrong. Seasonality and holidays need explicit handling. Feedback loops appear when autoscaling changes the metrics that the model uses as input. A black-box scaler is difficult to debug when it makes a surprising decision, and a bad prediction can cause both overprovisioning, which wastes money, and underprovisioning, which hurts users. For safety I keep deterministic guardrails: minimum and maximum replicas, rate limits on scaling, and reactive scaling on real load as a fallback, so predictive scaling only adds capacity early. I compare forecasts with actuals continuously, track the error, and alert when it grows. Human judgement still covers known events and long-lead purchases like hardware. Simple methods, such as seasonal baselines, often perform nearly as well and are far easier to operate.

Likely follow-ups

- Why keep reactive scaling as a fallback?
- When are simple seasonal baselines enough?

Forecasts fit the past and fail on new regimes; feedback loops and opaque decisions complicate debugging. Keep min and max bounds, reactive fallback, forecast error tracking and human input for known events. Simple seasonal methods often suffice.

```
kubectl get hpa -o wide
```

```
python forecast_eval.py --pred forecast.csv --actual actual.csv --metric mape
```

Interview tip: Say predictive scaling should only add capacity early, with a reactive backstop.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (capacity planning)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q133 How do you use AI to reduce alert noise and correlate alerts into incidents?

AI for Ops

🎙 Spoken answer · ~59 s

I start with the basics, because they remove most noise: deduplicate identical alerts, group by service, cluster and time window, suppress child alerts when a parent dependency is down, and alert on symptoms tied to SLOs rather than on every cause. Then correlation. Topology-aware grouping uses the service dependency graph, so alerts from a database and its callers form one incident. Text and label similarity can cluster alerts that share fields. A model can propose groupings and a short description, while engineers confirm. I measure alerts per incident, pages per on-call shift, the share of actionable pages, and time to acknowledge. Every grouping rule is reviewed after incidents, and feedback buttons mark alerts as useful or noise. I avoid black-box suppression that could hide a real outage, so suppressed alerts stay visible in an audit view, and critical SLO alerts always page.

Likely follow-ups

- What is parent-child alert suppression?
- How do you prove noise actually dropped?

Deduplicate, group by service and window, suppress child alerts, and alert on SLO symptoms. Add topology-aware correlation and model-proposed groupings that humans confirm. Measure alerts per incident and actionable pages, and never hide critical alerts.

```
amtool silence query --alertmanager.url=$AM
```

```
group_by: ['alertname','cluster','service']
```

Interview tip: Say suppressed alerts must stay auditable.

📚 Official docs

- Google SRE Book: Practical Alerting

interviewhub.eknathalabs.co
certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q134 How can LLMs help draft postmortems, and what must humans still own?

AI for Ops

🎙 **Spoken answer · ~62 s**

A model can save hours of clerical work. Given the incident timeline from chat, the alert history, deployment records, and ticket updates, it can draft a chronological summary, list impact figures from linked dashboards, extract action items, and propose a first version of the contributing factors. It can also check a draft against a template for missing sections. Humans must still own the analysis. The model does not know what people were thinking at decision points, it can invent plausible causes, and it tends toward neat single root causes when real incidents have several contributing factors. I therefore require every claim to cite a timeline entry, have the incident commander review the facts, and keep the review blameless, with people deciding the lessons and the priorities for follow-up. I also keep sensitive details restricted, and track whether action items get completed, since that is where postmortems create value.

Likely follow-ups

- Why avoid a single root cause?
- How do you keep it blameless?

Use a model to assemble the timeline, impact summary, action items and template checks from incident records, with citations. Humans validate facts, analyse contributing factors blamelessly, choose priorities and track the follow-up work.

```
python build_timeline.py --incident INC-4821 --sources slack,pagerduty,deploys
```

```
python check_template.py postmortem.md
```

Interview tip: Say the model drafts, but people own the lessons.

📚 **Official docs**

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Google SRE Book: Managing Critical State (consensus)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q135 How would you build a Kubernetes troubleshooting assistant for developers, and keep it safe?

AI for Ops

🎙 Spoken answer · ~67 s

I would build it as a read-only assistant first. It accepts a question such as why a pod is crash looping, and calls a small set of vetted tools: describe pod, recent events, logs of the previous container, resource usage, recent rollouts, and node conditions, all through a service account limited to the user's own namespaces. The model combines the evidence into an explanation with likely causes, for example an out-of-memory kill, a failing readiness probe or an image pull error, and suggests commands the user can run. Safety: no write access by default, outputs scanned for secrets, logs treated as untrusted text, rate limits, and audit logging. If I later allow actions such as restarting a deployment, they need confirmation from the user and run with the user's own permissions. I evaluate on a library of known failure scenarios in a test cluster, checking that the diagnosis is correct and that it asks for more data when evidence is thin.

Likely follow-ups

- Which tools would you expose first?
- How do you test it on known failures?

A read-only assistant with vetted tools for describe, events, logs, usage and rollouts under the user's own permissions. It explains likely causes with evidence, treats logs as untrusted, audits use, and is tested on a library of known failure scenarios.

```
kubectl describe pod $POD -n $NS
```

```
kubectl logs $POD -n $NS --previous --tail=100
```

```
kubectl get events -n $NS --sort-by=.lastTimestamp
```

Interview tip: Say read-only first, then confirmed actions with user permissions.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q136 Which techniques cut token spend without hurting quality, such as batch APIs, shorter prompts and context pruning?

Cost

🎙 Spoken answer · ~58 s

I look at where tokens go before optimising. Typical savings come from several places. Trim prompts: remove verbose instructions and redundant examples, and keep stable text at the start so prompt caching applies. Prune context: retrieve fewer, better chunks, summarise old conversation turns, and drop tool outputs once used. Limit output with maximum length and concise format instructions. Use the cheapest adequate model for each step, and route easy requests to smaller ones. Use batch or asynchronous processing for work that is not urgent, since many providers price batch jobs lower. Avoid unnecessary calls through caching and deduplication, and stop agent loops early. Each change is checked against the evaluation set, because a cheaper prompt that lowers accuracy costs more in rework. I track tokens and cost per task over time, and review the most expensive prompts regularly.

Likely follow-ups

- How do you verify quality after trimming?
- Which requests suit batch processing?

Measure first, then trim prompts, prune context, cap output, route to cheaper models, batch non-urgent work and cache repeats. Validate every change against the evaluation set and track cost per task.

```
python token_report.py --group-by prompt_id --top 20
```

```
python eval_run.py --prompt v7 --compare v6
```

Interview tip: Say each saving must pass the evaluation set.

📚 Official docs

- Prompt engineering overview (Claude docs)
- GNU coreutils manual

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q137 How do quantisation and model size choices change the cost of serving, and how do you decide?

Cost

🎙 **Spoken answer · ~61 s**

Cost per token is driven by how many GPUs are needed and how many tokens each can produce. A smaller model, or a quantised one using 8-bit or 4-bit weights, needs less memory, so it can fit on fewer or cheaper GPUs, leaves more room for the KV cache and larger batches, and often decodes faster because decoding is memory-bandwidth bound. The trade-off is potential quality loss, which varies by task: extraction and classification often tolerate it well, while complex reasoning or code may not. I decide empirically. I run the evaluation set on candidates, such as the full-precision model, an 8-bit version and a smaller model, and benchmark throughput at my latency target. Then I compute cost per thousand successful tasks, not just cost per token. I also consider operational factors: kernel support on my GPUs, ease of upgrades, and monitoring quality after changes in production.

Likely follow-ups

- Why is decoding memory-bandwidth bound?
- What is cost per successful task?

Smaller or quantised models need fewer GPUs and leave more KV cache room, raising throughput. Compare full precision, quantised and smaller models on the evaluation set and benchmarks, and decide by cost per successful task.

```
vllm serve model --quantization awq --max-model-len 8192
```

```
python benchmark_serving.py --model model --request-rate 8
```

Interview tip: Say you compare cost per successful task, not per token.

📚 **Official docs**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q138 How do you set up human review and labelling for evaluating an AI feature? Evaluation

🎙 Spoken answer · ~62 s

Human labels are the ground truth that automated metrics are checked against, so the process matters. I define clear guidelines with a rubric, examples of each score, and rules for edge cases, then pilot with a few reviewers and refine ambiguous instructions. I choose reviewers with the right expertise, such as support leads or engineers, and measure agreement between them, for example with Cohen's kappa; low agreement signals unclear criteria rather than bad reviewers. I sample cases deliberately: a random set for overall quality, plus failures, low-confidence outputs and important segments. Tooling should show the full context, hide model identity when comparing, and capture reasons for scores. I protect sensitive data by redacting and restricting access. Labels are versioned and used to calibrate LLM judges and to grow the regression set. Review is ongoing, with a small weekly sample to detect drift, because labelling every output is not realistic.

Likely follow-ups

- What does low reviewer agreement tell you?
- How do you sample cases to review?

Write a rubric with examples, pilot it, use qualified reviewers, measure agreement, sample randomly and by risk, blind comparisons, protect data, version labels and use them to calibrate automated judges and grow regression sets.

```
python agreement.py --labels labels.csv --metric kappa
```

```
python sample_for_review.py --random 50 --failures 50
```

Interview tip: Say low agreement usually means an unclear rubric.

📚 Start here (official docs)

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q139 What is benchmark contamination and how do you avoid fooling yourself with evaluation results?

Evaluation

🎙 **Spoken answer · ~62 s**

Contamination means the evaluation data, or something very similar, was seen by the model during training, so scores overstate real ability. Public benchmarks are especially exposed. For my own evaluation I keep a private test set built from real, recent traffic and domain cases, never published or used in prompts or fine-tuning, and I keep a separate development set for tuning so I do not overfit to the test set. I check overlap between fine-tuning data and test data with deduplication. Other traps include data leakage through retrieval of the answer key, tuning prompts against the same examples used to report results, small sample sizes with no confidence intervals, and only reporting aggregates that hide failures in important segments. I report uncertainty, break results down by category, refresh the test set periodically, and confirm gains with online experiments, since offline numbers do not always carry over to real users.

Likely follow-ups

- How do you detect overlap with training data?
- Why keep a separate dev set?

Contamination inflates scores when test data was seen in training. Use private, recent test sets separate from development data, deduplicate against fine-tuning data, report confidence intervals and segment results, refresh sets and confirm with online tests.

```
python dedupe_overlap.py --train ft.jsonl --test test.jsonl --threshold 0.9
```

```
python bootstrap_ci.py --scores run.csv
```

Interview tip: Say report confidence intervals and per-segment results.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q140 How do you evaluate guardrails, including false positives and false negatives?

Evaluation

🎙 **Spoken answer · ~65 s**

A guardrail is a classifier, so I evaluate it as one. I build a labelled set with two sides: harmful or policy-violating inputs and outputs that should be blocked, such as injection attempts, personal data and unsafe requests, and benign but similar-looking cases that should pass. I then measure the false negative rate, which is risk getting through, and the false positive rate, which blocks legitimate users and harms the experience. I look at both by category and at different thresholds to choose an operating point matched to the risk: strict for high-severity categories, more lenient elsewhere. I test against adversarial variants, such as paraphrases, encodings and other languages. I also measure latency added and cost. In production, I log triggers, sample blocked and allowed cases for review, and track rates over time to catch drift. The set is refreshed with new attacks found by red teaming and with real false positives reported by users.

Likely follow-ups

- How do you choose a threshold?
- Why test other languages?

Treat the guardrail as a classifier: a labelled set with harmful and benign look-alike cases, measure false negatives and positives per category and threshold, test adversarial variants, track latency, review samples in production and refresh the set.

```
python eval_guardrail.py --set guard.jsonl --threshold 0.6
```

```
python roc_curve.py --scores guard_scores.csv
```

Interview tip: Say you measure both blocked-good and missed-bad.

📚 **Official docs**

- AWS Control Tower
- OWASP Top 10 for LLM Applications

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q141 How does the position of information in a long prompt affect results, and how do you structure long contexts?

Prompting

🎙 **Spoken answer · ~64 s**

Models do not use all parts of a long context equally well. Information buried in the middle is often used less reliably than information at the beginning or the end, and irrelevant text can distract the model. I structure long prompts deliberately. Put stable instructions first, which also helps caching. Place the documents in clearly delimited sections with identifiers, and put the specific question or task after the documents, sometimes repeating key constraints at the end. Order retrieved passages by relevance and keep only what is needed, since more context is not always better. For multi-document work, ask the model to quote relevant passages first and then answer from them, which improves grounding. I test with questions whose answers sit at different positions in the context, using my own models and data, because behaviour varies by model. If results degrade, I shorten the context, summarise in stages, or split the task across calls.

Likely follow-ups

- What is the lost-in-the-middle effect?
- How do you test position sensitivity?

Put stable instructions first, delimit documents with identifiers, place the question after the material, order by relevance, ask for quotes before answers and trim context. Test answers at different positions and split tasks if quality degrades.

```
python position_test.py --doc long.txt --answer-pos 0,25,50,75,100
```

```
python build_prompt.py --docs retrieved.json --max-chunks 6
```

Interview tip: Say you test answer position on your own model.

📚 **Official docs**

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q142 How do you handle multilingual users in prompts and evaluation?

Prompting

🎙 Spoken answer · ~60 s

I first learn which languages matter and how well the model performs in each, because quality varies, often lower for less common languages, and tokenisation makes non-English text cost more tokens. Design choices: instruct the model to reply in the user's language, keep system instructions in one language the team can maintain, and avoid translating idioms in few-shot examples carelessly. Retrieval needs a multilingual embedding model, or translation before indexing, and I test cross-language retrieval. For evaluation, I build test sets per language with native-speaking reviewers, since machine translated test sets hide errors, and report results per language rather than as an average. Guardrails also need testing in each language, because injection and safety filters can be weaker outside English. Finally, I watch cost and latency per language, and use fallbacks, for example a stronger model for languages where the primary model is weak.

Likely follow-ups

- Why do non-English prompts cost more?
- How do you test retrieval across languages?

Measure per-language quality, instruct replies in the user's language, use multilingual embeddings, build native-reviewed test sets, evaluate guardrails per language and track token cost, with stronger-model fallbacks where needed.

```
python eval_by_language.py --set multi.jsonl --langs en,hi,te,ta
```

```
python token_count.py --text sample_hi.txt --model m
```

Interview tip: Say report per-language results, not one average.

📚 Official docs

- Prompt engineering overview (Claude docs)

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q143 How do you choose and upgrade an embedding model without breaking a RAG system?

RAG & vectors

🎙 **Spoken answer · ~63 s**

I choose by testing on my own data, not only public leaderboards: build a retrieval evaluation set of real questions with known relevant documents, then compare candidate models on recall at k and ranking quality, together with dimensions, maximum input length, language support, latency, cost and licence. Dimensions affect storage and search speed. Upgrading is disruptive, because vectors from different models are not compatible, so I treat it as a migration. Build a new index in parallel with the new model, run the evaluation on both, shadow production queries against the new index to compare results, then switch traffic gradually behind a flag and keep the old index for rollback. Queries and documents must use the same model and version, so I store the model name and version with each vector. The re-embedding cost for the corpus is estimated in advance, and I limit upgrades to cases where measured gains justify them.

Likely follow-ups

- Why can you not mix vectors from two models?
- How do you estimate re-embedding cost?

Choose by recall and ranking on your own evaluation set plus dimensions, length, language, cost and licence. Upgrade as a migration: build a parallel index, shadow queries, switch gradually, store the model version with vectors and keep rollback.

```
python eval_retrieval.py --model bge-m3 --set qa.jsonl --k 5
```

```
SELECT embedding_model, count(*) FROM chunks GROUP BY 1;
```

Interview tip: Say vectors from different models are incompatible, so upgrades are migrations.

📚 **Official docs**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q144 How do you design multi-tenant RAG so tenants never see each other's data?

RAG & vectors

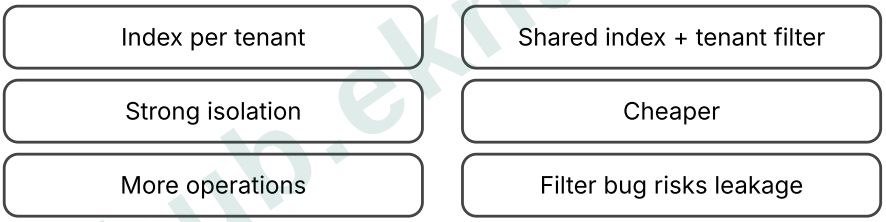
🎙 Spoken answer · ~60 s

Isolation must be enforced in the retrieval layer, not left to the prompt. There are three common designs. A separate index or collection per tenant gives the strongest isolation and simple deletion, at the cost of operational overhead with many tenants. A shared index with a mandatory tenant filter on every query is cheaper, but a bug in filter handling can leak data, so the filter must be applied by a server-side layer the application cannot skip. A hybrid places large or sensitive tenants in dedicated indexes and small ones in shared ones. Beyond tenants, document-level permissions are checked at query time. Other safeguards include per-tenant encryption keys where required, caches scoped by tenant, logs separated and access controlled, and tests that attempt cross-tenant queries and expect empty results. Deletion and offboarding procedures must remove vectors, source copies and caches, and be verifiable.

Likely follow-ups

- When would you prefer an index per tenant?
- How do you test for cross-tenant leakage?

🗺 Multi-tenant RAG isolation



Enforce isolation in retrieval: per-tenant indexes for strongest separation, or a shared index with a server-enforced tenant filter, or a hybrid. Add document-level permissions, scoped caches and logs, tests for cross-tenant queries and verifiable deletion.

```
python leak_test.py --tenant A --query "tenant B salary data" --expect-empty
```

```
curl -X DELETE $VDB/collections/tenant_b
```

Interview tip: Say the tenant filter must be server-enforced.

📚 Official docs

- Kubernetes: Multi-tenancy

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q145 Where should human approval sit in an agent workflow, and how do you design it so people do not just click approve?

Agents & tools

🎙 **Spoken answer · ~63 s**

I place approval on actions that are irreversible, costly, external-facing or high privilege, such as sending customer email, changing production, spending money or deleting data. Low-risk reads proceed automatically so the agent remains useful. To prevent approval fatigue, I keep the number of prompts low by grouping related steps into a plan that is approved once, with clear scope and limits. The request shows what will happen in plain language: the exact action, target, parameters, expected effect and a diff or preview where possible, plus the evidence the agent used. The default is deny, with timeouts that fail safe. Approvers need authority matching the action, and their decisions are logged with the agent's reasoning. I measure the approval rate and time; an approval rate near 100 percent suggests either the checks are unnecessary or people are rubber-stamping, and I adjust. Over time, well-understood actions can move to automatic within limits.

Likely follow-ups

- What does approval fatigue look like?
- Which actions can become automatic later?

Require approval for irreversible, costly, external or privileged actions, group steps into approved plans, show exact actions with previews and evidence, default to deny, log decisions, and track approval rates to detect rubber-stamping.

```
agent run --require-approval 'delete,send_email,deploy' --plan-mode
```

```
kubectl diff -f proposed.yaml
```

Interview tip: Say show a diff or preview, not a vague yes or no.

📚 **Start here (official docs)**

- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q146 How do you make agent actions reliable with retries, idempotency and failure recovery?

Agents & tools

🎙 **Spoken answer · ~65 s**

Agents call tools over networks and the model itself can repeat or mis-order steps, so I use ordinary distributed systems discipline. Tool calls get timeouts and bounded retries with backoff and jitter, and distinguish transient errors from permanent ones, returning clear error messages the model can act on. Write operations are idempotent: an idempotency key per intended action means a repeated call does not create a duplicate ticket or double charge. Long tasks checkpoint their state after each step, so a crash resumes from the last checkpoint rather than restarting or repeating completed side effects. Where multiple steps must succeed together, I design compensating actions or use a workflow engine with sagas. Loops are bounded by maximum steps, tokens and time, with detection of repeated identical calls. On unrecoverable failure, the agent stops, reports what was done and what remains, and hands over to a human. I test with injected failures to confirm these paths work.

Likely follow-ups

- What is an idempotency key?
- How do you inject failures for testing?

Use timeouts, bounded retries with backoff, idempotency keys for writes, checkpoints after each step, compensating actions for multi-step changes, bounded loops and a clear handoff on failure. Test with injected faults.

```
curl -H 'Idempotency-Key: 7f3c-ticket-991' -X POST $API/tickets -d @t.json
```

```
temporal workflow show --workflow-id agent-991
```

Interview tip: Say treat tool calls like any unreliable network call.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)
- Microsoft: Cloud design pattern – Retry

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q147 How do you choose between building an agent with a framework and writing a thin custom loop?

Agents & tools

🎙 Spoken answer · ~65 s

I decide based on how much the framework gives me versus the control and clarity I lose. Frameworks provide ready patterns: tool calling loops, memory, tracing integrations, multi-agent graphs, and connectors, which speeds up prototypes and gives a shared vocabulary. The costs are abstraction layers that obscure what is sent to the model, fast-changing interfaces, harder debugging, and dependency risk. A thin custom loop is often only a few dozen lines around the model API: send messages, execute requested tools, append results, stop on a condition. It is transparent, easy to test, and easy to add limits, logging and permissions to. My usual approach is to prototype in whichever is fastest, then for production keep the core loop simple and owned by us, using libraries for specific needs such as tracing, schema validation or protocol support like MCP. Whatever I choose, I keep tool definitions, prompts and evaluations independent of the framework to limit lock-in.

Likely follow-ups

- What would push you toward a framework?
- How do you avoid lock-in?

Frameworks speed up prototypes but add abstraction, churn and debugging cost. A thin owned loop is transparent and easy to control. Prototype quickly, keep the production core simple, use libraries selectively and keep tools, prompts and evaluations portable.

```
while not done: resp = llm(messages, tools); run_tools(resp); messages.append(results)
```

```
pip list | grep -E 'langgraph|langchain|llama-index'
```

Interview tip: Say keep tools, prompts and evals framework-independent.

📚 Official docs

- Model Context Protocol

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com

Q148 How do you serve many fine-tuned variants efficiently with LoRA adapters?

Serving

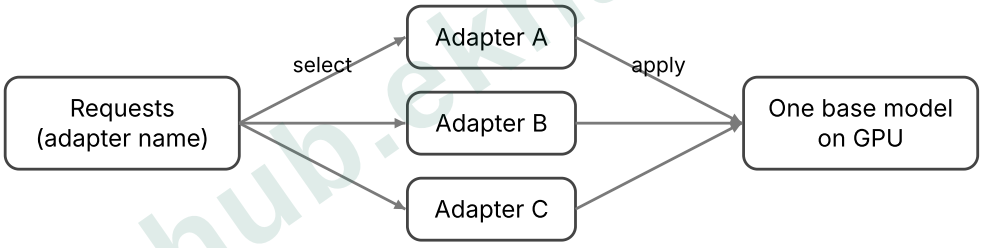
🎙 Spoken answer · ~66 s

Fine-tuning a full copy of the model per customer or task wastes GPU memory, because each copy holds billions of weights. LoRA produces small adapter weights, usually megabytes, that modify a shared base model. Serving engines such as vLLM can hold one base model on the GPU and load many adapters, applying the right adapter per request, even mixing different adapters in the same batch. This lets a few GPUs serve dozens or hundreds of variants. The practical points are: adapters are stored in a registry with versions and loaded on demand with an LRU cache, requests specify the adapter name, and capacity planning covers the number of adapters kept resident and the extra latency when one must load. Adapter quality is evaluated separately per variant. There is a small throughput cost compared with a merged model, so a very high-traffic variant may be merged and served on its own. Access control must stop one tenant using another's adapter.

Likely follow-ups

- What is the trade-off with merging adapters?
- How do you control which tenant can load which adapter?

🗺 Multi-LoRA serving



Keep one base model on the GPU and apply small LoRA adapters per request, loaded on demand from a versioned registry with caching. This serves many variants cheaply; merge very busy variants, evaluate each, and enforce adapter access control.

```
vllm serve base-model --enable-lora --lora-modules tenantA=/adapters/a tenantB=/adapters/b
```

```
curl $VLLM/v1/completions -d '{"model":"tenantA","prompt":"Hi"}'
```

Interview tip: Say one base model, many small adapters.

📚 Official docs

- vLLM documentation
- Hugging Face documentation

certs.eknathalabs.com
InterviewHub - interviewhub.ekn

Q149 What is prefix caching, and how can KV cache offloading improve serving efficiency?

Serving

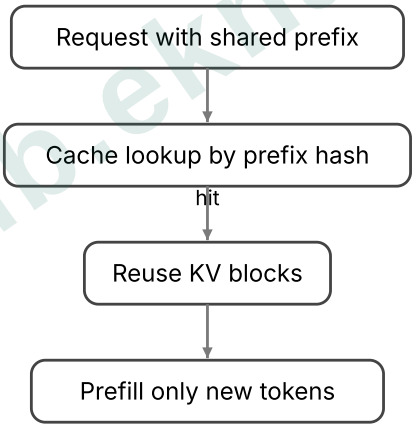
🎙 Spoken answer · ~64 s

Many requests share a long prefix, such as the system prompt, tool definitions or a shared document. Prefix caching stores the computed key-value cache for that prefix, so later requests reuse it instead of repeating the prefill computation. This cuts time to first token and GPU work, particularly for chat with long history and for RAG with repeated context. To benefit, prompts must keep the shared part identical and at the start, and routing should send related requests to the same replica, using cache-aware or session-affinity routing. The KV cache is large and GPU memory limited, so offloading moves less-used blocks to CPU memory or fast storage and brings them back when needed, trading some latency for the ability to hold more context and sessions. The costs are extra transfer time and complexity. I monitor cache hit rate, time to first token, and memory use, and test that eviction does not harm tail latency.

Likely follow-ups

- How does routing affect cache hit rate?
- When does offloading hurt latency?

🗺 Prefix caching



Prefix caching reuses KV cache for shared prompt prefixes, cutting prefill time and cost, if prompts are stable at the start and routing is cache-aware. Offloading moves cold KV blocks to CPU or storage to hold more sessions, at some latency cost.

```
vllm serve model --enable-prefix-caching
```

```
curl -s $VLLM/metrics | grep prefix_cache
```

Interview tip: Say keep shared text identical and at the start of prompts.

📚 Official docs

- vLLM documentation

certs.eknathalabs.com

Q150 How do you load test and capacity-plan an LLM service before launch?

Serving

🎙 **Spoken answer · ~62 s**

I use realistic workloads, because LLM load depends on prompt and output lengths, not only request counts. First, collect or synthesise a request distribution from real or expected traffic: input tokens, output tokens, concurrency, and the share of cached prefixes. Then run a load generator that ramps the request rate in steps and records time to first token, inter-token latency, end-to-end latency percentiles, throughput in tokens per second, error rate, queue depth, GPU utilisation and KV cache usage. The result is a curve showing the maximum rate per replica that still meets the latency SLO, which is the capacity number to plan with, with headroom of perhaps 30 percent for spikes and failures. I also test overload behaviour to confirm load shedding and autoscaling work, test failure of a replica, and repeat after each major model or engine change. Findings feed into autoscaling targets, quotas and the cost model.

Likely follow-ups

- Why test with realistic length distributions?
- What headroom do you keep?

Use realistic prompt and output length distributions, ramp load in steps, record TTFT, inter-token latency, throughput, errors and GPU and KV cache use, find the rate per replica that meets the SLO, add headroom, test overload and failures, and repeat after changes.

```
python benchmark_serving.py --backend vllm --request-rate 4 --num-prompts 500 --dataset-path real.jsonl
```

```
k6 run llm_ramp.js
```

Interview tip: Say capacity is the highest rate that still meets the latency SLO.

📚 **Official docs**

- Google SRE Book: Testing for Reliability

certs.eknathalabs.com
InterviewHub - interviewhub.eknathalabs.com